

zero.dev001's Dev Environment

A Complete Reference for the Terminal-First macOS Setup

zero.dev001

March 2026

Contents

Introduction	23
What This Book Is	23
Who This Book Is For	23
The Unifying Principles	23
Catppuccin Mocha Everywhere	23
Vim-Style Navigation Everywhere	24
How This Book Is Organized	24
Part 1: Foundations	24
Part 2: Navigation	25
Part 3: Editor	25
Part 4: Tmux	25
Part 5: Git	25
Part 6: Window Management	25
Part 7: CLI Toolbox	26
Part 8: Dev Environment	26
Part 9: Workflows	26
Appendix	26
How to Use This Book	26
The Dotfiles	27
Conventions Used in This Book	27
 Shell: Zsh + Starship	 28
Your Setup	28
The .zshrc in Full	28
The starship.toml in Full	29
Core Concepts	31
Why Zsh + Oh My Zsh with No Theme?	31
Plugin Stack	31
Oh My Zsh Settings	32
Editor Configuration	32
Man Pages in Neovim	32
Tool Initialization Order	32
PATH Construction	33
Essential Commands	33
Shell Navigation	33
Environment Variables	33

Practical Recipes	34
Recipe: Understanding the Prompt	34
Recipe: Using Zoxide as cd	35
Recipe: Using Atuin for Shell History	35
Recipe: NVM Auto-Switching	35
Recipe: Checking Tool Initialization	36
Advanced Usage	36
The Chezmoi Template System	36
Starship Module Ordering	36
Disabled Modules	37
Git Status Symbols	37
Language Detection	38
Command Duration	38
Shell Startup Performance	38
Interaction Between Tools	39
macOS Defaults	40
Your Setup	40
Core Concepts	40
How macOS Defaults Work	40
The run_once_ Prefix	41
Essential Commands	41
Keyboard Settings	41
Trackpad and Input	42
Finder	42
Save, Print, and Dialogs	43
Screenshots	43
Dock	44
Window and UI Speed	44
Security	45
Terminal and iTerm2	45
Activity Monitor	46
TextEdit	46
Mac App Store	46
Photos	46
Messages	47
Mail	47
Practical Recipes	47
Recipe: Applying Defaults to a New Machine	47
Recipe: Checking Current Values	47
Recipe: Resetting a Single Setting	48
Recipe: Verifying Key Repeat Settings	48
Advanced Usage	49
Understanding the Sudo Keep-Alive	49
Settings That Require Logout or Restart	49
Compatibility Notes	49
Monitoring Default Changes	50

Navigation: zoxide	51
Your Setup	51
Core Concepts	51
Frecency Algorithm	51
How Matching Works	52
The Database	52
Essential Commands	52
Database Management	52
Practical Recipes	52
Jump to a project by partial name	52
Navigate up then list	53
Fuzzy directory navigation with fed	53
Interactive mode with cdi	53
Check what zoxide would pick	53
Clean up stale entries	53
Seed the database for a new machine	53
Advanced Usage	53
Integration with Other Tools	53
How <code>--cmd cd</code> Changes the Shell	54
Precedence Rules	54
Excluding Directories	54
Environment Variables	54
Debugging Unexpected Jumps	55
The Dot Aliases and Zoxide	55
Fuzzy Finding: fzf	56
Your Setup	56
Core Concepts	56
fzf Is a Filter, Not a Finder	56
The Find-Pick-Act Pattern	56
Fuzzy Matching Syntax	57
Essential Commands	57
Shell Key Bindings	57
Custom Functions	57
fzf Options	58
Practical Recipes	58
Find and edit a file	58
Find and copy a file path	58
Find a directory and navigate to it	58
Multi-select files to open	58
Search git branches and switch	59
Preview files while searching	59
Filter process list and kill	59
Browse and select from any command output	59
Pipe ripgrep results through fzf	59
Advanced Usage	59
fzf as a UI Component in Other Tools	59
fzf-tmux	59

Customizing fzf with Environment Variables	60
Composing Custom Pickers	60
Key Bindings Inside fzf	60
The Mental Model	60
Search: fd and ripgrep	61
Your Setup	61
Core Concepts	61
fd: File Finding	61
ripgrep: Content Searching	61
Comparison Tables	62
fd vs find	62
rg vs grep	62
Essential Commands: fd	62
Essential Commands: ripgrep	63
Practical Recipes	63
Find all TypeScript files in a project	63
Find large files	63
Find recently modified files	64
Delete all .DS_Store files	64
Search for a function definition	64
Search and replace across files	64
Find TODOs with context	64
Search only in specific directories	64
Exclude test files from search	64
Count occurrences by file	64
Pipe fd to fzf to act (the standard pattern)	64
Pipe rg to fzf for interactive grep	65
Search for files containing two different patterns	65
List all file types ripgrep knows	65
Advanced Usage	65
Integration Map	65
fd Exec Patterns	65
ripgrep Configuration File	66
Performance Tips	66
Combining fd and rg	66
Using with git	66
File Manager: Yazi	67
Your Setup	67
Core Concepts	67
The Three-Pane Layout	67
Vim-Style Modes	68
The Two Quit Commands	68
Essential Commands	68
Movement	68
Selection	68
File Operations	69

Search and Find	69
Goto Shortcuts	70
Tabs	70
Copy Paths	70
Sorting	70
Linemode (Metadata Display)	71
Other	71
Practical Recipes	71
Browse and land in a directory	71
Quick file preview without opening	71
Bulk rename files	71
Move files between directories using tabs	71
Search for a file deep in the tree	72
Search file contents	72
Jump to a frequently visited directory	72
Jump using fzf	72
Filter the current directory	72
Run a shell command on selected files	72
Copy a file path to clipboard	72
Check file sizes in a directory	72
Advanced Usage	73
The Shell Wrapper in Detail	73
Yazi's Search Tools	73
Task Manager	73
Opener Configuration	73
Spot Mode	74
Input Mode (Rename, Create, Shell)	74
Configuration Highlights	74
Neovim Fundamentals	76
Your Setup	76
Core Concepts	76
Modal Editing	76
The Insert Mode Entry Points	77
Essential Commands	77
Movement – Characters and Lines	77
Movement – Words	77
Movement – Vertical	78
Movement – Find on Line	78
Operators	78
Text Objects	79
Practical Recipes	79
The Vim Grammar in Action	79
Counts as Multipliers	80
Registers	80
Marks	81
Macros	81
Advanced Usage	82

Search and Substitution	82
Visual Mode	82
The Dot Command	83
Undo and Redo	83
Window Splits	83
Command-Line Mode Essentials	83
LazyVim Distribution	85
Your Setup	85
Core Concepts	86
The Leader Key	86
lazy.nvim Package Manager	86
The Extras System	86
Essential Commands	87
File Finding (Telescope)	87
File Explorer (neo-tree)	87
Buffer Management	88
Search and Replace	88
Navigation	88
Git Integration	89
UI Toggles	89
Practical Recipes	89
Quick File Navigation Workflow	89
Project-Wide Search and Replace	90
Flash Jump	90
Managing the Plugin System	90
Overriding LazyVim Defaults	90
Advanced Usage	91
The Plugin Spec Format	91
Performance Notes	91
Session Management	91
Startup Sequence	92
Custom Plugins	93
Your Setup	93
Core Concepts	93
Plugin Loading Strategies	93
Local Plugins	94
Essential Commands	94
99.nvim – AI Integration	94
auto-save.nvim – Automatic Saving	95
conform.nvim – Code Formatting	95
dadbod – Database UI	95
hardtime.nvim – Vim Habit Trainer	96
img-clip.nvim – Image Clipboard Paste	96
stay-centered.nvim – Cursor Centering	97
vim-visual-multi – Multiple Cursors	97
Practical Recipes	98

The Auto-Save and Format Pipeline	98
Database Exploration with Dadbod	98
Multi-Cursor Editing Patterns	98
Advanced Usage	99
Disabling Plugins Temporarily	99
Conform Formatter Selection	99
Plugin Update Workflow	99
Writing Your Own Plugin Spec	99
LSP and Treesitter	101
Your Setup	101
Core Concepts	101
What Is LSP?	101
What Is Treesitter?	102
How They Work Together	102
Essential Commands	102
LSP Navigation	102
LSP Code Actions	103
Diagnostics	103
Mason Package Manager	103
Treesitter	104
Practical Recipes	104
Navigating a Codebase with LSP	104
Rename a Symbol Safely	104
Understanding Diagnostics	105
Incremental Selection with Treesitter	105
Inspecting the Syntax Tree	105
Advanced Usage	105
Common Language Servers	105
Mason-Managed Tools	106
Treesitter Text Objects	106
Treesitter Navigation	107
Diagnostics Severity Configuration	107
Troubleshooting LSP Issues	107
Performance Considerations	107
AI Integration: Claude and 99.nvim	108
Your Setup	108
Core Concepts	108
99.nvim Architecture	108
Claude Code in the Terminal	109
The IDE Layout	109
Essential Commands	110
99.nvim Key Mappings	110
IDE Layout Commands	110
Claude Code Terminal Commands	110
Practical Recipes	111
Using 99.nvim for Code Transformation	111

Switching AI Models	111
Stopping a Runaway Request	111
The Complete IDE Workflow	111
AI-Assisted Debugging	112
AI-Assisted Code Review	112
Advanced Usage	112
Developing 99.nvim	112
Provider System	113
Optimizing the IDE Layout	113
Contextual AI Usage Patterns	113
Security Considerations	114
Tmux Core	115
Your Setup	115
Core Concepts	116
The Tmux Hierarchy: Sessions, Windows, and Panes	116
The Prefix Key	116
Detach and Attach	116
Naming Conventions	116
Essential Commands	117
Starting and Managing Sessions	117
Window Management	117
Pane Management	117
Pane Navigation	117
Copy Mode	118
Practical Recipes	118
Start a named session and split into a working layout	118
Quickly switch between two windows	118
Zoom a pane for full-screen focus	119
Reorder windows after closing one	119
Copy text from terminal output	119
Move a pane to a different window	119
Resize panes	119
Advanced Usage	119
Why escape-time Is Zero	119
The 1-Million-Line Scrollback	119
Status Bar Position	120
Set-Titles for Terminal Tab Names	120
Base Index 1	120
Detach-on-Destroy Off	120
Mouse Support	120
Clipboard Integration	120
Tmux Plugins	121
Your Setup	121
Core Concepts	122
TPM: The Plugin Manager	122
Plugin Categories	122

Why vim-tmux-navigator Is Critical	122
Essential Commands	122
TPM Management	122
vim-tmux-navigator	122
Session Persistence (resurrect + continuum)	123
Session Picker (sessionx)	123
Floating Terminal (floax)	123
URL and Pattern Extraction	123
Clipboard (tmux-yank)	124
Practical Recipes	124
First-time plugin installation	124
Seamless Vim-Tmux navigation workflow	124
Surviving a reboot with resurrect and continuum	124
Quick session switching with sessionx	125
Pop open a floating terminal	125
Grab a URL from terminal output	125
Quick-copy a hash, path, or IP with thumbs	125
Advanced Usage	125
How vim-tmux-navigator Detects the Active Program	125
Resurrect Save Strategy for Neovim	125
Continuum Save Interval	126
Catppuccin Theme Customization	126
sessionx vs. sesh	126
floax Working Directory	126
fzf-url History Limit	126
Plugin Load Order	126
Troubleshooting Plugins	127
Session Management: sesh	128
Your Setup	128
Core Concepts	129
What sesh Does	129
Named Session Anatomy	129
The Picker Modes	129
sesh vs. sessionx	130
Essential Commands	130
Launching the Picker	130
Inside the sesh Picker	130
sesh CLI Commands	131
Practical Recipes	131
Jump to your dotfiles from anywhere	131
Edit tmux config quickly	131
Start a session for a project you recently visited	131
Discover and open a new project	131
Kill a session you no longer need	132
Use sesh from the command line	132
Advanced Usage	132
How sesh connect Works	132

Preview Commands	132
The fd-Based Find Mode	133
Session Naming from Zoxide Directories	133
Combining sesh with the IDE Layout	133
Startup Command Limitations	133
Customizing the Picker Appearance	134
When to Use sesh vs. tmux Directly	134
IDE Layout	135
Your Setup	135
Core Concepts	136
Why This Layout	136
How tmuxinator Works	136
The ide() Function: Why Not Just tmuxinator?	136
Pane Addressing	137
Essential Commands	137
Launching the IDE	137
Navigating Between Panes	137
Managing the Layout	137
Window Utilities	138
Practical Recipes	138
Start a new project session	138
The AI-assisted development loop	138
Zoom a pane for focused work	138
Run a long process in the terminal pane	139
Use the floating terminal for quick commands	139
Add a fourth pane	139
Open multiple IDE windows	139
Rename the window for clarity	139
Advanced Usage	140
Why -no-attach Is Essential	140
Pane Size Percentages	140
The main-vertical Layout	140
Customizing the Layout for Different Workflows	140
Session Persistence with the IDE Layout	141
Integration with sesh	141
Troubleshooting the Layout	141
Extending the ide() Function	141
The tab() Helper	142
Git Workflow	143
Your Setup	143
Core Concepts	144
The Commit Cycle	144
Branching Model	144
The diff3 Conflict Style	144
The Post-Commit Backup Hook	145
Essential Commands	145

Daily Workflow (Oh My Zsh Aliases)	145
Branching	145
Viewing History	145
Stashing	146
Practical Recipes	146
Start a feature branch	146
The full commit cycle	146
Quick commit everything	146
Stash work in progress	146
Rebase onto main	146
Resolve a merge conflict with diff3	147
Interactive rebase to clean up commits	147
View a specific file's history	147
Blame with delta coloring	147
Advanced Usage	148
The Post-Commit Hook in Detail	148
Overriding the Hook Per-Repository	148
colorMoved and Code Refactoring	148
Combining Aliases for Complex Workflows	148
Excluding Files Globally	148
Safe Directory for Homebrew	149
Beautiful Diffs: delta	150
Your Setup	150
Core Concepts	151
What Delta Replaces	151
Side-by-Side Mode	151
Navigation with n/N	151
Syntax Themes	152
Essential Commands	152
Commands That Automatically Use Delta	152
Delta-Specific Navigation (Inside the Pager)	152
Overriding Delta Temporarily	153
Practical Recipes	153
Review a feature branch diff	153
Review a single commit	153
Compare two commits	153
Review what you are about to commit	153
View a file's history with diffs	153
Blame with syntax highlighting	153
View stash contents before applying	154
Diff only specific file types	154
Advanced Usage	154
How Delta Integrates with the Pager	154
The Interactive Diff Filter	154
Delta and colorMoved	154
The diffnav Tool	154
Performance Considerations	155

Using Delta Outside of Git	155
Delta Configuration Options Not in This Setup	155
Catppuccin Consistency	155
Git TUI: lazygit	157
Your Setup	157
Core Concepts	157
The Five Panels	157
Staging Model	158
Navigation Philosophy	158
Essential Commands	158
Global Keys	158
Files Panel	158
Inside a File (Hunk View)	159
Branches Panel	159
Commits Panel	159
Push and Pull	159
Stash Panel	160
Practical Recipes	160
Stage specific hunks from a file	160
Stage specific lines within a hunk	160
Interactive rebase to clean history	160
Resolve merge conflicts	161
Cherry-pick a commit from another branch	161
Compare two branches visually	161
Undo a commit (soft reset)	161
Force push safely	161
Advanced Usage	162
The Command Log	162
Delta Integration	162
Lazygit in Neovim	162
Custom Keybindings	162
Filtering and Searching	162
Performance on Large Repositories	163
GitHub CLI and gh-dash	164
Your Setup	164
Core Concepts	164
What gh Replaces	164
The gh Extension System	165
gh-dash: The Dashboard	165
Essential Commands	165
Pull Request Workflow	165
Issue Workflow	165
Repository Operations	166
GitHub Actions	166
gh-dash Keys	166
Practical Recipes	167

Create a PR from a feature branch	167
Review and merge a teammate's PR	167
Monitor a CI run	167
Create an issue from the terminal	168
Quick open in browser	168
Clone a repo by name	168
Search for a PR	168
View another repo's PRs	168
Advanced Usage	168
gh API for Custom Queries	168
gh-dash Configuration	169
Aliases for Common gh Commands	169
Using gh in Scripts	170
gh with Multiple GitHub Accounts	170
Window Management: Aerospace	171
Your Setup	171
Core Concepts	172
Tiling vs. Floating	172
Workspaces	173
Container Layouts	173
Container Orientation	173
Normalizations	173
Mouse Follows Focus	174
Essential Commands	174
Main Mode Bindings	174
Focus (Navigate Between Windows)	174
Move Windows	174
Layout	174
Resize	174
Workspaces	175
Move Window to Workspace	175
Monitor Management	175
Mode Switch	175
Service Mode Bindings	175
Practical Recipes	176
Typical development layout	176
Move a misplaced window	176
Toggle a window to floating	176
Reorganize a cluttered workspace	176
Full-screen focus	176
Move workspace to a different monitor	176
Quick switch between two windows	177
Advanced Usage	177
Understanding the Workspace-to-Monitor Assignment	177
The Outer Top Gap by Monitor	177
Normalization Behavior	178
Join Operations in Service Mode	178

Aerospace CLI	178
When Not to Tile	178
Aerospace vs. Alternatives	178
Modern Replacements: bat and eza	179
Your Setup	179
Core Concepts	179
bat: A cat Clone with Wings	179
eza: A Modern ls	180
Essential Commands – bat	180
Essential Commands – eza	180
Practical Recipes	181
Preview files with fzf using bat	181
Quickly check git changes in a directory	181
Use bat as a MANPAGER	181
Coloured diff between two files	181
Explore a project structure before diving in	181
Tail a log file with syntax highlighting	181
Display all supported bat themes with a sample	182
Advanced Usage	182
bat Configuration File	182
Building Custom bat Syntaxes	182
eza Colour and Icon Customization	182
Combining bat and eza in Scripts	182
Using bat with git	182
HTTP Tools: curl and wget	183
Your Setup	183
Core Concepts	183
curl – Transfer Data with URLs	183
wget – Non-Interactive Network Downloader	183
Essential Commands – curl	183
Essential Commands – wget	184
Practical Recipes	184
GET a JSON API and pretty-print with jq	184
POST JSON to an API	185
Upload a file with a multipart form	185
Check if a URL is reachable	185
Measure response time	185
Download with a progress bar	185
Mirror a website for offline reading	185
Download all files of a type from a page	185
Retry a flaky download	185
Advanced Usage	186
curl with a Configuration File	186
Sending Multiple curl Requests in Parallel	186
Inspecting and Debugging TLS	186
Scripting Patterns with curl	186

When to Use Which Tool	186
JSON Processing: jq	188
Your Setup	188
Core Concepts	188
Type System	188
Essential Commands	188
Filter Reference	189
Selection and Filtering	189
Transformation	189
Construction and String Handling	190
Conditionals and Reduction	190
Practical Recipes	190
Pretty-print a curl response	190
Extract specific fields from API data	190
Filter an array of objects	190
Transform a list of objects into a summary	191
Create CSV output from JSON	191
Count items by category	191
Merge two JSON files	191
Update a nested value	191
Process NDJSON (newline-delimited JSON)	191
Use shell variables inside jq	191
Advanced Usage	191
Building Complex Pipelines	191
Defining Functions	192
The <code>env</code> Object	192
Error Handling with <code>try-catch</code>	192
Working with gh CLI Output	192
Text Processing: grep, sed, and awk	193
Your Setup	193
Core Concepts	193
grep – Global Regular Expression Print	193
sed – Stream Editor	193
awk – Pattern-Action Language	193
Essential Commands – grep	194
Essential Commands – sed	194
Essential Commands – awk	194
Practical Recipes	195
Find TODOs across a codebase	195
Replace a string across multiple files	195
Extract email addresses from a file	195
Remove blank lines from a file	195
Print a specific field from a CSV	195
Sum a column of numbers	196
Extract text between two markers	196
Add a header line to output	196

Count word frequency	196
Replace only on lines matching a condition	196
Transpose rows and columns	196
Advanced Usage	196
Combining grep, sed, and awk in Pipelines	196
awk as a Mini Programming Language	196
sed with Regular Expression Groups	197
Multi-line Processing with sed	197
The -i Flag Portability Problem	197
When to Reach for Each Tool	197
Media and Documents: pandoc, ffmpeg, and Friends	198
Your Setup	198
Core Concepts	198
pandoc – Universal Document Converter	198
ffmpeg – The Swiss Army Knife of Media	199
yt-dlp – Media Downloader	199
aria2 – High-Speed Downloader	199
imagemagick – Image Processing	199
Essential Commands – pandoc	199
Essential Commands – ffmpeg	199
Essential Commands – yt-dlp	200
Essential Commands – aria2	200
Essential Commands – imagemagick	201
Practical Recipes	201
Build a PDF book from multiple Markdown files	201
Create a high-quality GIF from a screen recording	201
Download a playlist as MP3 files	201
Batch convert images for web	202
Create a contact sheet from video	202
Fast download with aria2 (16 connections)	202
Advanced Usage	202
pandoc Metadata and YAML Front Matter	202
ffmpeg Filter Chains	202
Automating with a Makefile	202
ImageMagick for Screenshots	203
System Monitoring: btop, fastfetch, and Friends	204
Your Setup	204
Core Concepts	205
btop – Interactive Resource Monitor	205
fastfetch – System Information Display	205
Stats – Menu Bar Monitor	205
Essential Commands – btop	205
btop Configuration	205
Essential Commands – fastfetch	206
Shell Functions and Aliases	206
fs – File and Directory Sizes	206

gz – Gzip Compression Ratio	207
server – Quick HTTP Server	207
Networking Aliases	207
Practical Recipes	207
Diagnose high CPU usage	207
Check disk space across all volumes	208
Monitor network connections	208
Quick system overview for troubleshooting	208
Find what is eating disk space	208
Check if a port is in use	208
Kill a process by name	208
Advanced Usage	209
btop Themes	209
Scripting System Checks	209
Stats Menu Bar Customization	209
Maintenance Aliases	209
Shell History: atuin	211
Your Setup	211
Core Concepts	211
What atuin Replaces	211
How It Works	212
Sync Across Machines	212
Essential Commands	212
atuin CLI Commands	212
Configuration Reference	213
Additional Configuration Options	213
Practical Recipes	214
Find that docker command you ran last week	214
Search by directory	214
Find commands that failed	214
View your most-used commands	214
Import existing history	214
Exclude sensitive commands from history	214
Advanced Usage	215
Search Modes	215
Filter Modes	215
Syncing Across Machines	215
atuin and the Shell Initialization Order	216
Up Arrow Integration	216
Node.js: NVM and Bun	217
Overview	217
NVM Configuration	217
Automatic Installation	217
Essential NVM Commands	218
Per-Project Version Pinning	218
Bun Configuration	218

Essential Bun Commands	218
Why Bun Alongside Node	219
How They Interact	219
Chezmoi Integration	220
Troubleshooting	220
Quick Reference	220
Version Management: mise	222
Overview	222
Installation and Activation	222
Configuration Files	223
.tool-versions (asdf-compatible)	223
mise.toml (mise-native)	223
Global Configuration	223
Essential Commands	223
Installing and Using Tools	223
Querying State	224
Managing Configuration	224
How mise Resolves Versions	224
mise vs. NVM	225
Practical Workflows	225
Setting Up a Python Project	225
Setting Up a Multi-Language Project	225
Checking What Is Active	225
Updating a Tool	226
Plugins	226
Troubleshooting	226
Quick Reference	226
Dotfiles Management: chezmoi	228
Overview	228
Core Concepts	228
Source State vs. Target State	228
Naming Conventions	228
Essential Commands	229
Daily Usage	229
Managing Files	229
Inspection	229
Templates	230
Example: dot_zshrc.tmpl	230
Template Functions	230
Run Scripts	231
Scripts in This Environment	231
Auto-Sync with LaunchAgent	231
Workflow: Making a Configuration Change	232
Bootstrapping a New Machine	232
Troubleshooting	233

Package Management: Brewfiles	234
Overview	234
Tier 0: Base (00-base.Brewfile)	234
Installing Base Only	235
Tier 1: Essential (10-essential.Brewfile)	235
CLI Tools	235
GUI Applications (Casks)	236
Installing Essentials	236
Tier 2: Nice-to-Have (15-nice-to-have.Brewfile)	236
CLI Tools	236
GUI Applications (Casks)	236
Installing Nice-to-Have	237
Tier 3: Optional (20-optional.Brewfile)	237
CLI Tools	237
GUI Applications (Casks)	237
VS Code Extensions	238
UV Tools	238
Installing Optional	238
The Master Brewfile	238
How chezmoi Triggers Brew Bundle	238
Selective Installation	239
Quick Reference	239
 Workflow: Daily Startup	 240
Overview	240
Step 1: Open the Terminal	240
Step 2: Pick or Create a Project Session	240
Option A: Sesh (prefix + T)	241
Option B: SessionX (prefix + o)	241
Pre-Defined Sessions	241
Step 3: Launch the IDE Layout	241
Step 4: Navigate Between Projects	242
Step 5: Sessions Persist Automatically	242
Step 6: Workspace Organization with Aerospace	243
The Full Sequence	243
When Things Go Wrong	244
Quick Reference	244
 Workflow: Starting a New Project	 245
Overview	245
Step 1: Create the Project Directory	245
Step 2: Initialize the Project	245
Node.js Project with Bun	245
Node.js Project with npm	246
Python Project	246
General Project (No Runtime)	246
Step 3: Initialize Git	246
Step 4: Launch the IDE Layout	247

Step 5: Register with Sesh (Optional)	247
Step 6: Start Coding in Neovim	247
Finding Files	248
File Explorer	248
Creating New Files	248
Step 7: Git Workflow	248
Local Development	248
Creating a Pull Request	248
Scaffold Cheat Sheet	249
The Full Sequence	249
Tips	250
Troubleshooting	250
Workflow: Code Review	251
Overview	251
Step 1: Checkout the Pull Request	251
Step 2: View the Overall Diff	252
Step 3: Read the Diff with Delta	252
Step 4: Explore the Code in Neovim	253
Finding Changed Code	253
Understanding Code	253
Reviewing Specific Changes	254
Step 5: Explore Commits in Lazygit	254
Step 6: Submit the Review	254
Approve	254
Request Changes	254
Comment Only	254
Adding Inline Comments	255
Step 7: Merge When Approved	255
Reviewing Multiple PRs	255
The Full Sequence	255
Troubleshooting	256
Workflow: Debugging with AI	257
Overview	257
The IDE Layout	257
Step 1: Reproduce the Bug	258
Step 2: Navigate to the Problem in Neovim	258
Step 3: Ask 99.nvim About Specific Code	259
99.nvim Keybindings	259
Step 4: Ask Claude Code for Broader Context	259
Step 5: Navigate Between Panes Seamlessly	260
Step 6: Apply the Fix	260
Step 7: Verify and Commit	260
AI Debugging Patterns	261
Pattern: Rubber Duck with Claude Code	261
Pattern: Targeted Selection with 99.nvim	261
Pattern: Stack Trace Analysis	261

Pattern: Comparison	261
When to Use Which AI Tool	261
Troubleshooting	262
Appendix A: All Keybindings	263
Tmux: Prefix Commands	263
Session Management	263
Window Management	263
Pane Management	263
Copy Mode (vi-style)	264
Plugin Commands	264
Sesh Picker Sub-Commands	264
Tmux: No-Prefix Commands	265
Pane Navigation (Alt + Arrow)	265
Window Navigation	265
Vim-Tmux-Navigator (Seamless Pane/Split Navigation)	265
Neovim: Leader Commands	265
Files and Buffers	265
Search	266
Code Actions and Diagnostics	266
Git (from Neovim)	266
AI (99.nvim)	266
Plugins	267
Neovim: LSP Commands	267
Neovim: General Navigation	267
Aerospace: Main Mode	268
Aerospace: Service Mode	268
Yazi Navigation	269
Movement	269
File Operations	269
Search and Find	270
Goto Shortcuts	270
Other	270
fzf Shell Bindings	270
Inside an fzf Picker	271
Lazygit	271
Atuin (Shell History)	271
Appendix B: All Aliases and Functions	272
Editor Aliases	272
Usage	272
Miscellaneous Aliases	273
Usage	273
File Listing Aliases (eza)	273
Usage	273
Navigation Shortcuts	273
Usage	274
Navigation Functions	274

cx() – Change Directory and List	274
fcd() – Fuzzy Directory Picker	274
f() – Fuzzy File to Clipboard	274
fv() – Fuzzy File to Neovim	275
IDE Layout Function	275
ide() – Tmuxinator IDE Layout	275
Tab Rename Function	275
tab() – Rename Tmux Window or Terminal Tab	275
Yazi Wrapper Function	276
y() – Yazi with CWD Sync	276
Networking Aliases	276
Usage	276
Cleanup and Maintenance Aliases	277
Usage	277
macOS Toggle Aliases	277
Expansions	277
Utility Functions	277
mkd() – Make Directory and Enter	277
server() – Quick HTTP Server	278
fs() – File Sizes	278
gz() – Gzip Comparison	278
Complete Alias Quick-Scan Table	278
Complete Function Quick-Scan Table	279
Oh My Zsh Git Aliases	279
Environment Variables	280
Zsh Plugins	280
Appendix C: Quick Reference Card	281
Finding Things	281
Editing	281
Navigating	282
Git	282
Terminal Management (tmux)	283
System	284
File Operations	284
Package Management	285
Runtime Versions	285
Dotfiles (chezmoi)	286
Workspace Layout (Aerospace)	286
IDE Pane Layout	286

Introduction

A terminal-first macOS development environment, unified by Catppuccin Mocha and vim-style navigation.

What This Book Is

This is a comprehensive, print-ready reference for zero.dev001's development environment. It is not a tutorial that walks you through installation step by step. It is a field manual – the kind of book you keep on your desk, flip open to a chapter, and find the exact keybinding, command, or configuration snippet you need.

Every tool in this environment was chosen deliberately. Every keybinding was tested against real workflows. Every configuration option was tuned for speed, clarity, and consistency. This book documents all of it.

Who This Book Is For

This book is written for one person: zero.dev001. But it is useful for anyone who wants to understand how a modern, terminal-centric macOS development setup fits together. If you are building your own environment, you will find ideas worth stealing. If you are pair programming with zero.dev001, you will understand why his screen looks and behaves the way it does.

The Unifying Principles

Three principles hold this entire environment together.

Catppuccin Mocha Everywhere

Every tool in this setup uses the Catppuccin Mocha color palette. The terminal, the editor, the prompt, the file manager, the git interface, the system bar – all of them render in the same warm, low-contrast palette. This is not cosmetic vanity. Consistent colors reduce cognitive switching cost. When a filename is cyan in the terminal, it is cyan in Neovim, and it is cyan in Yazi. When a git addition is green in lazygit, it is the same green in delta, and the same green in the Neovim gutter.

The Catppuccin Mocha palette used throughout:

Role	Color Name	Hex	Usage
Rosewater	rosewater	#f5e0dc	Cursor, subtle highlights
Flamingo	flamingo	#f2cdcd	Warnings, paired brackets
Pink	pink	#f5c2e7	Git branches, keywords
Mauve	mauve	#cba6f7	Prompt user, time, vim mode
Red	red	#f38ba8	Errors, deletions
Maroon	maroon	#eba0ac	Secondary errors
Peach	peach	#fab387	Git status, numbers
Yellow	yellow	#f9e2af	Warnings, cmd duration
Green	green	#a6e3a1	Success, additions, hostname
Teal	teal	#94e2d5	Info, secondary highlights
Sky	sky	#89dceb	Directory paths, Go
Sapphire	sapphire	#74c7ec	Links
Blue	blue	#89b4fa	Docker, functions
Lavender	lavender	#b4befe	Headings
Text	text	#cdd6f4	Default foreground
Base	base	#1e1e2e	Background

Vim-Style Navigation Everywhere

The h, j, k, l keys mean the same thing in every context:

Key	Direction	In Neovim	In tmux	In Yazi	In lazygit	In Aerospace
h	Left	Move left	Pane left	Parent dir	Prev panel	Focus left
j	Down	Move down	Pane down	Next item	Next item	Focus down
k	Up	Move up	Pane up	Prev item	Prev item	Focus up
l	Right	Move right	Pane right	Open/enter	Next panel	Focus right

This consistency is not accidental. When you internalize hjkl in one tool, you have internalized it in every tool. The same applies to / for search, q for quit, G for go-to-bottom, and gg for go-to-top.

How This Book Is Organized

The book is divided into nine parts plus an appendix, ordered from the lowest layer of the stack (keyboard input) to the highest (complete workflows).

Part 1: Foundations

Chapters 2–3 cover the base layer that everything else depends on.

- **Chapter 2: Shell: Zsh + Starship** – Oh My Zsh plugins, environment variables, tool initialization order, and the two-line Starship prompt.
- **Chapter 3: macOS Defaults** – The defaults write commands that make macOS behave like a development OS: fast key repeat, no autocorrect, no animations, sensible Finder and Dock settings.

Part 2: Navigation

Chapters 5–8 cover how to move through the filesystem and find things.

- **Chapter 5: Zoxide** – Frequency-based directory jumping that replaces `cd`.
- **Chapter 6: fzf** – Fuzzy finding for files, history, processes, and anything else that produces lines of text.
- **Chapter 7: fd + ripgrep** – Modern replacements for `find` and `grep` that respect `.gitignore` and run in parallel.
- **Chapter 8: Yazi** – Terminal file manager with vim keybindings and image preview.

Part 3: Editor

Chapters 9–13 cover Neovim from fundamentals to AI-assisted coding.

- **Chapter 9: Neovim Fundamentals** – Modes, motions, text objects, registers, and macros.
- **Chapter 10: LazyVim** – The plugin framework, lazy-loading, and the lua/ directory structure.
- **Chapter 11: Plugins Deep Dive** – Telescope, Treesitter, Harpoon, Oil, and the rest of the plugin stack.
- **Chapter 12: LSP and Completion** – Language servers, Mason, `nvim-cmp`, and diagnostics.
- **Chapter 13: AI with 99.nvim** – AI-assisted coding inside Neovim.

Part 4: Tmux

Chapters 14–17 cover terminal multiplexing and session management.

- **Chapter 14: Tmux Core** – Windows, panes, the prefix key, and copy mode.
- **Chapter 15: Tmux Plugins** – TPM, Catppuccin status bar, `vim-tmux-navigator`, and `tmux-resurrect`.
- **Chapter 16: Sesh Sessions** – Fuzzy session management with `sesh` and its integration with `zoxide`.
- **Chapter 17: IDE Layout** – Scripted pane layouts for different project types.

Part 5: Git

Chapters 18–21 cover the complete git workflow.

- **Chapter 18: Git Workflow** – Branching strategy, commit conventions, and rebase workflows.
- **Chapter 19: Delta** – A syntax-highlighting pager for diffs with Catppuccin colors.
- **Chapter 20: Lazygit** – A terminal UI for git that makes interactive rebasing, staging hunks, and resolving conflicts visual.
- **Chapter 21: GitHub CLI** – `gh` for PRs, issues, actions, and `gh dash` for a dashboard.

Part 6: Window Management

- **Chapter 22: Aerospace** – A tiling window manager for macOS with vim-style workspace switching and window movement.

Part 7: CLI Toolbox

Chapters 23–25 cover the modern CLI replacements and utilities.

- **Chapter 23: Modern Replacements** – bat (cat), eza (ls), dust (du),procs (ps), and sd (sed).
- **Chapter 24: Data Processing** – jq, curl/wget, grep/sed/awk, and pandoc/ffmpeg.
- **Chapter 25: System Monitoring** – btop, fastfetch, atuin shell history.

Part 8: Dev Environment

Chapters 26–28 cover language runtimes, dotfile management, and package management.

- **Chapter 26: Node, NVM, and Bun** – JavaScript runtime management and the Bun bundler/runtime.
- **Chapter 27: mise** – Polyglot runtime version management (the successor to asdf).
- **Chapter 28: Chezmoi and Brewfiles** – Dotfile management across machines and tiered Homebrew bundles.

Part 9: Workflows

Chapters 29–32 tie everything together into complete workflows.

- **Chapter 29: Daily Startup** – From opening the laptop to a fully loaded development session.
- **Chapter 30: New Project Setup** – Scaffolding a project, initializing git, setting up the dev environment.
- **Chapter 31: Code Review** – Reviewing PRs with lazygit, delta, and Neovim.
- **Chapter 32: Debugging with AI** – Using AI tools in the terminal and editor to diagnose and fix issues.

Appendix

- **Complete Keybinding Reference** – Every keybinding in every tool, organized by context.
- **Alias Quick Reference** – All shell aliases and what they expand to.
- **Configuration File Index** – Where every config file lives and what it controls.

How to Use This Book

At your desk: Keep it open to the chapter you are learning. The “Essential Commands” tables in each chapter are designed for quick scanning.

When you forget a keybinding: Flip to the appendix. Every keybinding is listed by tool and by action.

When configuring a new machine: Follow Parts 1 and 8 in order. The chezmoi chapter explains how to bootstrap everything from the dotfiles repository.

When onboarding someone: Hand them the book. The Foundations chapters explain the “why” behind every choice, not just the “what.”

The Dotfiles

Every configuration file referenced in this book lives in a chezmoi-managed dotfiles repository. The source of truth is:

```
~/.local/share/chezmoi/
```

Chezmoi templates (files ending in `.tmpl`) allow machine-specific customization while keeping a single source of truth. The actual configuration files are deployed to their standard locations under `~/.config/` and `~/.`

When this book references a config file path like `~/.config/starship.toml`, the chezmoi source is `~/.local/share/chezmoi/dot_config/starship.toml`. Both paths are noted where relevant.

Conventions Used in This Book

Convention	Meaning
Ctrl+a	Hold Ctrl, press a
<leader>	Space key (Neovim leader)
<prefix>	Tmux prefix key (Ctrl+a)
\$ command	Run in a shell
:command	Run in Neovim command mode
Bold text	UI element or important term
monospace	Code, file paths, keybindings
Lines starting with >	Tips or important notes

This environment is a living system. Tools get updated, configurations evolve, and new workflows replace old ones. But the principles – Catppuccin Mocha everywhere, vim-style navigation, and chezmoi-managed dotfiles – remain constant. This book captures the system as it stands today: a fast, ergonomic, visually consistent development environment built for someone who lives in the terminal.

Shell: Zsh + Starship

The shell is where intent becomes action – its prompt should inform, and its configuration should never get in the way.

Your Setup

The shell layer consists of two components: Zsh as the shell (with Oh My Zsh for plugin management) and Starship as the prompt. Configuration lives at:

- **Zsh config:** `~/.zshrc`
- **Chezmoi source:** `~/.local/share/chezmoi/dot_zshrc.tmpl` (templated for machine-specific paths)
- **Starship config:** `~/.config/starship.toml`
- **Chezmoi source:** `~/.local/share/chezmoi/dot_config/starship.toml`
- **Aliases:** `~/.aliases.zsh` (sourced from `.zshrc`)
- **Oh My Zsh path:** `~/.oh-my-zsh`

The .zshrc in Full

```
export PATH=$HOME/bin:$HOME/.local/bin:/usr/local/bin:$PATH

# Oh My Zsh (no theme – Starship handles the prompt)
export ZSH="$HOME/.oh-my-zsh"
ZSH_THEME=""
ENABLE_CORRECTION="true"
DISABLE_UPDATE_PROMPT=true
DISABLE_AUTO_TITLE="true"
plugins=(git zsh-syntax-highlighting zsh-autosuggestions)
source $ZSH/oh-my-zsh.sh

# Editor
if [[ -n $SSH_CONNECTION ]]; then
    export EDITOR='vim'
else
    export EDITOR='nvim'
fi
```

```
# Man pages in Neovim with syntax highlighting
export MANPAGER='nvim +Man!'

# NVM
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"

# PATH additions
[ -f "$HOME/.local/bin/env" ] && . "$HOME/.local/bin/env"
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"
[ -s "$HOME/.bun/_bun" ] && source "$HOME/.bun/_bun"
[ -d "$(brew --prefix)/opt/mysql-client/bin" ] && \
  export PATH="$(brew --prefix)/opt/mysql-client/bin:$PATH"
[ -d "$(brew --prefix)/opt/llvm/bin" ] && \
  export PATH="$(brew --prefix)/opt/llvm/bin:$PATH"

# Tool init
[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh
eval "$(zoxide init zsh --cmd cd)"
eval "$(atuin init zsh)"
eval "$(mise activate zsh)"

# Aliases and shell functions
[ -f ~/.aliases.zsh ] && source ~/.aliases.zsh

# Starship prompt (must be last)
eval "$(starship init zsh)"
```

The starship.toml in Full

```
format = ""
$username\
$hostname\
$time\
$all\
$directory
$character
""

[character]
success_symbol = '[>>>](#a6e3a1 bold)'
error_symbol = '[XXXX](#f38ba8 bold)'
vicmd_symbol = '[<<<<](#cba6f7 bold)'

[battery]
disabled = true
```

```
[gcloud]
disabled = true

[time]
style = '#cba6f7 bold'
disabled = false
format = '[\[$time\]]($style) '
time_format = '%y/%m/%d'

[username]
style_user = '#cba6f7 bold'
style_root = 'white bold'
format = '[$user]($style).@.'
show_always = true

[hostname]
ssh_only = true
format = '(white bold)[$hostname](#a6e3a1 bold)'

[directory]
style = '#89dceb bold'
truncation_length = 0
truncate_to_repo = false

[git_branch]
style = '#f5c2e7 bold'

[git_status]
style = '#fab387 bold'

[nodejs]
style = '#a6e3a1 bold'

[python]
style = '#f9e2af bold'

[rust]
style = '#fab387 bold'

[golang]
style = '#89dceb bold'

[docker_context]
style = '#89b4fa bold'

[cmd_duration]
style = '#f9e2af bold'
```

```
min_time = 2_000

[ruby]
detect_variables = []
detect_files = ['Gemfile', '.ruby-version']
```

Core Concepts

Why Zsh + Oh My Zsh with No Theme?

Oh My Zsh provides two things: a plugin system and themes. This setup uses it only for the plugin system. The `ZSH_THEME=""` setting disables Oh My Zsh's prompt entirely, leaving prompt rendering to Starship.

This separation gives the best of both worlds:

- **Oh My Zsh plugins** provide git aliases, syntax highlighting, and autosuggestions without needing to install and configure each one manually.
- **Starship** provides a fast, highly configurable prompt that renders in Rust and works across shells (Zsh, Bash, Fish, PowerShell).

Plugin Stack

The three plugins loaded by Oh My Zsh:

git – Provides approximately 150 git aliases. The most frequently used:

Alias	Expansion	Purpose
gst	git status	Check working tree
ga	git add	Stage files
gc	git commit	Commit
gco	git checkout	Switch branches
gp	git push	Push to remote
gl	git pull	Pull from remote
gd	git diff	View diff
glog	git log --oneline --decorate --graph	Visual log
gb	git branch	List branches
grb	git rebase	Rebase
gsta	git stash push	Stash changes
gstp	git stash pop	Pop stash

zsh-syntax-highlighting – Highlights commands as you type them. Valid commands appear green, invalid commands appear red, strings appear in quotes highlighting, and paths that exist are underlined. This provides instant feedback about typos before you press Enter.

zsh-autosuggestions – Shows a ghosted suggestion of the most recent matching command from history as you type. Press the right arrow key (or End) to accept the full suggestion, or press Ctrl+Right to accept one word at a time.

Oh My Zsh Settings

Setting	Value	Purpose
ZSH_THEME	""	Disable Oh My Zsh theme (Starship instead)
ENABLE_CORRECTION	true	Suggest corrections for mistyped commands
DISABLE_UPDATE_PROMPT	true	Auto-update without asking
DISABLE_AUTO_TITLE	true	Do not set terminal title (tmux handles it)

The `DISABLE_AUTO_TITLE` setting is important when using `tmux`. Without it, Oh My Zsh would set the terminal title on every command, which conflicts with `tmux`'s own title management and can cause flickering in the `tmux` status bar.

Editor Configuration

```
if [[ -n $SSH_CONNECTION ]]; then
  export EDITOR='vim'
else
  export EDITOR='nvim'
fi
```

When working locally, the default editor is Neovim. When connected via SSH, it falls back to Vim, which is more likely to be available on remote servers without a full Neovim installation.

The `EDITOR` variable is used by many tools: `git` (for commit messages, interactive rebase), `tmux` (for copy mode editing), `fc` (for editing the last command), and any CLI that opens an editor.

Man Pages in Neovim

```
export MANPAGER='nvim +Man!'
```

This replaces the default `less` man page viewer with Neovim. The `+Man!` argument tells Neovim to activate its built-in Man plugin, which provides:

- Syntax highlighting for man pages
- Clickable cross-references (press `K` on a referenced command to jump to its man page)
- Full Neovim navigation (search with `/`, jump with `gg/G`, etc.)
- Catppuccin Mocha colors applied to man page content

Tool Initialization Order

The order of `eval` statements in `.zshrc` matters. Each tool that hooks into the shell must be initialized in the correct sequence:

1. Oh My Zsh — loads first, provides base shell config and plugins
2. `fzf` — fuzzy finder shell integration (`Ctrl+R`, `Ctrl+T`, `Alt+C`)
3. `zoxide` — replaces `cd` with frequency-based jumping
4. `atuin` — replaces shell history with a searchable database
5. `mise` — activates language runtime version management
6. `aliases` — loaded after all tools so aliases can reference them
7. Starship — MUST be last; it wraps the prompt function

Starship must be initialized last because it replaces the `precmd` and `preexec` hook functions. If another tool initializes after Starship, it might overwrite these hooks and break the prompt.

PATH Construction

The PATH is built up in layers:

<code>\$HOME/bin</code>	— personal scripts
<code>\$HOME/.local/bin</code>	— locally installed binaries
<code>/usr/local/bin</code>	— Homebrew (Intel) / system
<code>\$BUN_INSTALL/bin</code>	— Bun runtime
<code>\$(brew --prefix)/opt/mysql-client/bin</code>	— MySQL client (if installed)
<code>\$(brew --prefix)/opt/llvm/bin</code>	— LLVM toolchain (if installed)

The `chezmoi` template (`dot_zshrc.tmpl`) allows machine-specific PATH entries:

```
{{ if dig "projectBinPath" "" . }}
export PATH="$HOME/{{ .projectBinPath }}:$PATH"
{{ end }}
```

This conditionally adds a project-specific bin directory only on machines where `projectBinPath` is defined in the `chezmoi` config.

Essential Commands

Shell Navigation

Action	Keybinding / Command	Notes
Accept autosuggestion	Right Arrow or End	Completes ghosted suggestion
Accept one word	Ctrl+Right	Partial autosuggestion accept
Search history (atuin)	Ctrl+R	Opens atuin TUI
Fuzzy find file	Ctrl+T	fzf file finder
Fuzzy cd into directory	Alt+C	fzf directory finder
Clear screen	Ctrl+L	Keeps scrollbar
Cancel current line	Ctrl+C	Aborts current input
Move to beginning of line	Ctrl+A	Home
Move to end of line	Ctrl+E	End
Delete word backward	Ctrl+W	Kills word before cursor
Delete to beginning of line	Ctrl+U	Kills to start
Delete to end of line	Ctrl+K	Kills to end
Undo	Ctrl+Z	Undo last edit
Spell correction	Automatic	Suggests Did you mean...?

Environment Variables

Variable	Value	Used By
EDITOR	<code>nvim</code> (local) / <code>vim</code> (SSH)	git, tmux, fc, etc.

Variable	Value	Used By
MANPAGER	nvim +Man!	man command
ZSH	\$HOME/.oh-my-zsh	Oh My Zsh framework
NVM_DIR	\$HOME/.nvm	Node Version Manager
BUN_INSTALL	\$HOME/.bun	Bun JavaScript runtime

Practical Recipes

Recipe: Understanding the Prompt

The Starship prompt renders on two lines. Here is what a typical prompt looks like and what each part means:

```
zero.@[26/03/11] main *!? v20.11.0 via v3.12.0 47s
~/projects/my-app
>>>>
```

Breaking this down piece by piece:

Segment	Config Section	Color (Catppuccin)	Meaning
zero	[username]	Mauve (#cba6f7)	Current user
.@.	[username]	Mauve	Separator (hostname omitted locally)
[26/03/11]	[time]	Mauve (#cba6f7)	Date: year/month/day
main	[git_branch]	Pink (#f5c2e7)	Git branch name
*!?	[git_status]	Peach (#fab387)	Modified, staged, untracked
v20.11.0	[nodejs]	Green (#a6e3a1)	Node.js version (if detected)
via v3.12.0	[python]	Yellow (#f9e2af)	Python version (if detected)
47s	[cmd_duration]	Yellow (#f9e2af)	Last command took 47 seconds
~/projects/my-app	[directory]	Sky (#89dceb)	Full path, no truncation
>>>>	[character]	Green (#a6e3a1)	Success (last exit code was 0)

When the last command fails (non-zero exit code):

```
zero.@[26/03/11]
~/projects/my-app
XXXX
```

The >>>> changes to XXXX in red (#f38ba8), making errors immediately visible.

When in Zsh vi command mode (if enabled):

```
zero.@[26/03/11]
~/projects/my-app
<<<<
```

The <<<< in mauve (#cba6f7) indicates vi normal mode.

Recipe: Using Zoxide as cd

Zoxide is initialized with `--cmd cd`, which means it replaces the built-in `cd` command:

```
# Traditional cd still works for explicit paths
cd /usr/local/bin
cd ..
cd ~

# But now cd also supports frequency jumping
cd projects      # Jumps to most frequently/recently visited "projects" dir
cd my-app        # Jumps to the best match for "my-app"
cd proj app      # Multiple arguments narrow the search: matches "projects/my-app"

# Interactive selection when multiple matches exist
cdi              # Opens fzf to select from matching directories
```

Zoxide tracks every directory you visit and ranks them by frequency and recency (frecency). Over time, short fragments like `cd app` reliably jump to the right directory without typing the full path.

Recipe: Using Atuin for Shell History

Atuin replaces the default Zsh history with a SQLite database that supports:

```
# Search history interactively
Ctrl+R          # Opens the atuin TUI

# In the atuin TUI:
# - Type to filter commands
# - Up/Down to navigate results
# - Enter to select and execute
# - Esc to cancel

# History is synchronized across machines (if configured)
# History includes: command, exit code, duration, directory, hostname
```

Recipe: NVM Auto-Switching

NVM (Node Version Manager) is loaded early in `.zshrc`. Combined with a `.nvmrc` file in a project directory:

```
# In your project directory, create .nvmrc:
echo "20" > .nvmrc

# NVM can be configured to auto-switch on cd:
# (add to .zshrc after NVM init)
autoload -U add-zsh-hook
load-nvmrc() {
  local nvmrc_path="$(nvm_find_nvmrc)"
  if [ -n "$nvmrc_path" ]; then
```

```

    local nvmrc_node_version=$(nvm version "$(cat "${nvmrc_path}")")
    if [ "$nvmrc_node_version" = "N/A" ]; then
        nvm install
    elif [ "$nvmrc_node_version" != "$(nvm version)" ]; then
        nvm use
    fi
fi
}
add-zsh-hook chpwd load-nvmrc
load-nvmrc

```

Recipe: Checking Tool Initialization

To verify all tools are properly initialized:

```

# Check each tool
which nvim           # Should point to Neovim
echo $EDITOR         # Should be 'nvim'
which zoxide         # Should be in PATH
which atuin          # Should be in PATH
which starship       # Should be in PATH
mise --version       # Should print version
bun --version        # Should print version
nvm --version        # Should print version
fzf --version        # Should print version

```

Advanced Usage

The Chezmoi Template System

The `.zshrc` is managed as a chezmoi template (`dot_zshrc.tmpl`). This allows machine-specific customization:

```

{{ if dig "projectBinPath" "" . }}
export PATH="$HOME/{{ .projectBinPath }}:$PATH"
{{ end }}

```

The `dig` function safely accesses nested chezmoi data. If `projectBinPath` is defined in `~/.config/chezmoi/chezmoi.toml`, the `PATH` entry is included. If not, it is silently omitted.

To add machine-specific configuration:

```

# ~/.config/chezmoi/chezmoi.toml
[data]
  projectBinPath = "work/tools/bin"

```

Starship Module Ordering

The format string controls what appears in the prompt and in what order:

```
format = ""
$username\
$hostname\
$time\
$all\
$directory
$character
""
```

The first line contains: username, hostname (SSH only), time, and then \$all which expands to all auto-detected modules (git, languages, docker, cmd_duration, etc.). The second line contains the directory. The third line contains the prompt character.

This two-line layout ensures that the command you type always starts at a predictable position (after >>>>) regardless of how long the first line gets with git info, language versions, and other context.

Disabled Modules

Two modules are explicitly disabled:

```
[battery]
disabled = true
```

```
[gcloud]
disabled = true
```

Battery is disabled because this is a desktop-focused setup where battery status is visible in the macOS menu bar. Google Cloud is disabled to avoid slow API calls that would delay prompt rendering.

Git Status Symbols

The [git_status] module shows compact symbols for the state of the working tree:

Symbol	Meaning
*	Modified files
+	Staged files
!	Conflicted files
?	Untracked files
\$	Stashed changes
>	Ahead of remote
<	Behind remote
=	Diverged from remote

These appear in peach (#fab387), making them stand out against the pink git branch name.

Language Detection

Starship automatically detects the project language and shows version information:

Language	Detection Files	Color
Node.js	package.json, .node-version	Green (#a6e3a1)
Python	requirements.txt, pyproject.toml, .python-version	Yellow (#f9e2af)
Rust	Cargo.toml	Peach (#fab387)
Go	go.mod	Sky (#89dceb)
Docker	Dockerfile, docker-compose.yml	Blue (#89b4fa)
Ruby	Gemfile, .ruby-version	Red (default)

Ruby detection is customized to avoid false positives:

```
[ruby]
detect_variables = []
detect_files = ['Gemfile', '.ruby-version']
```

The `detect_variables = []` prevents Ruby from being detected based on environment variables, which could trigger in directories that happen to have Ruby-related variables set but are not Ruby projects.

Command Duration

```
[cmd_duration]
style = '#f9e2af bold'
min_time = 2_000
```

Commands that take longer than 2 seconds (2000ms) display their duration in yellow. This is useful for:

- Noticing when a build is slower than usual
- Timing long-running operations without needing to prefix them with time
- Quick feedback on whether an optimization made a difference

Shell Startup Performance

The `.zshrc` uses conditional loading to avoid unnecessary work:

```
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh
[ -f "$HOME/.local/bin/env" ] && . "$HOME/.local/bin/env"
[ -d "$(brew --prefix)/opt/mysql-client/bin" ] && ...
```

Each line checks whether a file or directory exists before sourcing or adding it to `PATH`. This means:

- On machines without NVM, the NVM init is skipped.
- On machines without fzf, the fzf shell integration is skipped.
- On machines without MySQL client, no broken `PATH` entries are added.

To profile shell startup time:

```
# Time shell startup
time zsh -i -c exit
```

```
# Detailed profiling (add to top of .zshrc temporarily)
zmodload zsh/zprof
# ... rest of .zshrc ...
# Add to bottom: zprof
```

A healthy startup time is under 200ms. If startup exceeds 500ms, the most common culprits are NVM initialization and brew `--prefix` calls (which spawn a subprocess).

Interaction Between Tools

The initialization chain creates several interactions worth noting:

1. **zoxide replaces cd:** After eval `"$(zoxide init zsh --cmd cd)"`, the `cd` command is a Zsh function that calls zoxide. The original `cd` is still accessible as builtin `cd`.
2. **atuin replaces Ctrl+R:** After eval `"$(atuin init zsh)"`, pressing `Ctrl+R` opens atuin's search UI instead of Zsh's built-in reverse-i-search.
3. **fzf provides Ctrl+T and Alt+C:** The fzf shell integration adds file-finding (`Ctrl+T`) and directory-changing (`Alt+C`) keybindings.
4. **mise activates shims:** After eval `"$(mise activate zsh)"`, language runtimes managed by mise are available in `PATH` via shims.
5. **Starship wraps precmd:** Starship's `init` replaces the prompt-rendering function. All tool-specific prompt modifications must happen before Starship `init`.

macOS Defaults

A stock macOS installation is designed for casual users – these defaults transform it into a development operating system.

Your Setup

macOS stores application and system preferences in a property list (plist) database. The defaults command reads and writes entries in this database. zero.dev001's macOS defaults script runs once via chezmoi and configures approximately 60 settings across 15 categories.

- **Script:** run_once_macos—defaults.sh
- **Chezmoi source:** ~/.local/share/chezmoi/run_once_macos—defaults.sh
- **Execution:** Runs automatically on first chezmoi apply, never again (prefix run_once_)
- **Requires:** Administrator password (uses sudo)
- **Post-run:** Kills affected applications to apply changes immediately
- **Based on:** Mathias Bynens' .macos dotfile, adapted for macOS Sequoia and Apple Silicon

The script begins by closing System Settings (to prevent it from overwriting changes) and establishing a sudo keep-alive:

```
osascript -e 'tell application "System Settings" to quit' 2>/dev/null
sudo -v
while true; do sudo -n true; sleep 60; kill -0 "$$" || exit; done 2>/dev/null &
```

Core Concepts

How macOS Defaults Work

Every macOS application stores its preferences in plist files under ~/Library/Preferences/. The defaults command provides a CLI to read and write these files without opening System Settings.

```
# Read a value
defaults read com.apple.dock tileSize
```

```
# Write a value
defaults write com.apple.dock tileSize -int 36
```

```
# Delete a value (reset to system default)
```

```
defaults delete com.apple.dock tilesize
```

```
# Read all values for an app
defaults read com.apple.dock
```

Changes to most settings require restarting the affected application. The script handles this by killing all affected apps at the end.

Important: Some defaults write commands modify NSGlobalDomain, which applies to all applications. Others target specific app domains like com.apple.dock or com.apple.finder.

The run_once_ Prefix

The run_once_ prefix in the filename tells chezmoi to run this script exactly once. Chezmoi tracks whether it has been executed by storing a hash of the file. If you modify the script, chezmoi will re-run it on the next chezmoi apply.

To force a re-run:

```
# Re-run by changing the script content (even a comment change works)
chezmoi edit run_once_macos-defaults.sh
chezmoi apply
```

```
# Or run it directly
bash ~/.local/share/chezmoi/run_once_macos-defaults.sh
```

Essential Commands

Keyboard Settings

These are the most impactful settings for a developer. They eliminate the delay that macOS adds to key input and disable text transformations that break code.

Setting	Domain	Value	Effect
ApplePressAndHoldEnabled	NSGlobalDomain	false	Disable accent popup on key hold
KeyRepeat	NSGlobalDomain	1	Fastest key repeat rate
InitialKeyRepeat	NSGlobalDomain	10	Shortest delay before repeat
AppleKeyboardUIMode	NSGlobalDomain	3	Tab through all UI controls
NSAutomaticCapitalizationEnabled	NSGlobalDomain	false	No auto-capitalize
NSAutomaticDashSubstitutionEnabled	NSGlobalDomain	false	No smart dashes (– to em-dash)
NSAutomaticPeriodSubstitutionEnabled	NSGlobalDomain	false	No double-space period
NSAutomaticQuoteSubstitutionEnabled	NSGlobalDomain	false	No smart quotes (” to curly)
NSAutomaticSpellingCorrectionEnabled	NSGlobalDomain	false	No autocorrect

Why these matter:

- **Press-and-hold disabled:** On a stock Mac, holding a key shows an accent picker (e, é, etc.). This is useless for development and prevents key repeat. Disabling it restores key repeat behavior, which is essential for holding j in Vim to scroll down.
- **KeyRepeat=1, InitialKeyRepeat=10:** These are the lowest possible values. The key starts repeating almost instantly (10/60th of a second delay) and repeats at the maximum rate (1/60th of a second between repeats). This makes keyboard-driven navigation extremely responsive.
- **Full keyboard access (mode 3):** Allows Tab to cycle through all controls in dialogs, not just text fields. This means you can Tab to “Don’t Save” in a save dialog instead of reaching for the mouse.
- **All auto-substitutions disabled:** Smart quotes turn "hello" into curly quotes that break code. Smart dashes turn -- into em-dashes. Auto-period turns double-space into a period. Auto-capitalize changes the first letter after a period. All of these are destructive for programming.

Trackpad and Input

Setting	Domain	Value	Effect
Clicking (Bluetooth trackpad) com.apple.mouse.tapBehavior	com.apple.driver.AppleBluetoothMultitouch.trackpad NSGlobalDomain	1	Tap to click system-wide
closeViewScrollWheelToggle	com.apple.universalaccess	true	Ctrl+scroll to zoom
HIDScrollZoomModifierMask	com.apple.universalaccess	262144	Use Ctrl key for zoom
closeViewZoomFollowsFocus	com.apple.universalaccess	true	Zoom follows keyboard focus

Tap to click avoids the need to physically press the trackpad. A light tap registers as a click.

Ctrl+scroll zoom is invaluable for presentations, reading small text, or inspecting UI details.

Finder

Setting	Domain	Value	Effect
AppleShowAllExtensions	NSGlobalDomain	true	Show file extensions always
ShowStatusBar	com.apple.finder	true	Show item count and disk space
ShowPathbar	com.apple.finder	true	Show path breadcrumb at bottom
__FXShowPosixPathInTitle	com.apple.finder	true	Full POSIX path in title bar
__FXSortFoldersFirst	com.apple.finder	true	Folders above files when sorting
FXDefaultSearchScope	com.apple.finder	SCcf	Search current folder, not Mac
FXEnableExtensionChangeWarning	com.apple.finder	false	No warning on extension change

Setting	Domain	Value	Effect
DSDontWriteNetworkStores	com.apple.desktopservices	true	No .DS_Store on network volumes
DSDontWriteUSBStores	com.apple.desktopservices	true	No .DS_Store on USB volumes
FXPreferredViewStyle	com.apple.finder	Nlsv	Default to list view
DisableAllAnimations	com.apple.finder	true	No animations
Show ~/Library	chflags nohidden	–	Unhide the Library folder
Expanded Info panes	FXInfoPanelsExpanded	dict	General, Open With, Privileges

Key choices explained:

- **POSIX path in title bar** shows /Users/zero/projects/app instead of “app” in the Finder title. This makes it easy to copy the path and use it in the terminal.
- **Search current folder** prevents the maddening default of searching the entire Mac when you press Cmd+F in a Finder window.
- **No .DS_Store on network/USB** prevents littering shared drives and external media with macOS metadata files that are meaningless on other operating systems.
- **List view** is the most information-dense Finder view and the closest to a terminal listing.

Save, Print, and Dialogs

Setting	Domain	Value	Effect
NSNavPanelExpandedStateForSaveMode	NSGlobalDomain	true	Expanded save dialog
NSNavPanelExpandedStateForSaveMode2	NSGlobalDomain	true	Expanded save dialog (variant)
PMPrintingExpandedStateForPrint	NSGlobalDomain	true	Expanded print dialog
PMPrintingExpandedStateForPrint2	NSGlobalDomain	true	Expanded print dialog (variant)
NSDocumentSaveNewDocumentsToCloud	NSGlobalDomain	false	Save to local disk, not iCloud
Quit When Finished	com.apple.print.PrintingPrefs	true	Auto-quit printer app
LSQuarantine	com.apple.LaunchServices	false	No quarantine dialog

Expanded save/print panels show the full file browser instead of the minimized dropdown. The minimized version hides the directory tree and forces you to navigate through a tiny interface.

Save to disk prevents the infuriating default of saving new documents to iCloud instead of the local filesystem.

No quarantine dialog suppresses the “Are you sure you want to open this application downloaded from the internet?” dialog. Since all applications are installed through Homebrew or the App Store, this dialog adds friction without adding security.

Screenshots

Setting	Domain	Value	Effect
location	com.apple.screencapture	~/Desktop/screenshots	Custom save location
type	com.apple.screencapture	png	PNG format (lossless)
disable—shadow	com.apple.screencapture	true	No window shadow

Screenshots go to a dedicated ~/Desktop/screenshots directory instead of cluttering the Desktop. PNG format preserves quality for documentation and bug reports. Disabling the shadow removes the macOS drop shadow from window screenshots, producing cleaner images.

Dock

Setting	Domain	Value	Effect
tilesize	com.apple.dock	36	Small 36px icons
mineffect	com.apple.dock	scale	Scale effect (faster than Genie)
minimize—to—application	com.apple.dock	true	Minimize into app icon
launchanim	com.apple.dock	false	No bounce animation on launch
expose—animation—duration	com.apple.dock	0.1	Fast Mission Control (100ms)
mru—spaces	com.apple.dock	false	No auto-rearrange Spaces
autohide	com.apple.dock	true	Auto-hide the Dock
autohide—delay	com.apple.dock	0	No delay before hiding
autohide—time—modifier	com.apple.dock	0	No hide/show animation
showhidden	com.apple.dock	true	Translucent icons for hidden apps
show—recents	com.apple.dock	false	No recent apps section

The Dock philosophy: The Dock is an obstacle in a keyboard-driven workflow. These settings minimize its footprint:

- **Auto-hide with zero delay and zero animation** means the Dock vanishes instantly when not needed and reappears instantly on hover. There is no animation lag.
- **36px icons** make the Dock as small as practical when it does appear.
- **No MRU Spaces** prevents macOS from reordering your Spaces based on usage. With Aerospace managing window positions, unpredictable Space reordering would break spatial muscle memory.
- **No recent apps** removes the “suggested” section from the Dock that wastes space.
- **No launch animation** eliminates the bouncing icon that plays when starting an app.
- **Scale effect** is the fastest minimize animation. Genie is visually dramatic but slower.

Window and UI Speed

Setting	Domain	Value	Effect
NSWindowResizeTime	NSGlobalDomain	0.001	Near-instant window resize
NSUseAnimatedFocusRing	NSGlobalDomain	false	No focus ring animation
DevMode (Help Viewer)	com.apple.helpviewer	true	Non-floating Help Viewer
NSQuitAlwaysKeepsWindows	System Preferences	false	Disable Resume system-wide

Near-instant resize (0.001 seconds) makes Cocoa applications resize without visible animation. The default includes a smooth resize animation that feels sluggish in a tiling window manager where windows resize frequently.

Non-floating Help Viewer allows the Help window to go behind other windows. By default, Help windows float above everything, which is annoying when you want to read help and type commands simultaneously.

Disable Resume prevents applications from reopening their previous windows when launched. This avoids the situation where opening TextEdit launches with six old documents from months ago.

Security

Setting	Domain	Value	Effect
askForPassword	com.apple.screensaver	1	Require password after screensaver
askForPasswordDelay	com.apple.screensaver	0	No grace period

Password is required immediately after sleep or the screen saver begins. There is no delay – the instant the screen locks, a password is needed to unlock it. This is a basic security measure for any machine used in shared spaces.

Terminal and iTerm2

Setting	Domain	Value	Effect
StringEncodings	com.apple.terminal	4	UTF-8 only
SecureKeyboardEntry	com.apple.terminal	true	Prevents keylogging
ShowLineMarks	com.apple.Terminal	0	No line marks
PromptOnQuit	com.googlecode.iterm2	false	No quit confirmation

Terminal.app is configured as a fallback. UTF-8 encoding prevents character display issues. Secure keyboard entry prevents other applications from intercepting keystrokes sent to the terminal – an important security measure when typing passwords or tokens.

The iTerm2 quit prompt is disabled because iTerm2 is the primary terminal and the confirmation dialog adds unnecessary friction when quitting.

Activity Monitor

Setting	Domain	Value	Effect
OpenMainWindow	com.apple.ActivityMonitor	true	Show window on launch
IconType	com.apple.ActivityMonitor	5	CPU graph in Dock icon
ShowCategory	com.apple.ActivityMonitor	0	Show all processes
SortColumn	com.apple.ActivityMonitor	CPUUsage	Sort by CPU
SortDirection	com.apple.ActivityMonitor	0	Descending (highest first)

Activity Monitor shows a live CPU usage graph in its Dock icon (IconType 5), providing a glanceable indicator of system load. Processes are sorted by CPU usage in descending order so the heaviest processes are always visible at the top.

TextEdit

Setting	Domain	Value	Effect
RichText	com.apple.TextEdit	0	Plain text by default
PlainTextEncoding	com.apple.TextEdit	4	Open files as UTF-8
PlainTextEncodingForWrite	com.apple.TextEdit	4	Save files as UTF-8

TextEdit defaults to rich text (RTF), which is almost never what a developer wants. These settings make it open and save plain text with UTF-8 encoding, turning it into a simple, fast notepad for quick edits.

Mac App Store

Setting	Domain	Value	Effect
AutomaticCheckEnabled	com.apple.SoftwareUpdate	true	Enable update checks
ScheduleFrequency	com.apple.SoftwareUpdate	1	Check daily
AutomaticDownload	com.apple.SoftwareUpdate	1	Download in background
CriticalUpdateInstall	com.apple.SoftwareUpdate	1	Auto-install security updates
AutoUpdate	com.apple.commerce	true	Auto-update App Store apps

Updates are checked daily, downloaded in the background, and security updates are installed automatically. This keeps the system patched without manual intervention.

Photos

Setting	Domain	Value	Effect
disableHotPlug	com.apple.ImageCapture	true	No auto-open on device connect

Prevents Photos from opening automatically when an iPhone, camera, or SD card is connected. This is a frequent annoyance when plugging in a phone to charge.

Messages

Setting	Domain	Value	Effect
<code>automaticEmojiSubstitutionEnabled</code>	Messages controller	false	No auto-emoji
<code>automaticQuoteSubstitutionEnabled</code>	Messages controller	false	No smart quotes

Prevents Messages from converting text like :) into emoji and straight quotes into curly quotes. Both of these transformations are destructive when sharing code snippets in messages.

Mail

Setting	Domain	Value	Effect
<code>AddressesIncludeNameOnPasteboard</code>	com.apple.mail	false	Copy as foo@bar.com not Name <foo@bar.com>
<code>DisableInlineAttachmentViewing</code>	com.apple.mail	true	Show attachment icons, not inline

Plain email addresses means copying an address from Mail gives you user@example.com instead of User Name <user@example.com>. The plain format is what you actually need when pasting into forms, scripts, or configuration files.

Disable inline attachments shows attached files as icons instead of rendering them inline in the email body. This makes emails with attachments cleaner and prevents large images from taking over the view.

Practical Recipes

Recipe: Applying Defaults to a New Machine

Option 1: Through chezmoi (runs automatically on first apply)
`chezmoi init --apply your-github-username`

Option 2: Run the script directly
`bash ~/.local/share/chezmoi/run_once_macos-defaults.sh`

Option 3: Apply a single setting manually
`defaults write com.apple.dock autohide --bool true`
`killall Dock`

After running the full script, a logout or restart is recommended to ensure all settings take effect. Most settings apply immediately after the affected app is restarted, but some (like keyboard repeat rate) may require a logout.

Recipe: Checking Current Values

Before making changes, you can read the current value of any setting:

```
# Check keyboard repeat rate
defaults read NSGlobalDomain KeyRepeat
defaults read NSGlobalDomain InitialKeyRepeat

# Check Dock settings
defaults read com.apple.dock tilesize
defaults read com.apple.dock autohide

# Check if autocorrect is off
defaults read NSGlobalDomain NSAutomaticSpellingCorrectionEnabled

# Dump all settings for an app
defaults read com.apple.finder

# Find a setting by name across all domains
defaults find "KeyRepeat"
```

Recipe: Resetting a Single Setting

If a setting causes problems, delete it to restore the macOS default:

```
# Reset Dock icon size to system default
defaults delete com.apple.dock tilesize
killall Dock

# Reset keyboard repeat to system default
defaults delete NSGlobalDomain KeyRepeat
defaults delete NSGlobalDomain InitialKeyRepeat
# Logout required
```

Recipe: Verifying Key Repeat Settings

After applying the keyboard defaults, test that key repeat works correctly:

```
# Open any text editor and hold down a key
# With KeyRepeat=1 and InitialKeyRepeat=10:
# - The key should start repeating almost immediately (~167ms)
# - Characters should appear very rapidly once repeating starts

# If the accent popup appears instead of key repeat:
defaults read NSGlobalDomain ApplePressAndHoldEnabled
# Should return 0 (false). If not:
defaults write NSGlobalDomain ApplePressAndHoldEnabled -bool false
# Then restart the application where you are testing
```

Advanced Usage

Understanding the Sudo Keep-Alive

The script uses a background loop to maintain sudo privileges:

```
sudo -v
while true; do sudo -n true; sleep 60; kill -0 "$$" || exit; done 2>/dev/null &
```

This works as follows:

1. `sudo -v` prompts for the password and caches the credentials.
2. The while loop runs in the background, refreshing the sudo timestamp every 60 seconds.
3. `kill -0 "$$"` checks if the parent script is still running. If the script finishes, the loop exits.
4. The `2>/dev/null` suppresses error output from the loop.

This prevents the script from stalling mid-execution to ask for the password again.

Settings That Require Logout or Restart

Most settings take effect immediately after restarting the affected application. However, some require a full logout or restart:

Setting	Requires
Keyboard repeat rate	Logout (sometimes app restart)
Full keyboard access	Logout
Tap to click	Logout
Trackpad zoom	Logout
Screenshot location	App restart (screenshot)
Dock changes	killall Dock
Finder changes	killall Finder

The script kills affected applications at the end:

```
for app in "Activity_Monitor" "cfprefsd" "Dock" "Finder" \
  "Mail" "Messages" "Photos" "SystemUIServer" "Terminal"; do
  killall "${app}" &>/dev/null
done
```

Note that `cfprefsd` is the preferences daemon. Killing it forces macOS to re-read all preference files from disk.

Compatibility Notes

This script is adapted for macOS Sequoia on Apple Silicon. Some settings from older .macos scripts are intentionally excluded:

- **SIP-protected settings** that cannot be changed without disabling System Integrity Protection.
- **Deprecated domains** that no longer exist in modern macOS versions.
- **Settings that break on Apple Silicon** due to the different architecture.

If a setting does not take effect, check whether it has been superseded by a newer macOS version. Apple occasionally moves settings to new domains or removes them entirely.

Monitoring Default Changes

To discover what settings an application uses, you can watch for changes:

```
# Before changing a setting in System Settings, dump the current state:  
defaults read > /tmp/before.plist
```

```
# Make the change in System Settings GUI
```

```
# Dump the new state:  
defaults read > /tmp/after.plist
```

```
# Diff to see what changed:  
diff /tmp/before.plist /tmp/after.plist
```

This technique is useful for discovering undocumented defaults write commands that correspond to GUI toggles in System Settings.

Navigation: zoxide

A smarter `cd` that learns your habits – replacing the built-in entirely so every directory change gets smarter over time.

Your Setup

Zoxide is initialized in `~/.zshrc` with a critical flag that most guides overlook:

```
eval "$(zoxide_init_zsh --cmd cd)"
```

The `--cmd cd` argument replaces the shell built-in `cd` command with zoxide. This means you do not use a separate `z` command. Every time you type `cd`, you are invoking zoxide's frequency-based directory jumper. The `z` command does not exist in this setup – `cd` is zoxide.

Additionally, `cdi` is automatically created as the interactive variant, launching `fzf` for directory selection.

Navigation aliases are defined in `~/.aliases.zsh`:

```
alias ..="cd .."
alias ...="cd ../.."
alias ....="cd ../../.."
alias .....="cd ../../../.."
alias ____="cd _"
```

Custom navigation functions build on top of zoxide:

```
cx() { cd "$@" && 1; }           # cd + list directory contents
fcd() { cd "$(fd --type d --hidden --follow --exclude .git | fzf)" && 1; }
```

Core Concepts

Frequency Algorithm

Zoxide ranks directories by “frequency” – a combination of frequency and recency. Directories you visit often and recently score higher. The database updates every time you `cd` into a directory.

- **Frequency:** How many times you have visited a directory.
- **Recency:** How recently you visited. Recent visits are weighted more heavily.
- **Aging:** Scores decay over time. Directories you stop visiting gradually lose rank.

How Matching Works

When you type `cd foo`, zoxide does not simply look for a directory named `foo` in the current directory. It searches its database of all directories you have ever visited and picks the highest-scoring match:

- `cd proj` – jumps to the highest-ranked directory containing “proj”
- `cd proj api` – matches directories containing both “proj” and “api” as substrings, in order
- `cd /exact/path` – absolute paths bypass zoxide and work as traditional `cd`

If zoxide has no match, it falls back to standard `cd` behavior in the current directory.

The Database

Zoxide stores its database at `~/.local/share/zoxide/db.zo`. The database is updated automatically on every successful `cd`. You can query and manage it with `zoxide query`.

Essential Commands

Command	Action	Notes
<code>cd foo</code>	Jump to best “foo” match	Uses frequency ranking
<code>cd foo bar</code>	Jump to match for both terms	Terms matched in order
<code>cd ..</code>	Parent directory	Alias: <code>..</code>
<code>cd ../..</code>	Two levels up	Alias: <code>...</code>
<code>cd ../../..</code>	Three levels up	Alias: <code>....</code>
<code>cd -</code>	Previous directory	Alias: <code>-</code>
<code>cdi</code>	Interactive directory picker	fzf-powered selection
<code>cx foo</code>	<code>cd</code> + list contents	Custom function
<code>fed</code>	<code>fd</code> + <code>fzf</code> + <code>cd</code> + list	Custom function

Database Management

Command	Action
<code>zoxide query</code>	List all tracked directories with scores
<code>zoxide query foo</code>	Show best match for “foo”
<code>zoxide query foo --list</code>	List all matches for “foo” with scores
<code>zoxide query --all</code>	Show all directories sorted by score
<code>zoxide add /path</code>	Manually add a directory to the database
<code>zoxide remove /path</code>	Remove a directory from the database
<code>zoxide edit</code>	Open the database in your editor

Practical Recipes

Jump to a project by partial name

```
cd dotfiles          # Jumps to ~/.local/share/chezmoi if that is your top match
cd myapp api         # Jumps to ~/projects/myapp/packages/api
```

Navigate up then list

The `cx` function combines `cd` with the `l` alias (which runs `eza -l --icons --git -a`), giving you an immediate view of what is in the target directory:

```
cx ..                # Go up one level and list
cx ~/projects        # Jump to projects and list
```

Fuzzy directory navigation with `fcd`

The `fcd` function uses `fd` to find all directories (including hidden ones, excluding `.git`), pipes them through `fzf` for interactive selection, then `cds` into the chosen directory and lists its contents:

```
fcd                # Opens fzf with all directories under current path
```

This is useful when you do not remember the exact directory name but want to browse visually.

Interactive mode with `cdi`

When `zoxide` has multiple candidates and you want to choose manually:

```
cdi proj            # Shows all "proj" matches in fzf, lets you pick
cdi                 # Shows all tracked directories in fzf
```

Check what `zoxide` would pick

```
zoxide query proj   # Prints the path zoxide would jump to
zoxide query proj --list # Shows all candidates with scores
```

Clean up stale entries

Over time, deleted directories accumulate in the database. `Zoxide` automatically removes entries when you try to `cd` into a directory that no longer exists, but you can also clean manually:

```
zoxide remove /old/path/that/no/longer/exists
```

Seed the database for a new machine

After a fresh setup, the database is empty. Visit your important directories once to seed it:

```
cd ~/projects/app1 && cd ~/projects/app2 && cd ~/.config && cd ~
```

After that, `cd app1` will work from anywhere.

Advanced Usage

Integration with Other Tools

`Zoxide` is not just for the shell. It powers directory navigation across multiple tools in this environment:

Tool	How Zoxide Is Used
Yazi	Press Z to jump to any directory via zoxide
sessh	Session picker uses zoxide for project directory suggestions
tmux sessionx	Can browse zoxide directories when creating sessions
cdi	The zoxide-generated interactive variant with fzf

How `--cmd cd` Changes the Shell

The `--cmd cd` flag generates shell functions that replace `cd` with `__zoxide_z` and create `cdi` as `__zoxide_zi`. The actual shell integration looks roughly like:

```
function cd() {
    __zoxide_z "$@"
}
function cdi() {
    __zoxide_zi "$@"
}
```

This means `cd` gains smart matching, but standard `cd` behavior (absolute paths, `cd -`, `cd` with no arguments to go home) is preserved.

Precedence Rules

Zoxide applies this matching logic in order:

1. If the argument is empty, go to `$HOME` (standard `cd` behavior).
2. If the argument is `-`, go to `$OLDPWD` (previous directory).
3. If the argument is an absolute path or a valid relative path, use it directly.
4. Otherwise, search the database for the best frequency match.

This means `cd ./foo` always goes to the literal `./foo` subdirectory, while `cd foo` searches the database.

Excluding Directories

You can prevent zoxide from tracking certain directories by setting `_ZO_EXCLUDE_DIRS`:

```
export _ZO_EXCLUDE_DIRS="$HOME:$HOME/private/*:/tmp/*"
```

Environment Variables

Variable	Default	Purpose
<code>_ZO_DATA_DIR</code>	<code>~/.local/share/zoxide</code>	Database location
<code>_ZO_ECHO</code>	<code>0</code>	Print matched directory before jumping
<code>_ZO_EXCLUDE_DIRS</code>	<code>\$HOME</code>	Directories to exclude from tracking
<code>_ZO_FZF_OPTS</code>	(none)	Extra options passed to <code>fzf</code> in interactive mode

Variable	Default	Purpose
<code>_ZO_MAXAGE</code>	10000	Maximum total score before aging kicks in
<code>_ZO_RESOLVE_SYMLINKS</code>	0	Resolve symlinks before adding to database

Debugging Unexpected Jumps

If `cd foo` takes you somewhere unexpected:

```
zoxide query foo —list      # See all candidates and their scores
zoxide remove /wrong/path   # Remove the unwanted entry
cd /correct/path           # Visit the correct one to boost its score
```

The Dot Aliases and Zoxide

The dot aliases (`..`, `...`, `....`, `.....`) call `cd ..`, `cd ../..`, and so on. Because `cd` is `zoxide`, these relative paths are passed through. Since `..` resolves to a valid relative path, `zoxide` uses it directly without database lookup – exactly as you would expect.

The `—` alias calls `cd —`, which triggers `zoxide`'s built-in support for `$OLDPWD`, toggling between your two most recent directories.

Fuzzy Finding: fzf

A general-purpose fuzzy finder that turns any list into an interactive, filterable menu – it finds nothing on its own but makes everything else searchable.

Your Setup

fzf is sourced in `~/.zshrc`:

```
[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh
```

This loads fzf's key bindings and shell completion into the zsh session. fzf itself is a filter: it reads lines from stdin, presents an interactive fuzzy search interface, and outputs the selected line(s) to stdout.

Custom functions in `~/.aliases.zsh` build on fzf:

```
f() { echo "$(fd —type f —hidden —follow —exclude .git | fzf)" | pbcopy; }
fv() { nvim "$(fd —type f —hidden —follow —exclude .git | fzf)"; }
fcd() { cd "$(fd —type d —hidden —follow —exclude .git | fzf)" && 1; }
```

Core Concepts

fzf Is a Filter, Not a Finder

This is the single most important thing to understand. fzf does not search your filesystem. It takes a list of strings as input, lets you narrow that list interactively, and outputs the selection. The pattern is always:

```
generate list | fzf | do something with selection
```

For example:

```
fd —type f | fzf | xargs nvim      # fd finds, fzf picks, nvim opens
git branch | fzf | xargs git checkout # git lists, fzf picks, git acts
```

The Find-Pick-Act Pattern

Nearly every fzf workflow follows three steps:

1. **Find** – A fast tool generates a list (fd, rg, git, ls, etc.)
2. **Pick** – fzf presents the list for fuzzy selection

3. **Act** – The selection is passed to another tool (cd, nvim, pbcopy, etc.)

Your custom functions implement exactly this pattern:

Function	Find	Pick	Act
f	fd --type f	fzf	pbcopy (copy path)
fv	fd --type f	fzf	nvim (edit file)
fed	fd --type d	fzf	cd + l (navigate + list)

Fuzzy Matching Syntax

fzf supports several matching operators within the search query:

Token	Match Type	Example
foo	Fuzzy match	Items containing f, o, o in order
'foo	Exact match	Items containing literal “foo”
^foo	Prefix exact match	Items starting with “foo”
foo\$	Suffix exact match	Items ending with “foo”
!foo	Inverse exact match	Items NOT containing “foo”
!^foo	Inverse prefix	Items NOT starting with “foo”
!foo\$	Inverse suffix	Items NOT ending with “foo”

Combine tokens with spaces (AND logic): `^src .ts$ !test` matches files starting with “src”, ending in “.ts”, excluding “test”.

Essential Commands

Shell Key Bindings

Key	Action	Notes
Ctrl+T	Find file, paste path	Inserts selected file path at cursor
Alt+C	Find directory, cd into it	Changes to selected directory
Ctrl+R	Search shell history	Overridden by Atuin in this setup

Note: Since Atuin is initialized after fzf in `~/.zshrc`, Atuin’s Ctrl+R takes precedence. fzf’s history search is effectively replaced by Atuin’s richer interface.

Custom Functions

Command	Action	What Happens
f	Find file, copy path to clipboard	fd files -> fzf -> pbcopy
fv	Find file, open in Neovim	fd files -> fzf -> nvim
fcd	Find directory, cd + list	fd dirs -> fzf -> cd + eza listing

fzf Options

Flag	Purpose	Example
-m	Multi-select (Tab to toggle)	fzf -m
-q	Start with a pre-filled query	fzf -q ".md"
--preview	Show file preview	fzf --preview 'bat {}'
--height	Limit to percentage of terminal	fzf --height 40%
--reverse	Reverse layout (list at top)	fzf --reverse
--border	Draw border around finder	fzf --border
--header	Display a header line	fzf --header "Pick a file"
--bind	Custom key bindings	fzf --bind 'ctrl-y:accept'
--delimiter	Field delimiter	fzf --delimiter :
--with-nth	Display specific fields	fzf --with-nth 1,3
--preview-window	Preview pane position/size	fzf --preview-window right:60%

Practical Recipes

Find and edit a file

```
fv                                # Uses the custom function
# Or manually:
fd --type f | fzf --preview 'bat --color=always {}' | xargs nvim
```

Find and copy a file path

```
f                                # Uses the custom function -- path goes to clipboard
# Then paste with Cmd+V anywhere
```

Find a directory and navigate to it

```
fcd                              # Uses the custom function
# Or use the shell binding:
# Press Alt+C and type to filter
```

Multi-select files to open

```
fd --type f -e md | fzf -m --preview 'bat --color=always {}' | xargs nvim
```

Press Tab to toggle selection on individual items, then Enter to confirm.

Search git branches and switch

```
git branch --all | fzf --height 40% | sed 's/^[* ]*//' | sed 's|remotes/origin/||'
```

Preview files while searching

```
fd --type f | fzf --preview 'bat --color=always --line-range=:100 {}' --preview-window
```

Filter process list and kill

```
ps aux | fzf --height 40% --header "Select process to kill" | awk '{print $2}' | xargs
```

Browse and select from any command output

```
brew list | fzf # Pick an installed package
docker ps --format '{{.Names}}' | fzf # Pick a container
npm run --json 2>/dev/null | jq -r 'keys[]' | fzf # Pick an npm script
```

Pipe ripgrep results through fzf

```
rg --line-number "TODO" | fzf --delimiter : --preview 'bat --color=always --highlight'
```

This searches for “TODO” with rg, pipes the results through fzf with a bat preview that highlights the matching line.

Advanced Usage

fzf as a UI Component in Other Tools

fzf is embedded deeply across this dev environment. It acts as the interactive selection layer for many tools:

Tool	How fzf Is Used
sesh	sesh connect uses fzf-tmux -p 55%,60% for session picking
tmux-fzf-url	Extracts URLs from tmux pane, picks with fzf
tmux-fzf	Fuzzy control of tmux sessions, windows, panes
Yazi	Press z to jump via fzf plugin
Neovim Telescope	Uses fzf-native sorter for fast matching
zoxide cdi	Interactive mode powered by fzf
Ctrl+T binding	Shell-level file finding
Alt+C binding	Shell-level directory navigation

fzf-tmux

The fzf-tmux command runs fzf in a tmux popup or split instead of inline. This is how sesh presents its session picker:

```
sesh connect "$(sesh list | fzf-tmux -p 55%,60%)"
```

The -p 55%,60% flag creates a centered popup at 55% width and 60% height.

Customizing fzf with Environment Variables

You can set defaults via environment variables in your shell configuration:

Variable	Purpose	Example
FZF_DEFAULT_COMMAND	Default input when no stdin	<code>fd --type f --hidden --follow</code>
FZF_DEFAULT_OPTS	Default fzf options	<code>--height 40% --reverse --border</code>
FZF_CTRL_T_COMMAND	Command for Ctrl+T binding	<code>fd --type f --hidden --follow</code>
FZF_CTRL_T_OPTS	Options for Ctrl+T	<code>--preview 'bat --color=always {}'</code>
FZF_ALT_C_COMMAND	Command for Alt+C binding	<code>fd --type d --hidden --follow</code>
FZF_ALT_C_OPTS	Options for Alt+C	<code>--preview 'eza --tree --level=1 {}'</code>

Composing Custom Pickers

The real power is building one-off pickers for any situation:

```
# Pick a Brewfile to install from
ls ~/.config/brew/Brewfile* | fzf | xargs brew bundle --file

# Pick a dotfile to edit
chezmoi managed | fzf --preview 'bat --color=always ~/{{}}' | xargs chezmoi edit

# Pick a git stash to apply
git stash list | fzf --height 30% | cut -d: -f1 | xargs git stash pop
```

Key Bindings Inside fzf

While the fzf interface is active, these keys are available by default:

Key	Action
Enter	Confirm selection
Esc / Ctrl+C	Cancel
Ctrl+J / Ctrl+N	Move down
Ctrl+K / Ctrl+P	Move up
Tab	Toggle selection (multi-select mode)
Shift+Tab	Deselect (multi-select mode)
Ctrl+A	Select all (multi-select mode)
Ctrl+/ Shift+Up/Down	Toggle preview wrap
Page Up/Down	Scroll preview
	Scroll results

The Mental Model

Think of fzf as a universal adapter. Any tool that produces a list of lines can have an interactive fuzzy interface added in front of it, and any tool that accepts an argument can act on the result. If you find yourself scanning through long command output looking for one item, pipe it through fzf instead.

Search: fd and ripgrep

Two Rust-powered search tools that replace find and grep – fd locates files by name, ripgrep searches file contents, and both are fast enough to pipe through fzf interactively.

Your Setup

Both tools are installed via Homebrew and used extensively throughout the environment:

- **fd** feeds file lists to fzf (Ctrl+T binding, custom functions), Neovim Telescope, and Yazi search-by-name.
- **ripgrep (rg)** powers content search in Neovim Telescope live grep and Yazi search-by-content.

Custom functions from ~/.aliases.zsh that depend on fd:

```
f() { echo "$(fd_type f hidden follow exclude .git | fzf)" | pbcopy; }
fv() { nvim "$(fd_type f hidden follow exclude .git | fzf)"; }
fcd() { cd "$(fd_type d hidden follow exclude .git | fzf)" && 1; }
```

Both tools respect .gitignore by default, which makes them fast and noise-free in project directories.

Core Concepts

fd: File Finding

fd is a fast, user-friendly alternative to find. Key design principles:

- **Smart defaults:** Ignores hidden files and .gitignore patterns by default.
- **Regex by default:** The pattern argument is a regular expression, not a glob.
- **Colorized output:** Results are syntax-highlighted by file type.
- **Parallel execution:** The --exec flag runs commands in parallel.

ripgrep: Content Searching

ripgrep (invoked as rg) is a fast alternative to grep for searching file contents. Key design principles:

- **Recursive by default:** Searches the current directory tree without flags.
- **Respects .gitignore:** Skips ignored files automatically.
- **Smart case:** With -S, lowercase patterns are case-insensitive; patterns with uppercase are case-sensitive.
- **Line-oriented:** Outputs file path, line number, and matching line.

Comparison Tables

fd vs find

Task	fd	find
Find files named “config”	fd config	find . -name '*config*'
Find only directories	fd -t d	find . -type d
Find by extension	fd -e json	find . -name '*.json'
Include hidden files	fd -H	Default behavior
Exclude a directory	fd --exclude node_modules	find . -not -path '*/node_modules/*'
Execute command	fd -e js --exec wc -l	find . -name '*.js' -exec wc -l {} \;
Max depth	fd -d 2	find . -maxdepth 2
Case insensitive	fd -i pattern	find . -iname '*pattern*'
Respects .gitignore	Yes (default)	No

rg vs grep

Task	rg	grep
Search for “TODO” recursively	rg TODO	grep -r TODO .
Case insensitive	rg -i todo	grep -ri todo .
Show only filenames	rg -l TODO	grep -rl TODO .
Context lines	rg -C 3 TODO	grep -C 3 -r TODO .
Word boundary match	rg -w error	grep -rw error .
Search specific file type	rg -t py import	grep -r --include='*.py' import .
Invert match	rg -v pattern	grep -rv pattern .
Count matches	rg -c pattern	grep -rc pattern .
Respects .gitignore	Yes (default)	No
Regex by default	Yes	Basic (use -E for extended)

Essential Commands: fd

Command	Action
fd pattern	Find files/dirs matching regex pattern
fd -e ext	Find by extension (e.g., fd -e rs)
fd -t f	Files only
fd -t d	Directories only
fd -t l	Symlinks only
fd -H	Include hidden files
fd -I	Do not respect .gitignore
fd --no-ignore	Skip all ignore files
fd -d N	Limit search depth to N
fd --exclude dir	Exclude a directory
fd -E '*.min.js'	Exclude by glob pattern
fd -x cmd	Execute command for each result (parallel)
fd -X cmd	Execute command with all results as arguments

Command	Action
<code>fd -0</code>	Null-separated output (for xargs -0)
<code>fd pattern /path</code>	Search in a specific directory
<code>fd -a</code>	Show absolute paths
<code>fd --changed-within 1d</code>	Modified within last day
<code>fd --changed-before 1w</code>	Modified before last week
<code>fd -S +1m</code>	Files larger than 1 megabyte

Essential Commands: ripgrep

Command	Action
<code>rg pattern</code>	Search current directory tree for pattern
<code>rg -i pattern</code>	Case insensitive search
<code>rg -S pattern</code>	Smart case (lowercase = insensitive)
<code>rg -w pattern</code>	Match whole words only
<code>rg -l pattern</code>	List only filenames with matches
<code>rg -c pattern</code>	Count matches per file
<code>rg -C N pattern</code>	Show N context lines around matches
<code>rg -A N pattern</code>	Show N lines after each match
<code>rg -B N pattern</code>	Show N lines before each match
<code>rg -t type pattern</code>	Restrict to file type (e.g., <code>rg -t py</code>)
<code>rg -T type pattern</code>	Exclude a file type
<code>rg -g 'glob' pattern</code>	Filter files by glob
<code>rg -g '!test*' pattern</code>	Exclude files matching glob
<code>rg --hidden pattern</code>	Include hidden files
<code>rg --no-ignore pattern</code>	Skip gitignore rules
<code>rg -e pat1 -e pat2</code>	Multiple patterns (OR logic)
<code>rg -v pattern</code>	Invert match (non-matching lines)
<code>rg -F 'literal '</code>	Fixed string (no regex)
<code>rg -m N pattern</code>	Limit to N matches per file
<code>rg --json pattern</code>	Output in JSON format
<code>rg -o pattern</code>	Print only the matching portion
<code>rg --sort path pattern</code>	Sort results by file path
<code>rg -U 'multi\nline'</code>	Multiline matching

Practical Recipes

Find all TypeScript files in a project

```
fd -e ts -e tsx
```

Find large files

```
fd -S +10m          # Files over 10 MB
fd -S +100k -e log  # Log files over 100 KB
```

Find recently modified files

```
fd --changed-within 2h          # Modified in the last 2 hours
fd --changed-within 1d -e rs    # Rust files modified today
```

Delete all .DS_Store files

```
fd -H .DS_Store -x rm          # The -H flag includes hidden files
```

Search for a function definition

```
rg "fn_main" -t rust           # In Rust files
rg "function.*export" -t js     # Exported functions in JavaScript
rg "def_(test_|it_)" -t py      # Test functions in Python
```

Search and replace across files

```
rg -l "old_name" | xargs sed -i '' 's/old_name/new_name/g'
```

Find TODOs with context

```
rg "TODO|FIXME|HACK" -C 2 --heading
```

The `--heading` flag groups results by file for readability.

Search only in specific directories

```
rg pattern src/                # Only search in src/
rg pattern -g 'src/**/*.ts'    # Only TypeScript files under src/
```

Exclude test files from search

```
rg pattern -g '!*test*' -g '!*spec*'
```

Count occurrences by file

```
rg -c "import" -t ts --sort path
```

Pipe fd to fzf to act (the standard pattern)

```
# Find a file and open it
fd -e md | fzf --preview 'bat --color=always {}' | xargs nvim
```

```
# Find a directory and cd into it
cd "$(fd -t d | fzf)"
```

```
# Find a file and copy its path
fd -t f | fzf | pbcopy
```

Pipe rg to fzf for interactive grep

```
rg --line-number --color=always pattern | fzf --ansi --delimiter : \
  --preview 'bat --color=always --highlight-line {2} {1}' \
  --preview-window '+{2}/2'
```

Search for files containing two different patterns

```
rg -l "pattern1" | xargs rg -l "pattern2"
```

List all file types ripgrep knows

```
rg --type-list                # Shows all supported types
rg --type-list | rg python    # Find python-related types
```

Advanced Usage

Integration Map

Both tools are embedded throughout the dev environment:

Tool	fd Usage	rg Usage
fzf Ctrl+T	File list generation	–
fzf Alt+C	Directory list generation	–
Custom f/fv/fcd	File and directory lists	–
Neovim Telescope	File finder backend	Live grep backend
Yazi	s key: search by name	S key: search by content

fd Exec Patterns

fd's `-x` (each) and `-X` (batch) flags are powerful alternatives to `xargs`:

```
# Run a command for each file (parallel by default)
fd -e jpg -x convert {} {.}.png      # Convert all JPGs to PNG
fd -e rs -x wc -l                     # Count lines in each Rust file

# Run a command with all files as arguments (single invocation)
fd -e ts -X prettier --write          # Format all TypeScript files at once

# Placeholder syntax in -x:
# {}    full path
# {.}   path without extension
# {/}   filename only
# {//}  parent directory
# {/}.} filename without extension
```

ripgrep Configuration File

ripgrep supports a configuration file at `$RIPGREP_CONFIG_PATH`:

```
export RIPGREP_CONFIG_PATH="$HOME/.config/ripgrep/config"
```

Example config file:

```
—smart—case
—hidden
—glob=!.git
—glob=!node_modules
```

Performance Tips

Both tools are fast by default, but you can squeeze more speed:

- **Limit depth:** `fd -d 3` and `rg --max-depth 3` avoid deeply nested directories.
- **Target directories:** `rg pattern src/` is faster than `rg pattern` when you know where to look.
- **Use type filters:** `rg -t py` avoids checking files that cannot match.
- **Use fixed strings:** `rg -F "exact.string"` avoids regex overhead.
- **Parallel output:** `fd` defaults to parallel execution with `-x`; `rg` defaults to parallel search across files.

Combining fd and rg

Sometimes you want `fd`'s file-finding ability with `rg`'s content searching. Use `fd` to generate the file list and pass it to `rg`:

```
# Search only in recently modified files
fd —changed—within 1d -e ts -X rg "TODO"
```

```
# Search in files matching a complex name pattern
fd "controller" -e rb -X rg "before_action"
```

```
# Search excluding certain directories that rg's globs cannot express
fd -t f —exclude legacy —exclude vendor -X rg "deprecated"
```

Using with git

```
# Find untracked files matching a pattern
git ls-files —others —exclude-standard | rg "test"
```

```
# Search only in files tracked by git
rg pattern $(git ls-files)
```

```
# Find all files that differ from main
git diff —name-only main | xargs rg "TODO"
```

File Manager: Yazi

A terminal file manager with Vim motions, image previews, and deep integration with fd, rg, fzf, and zoxide – launched with a shell wrapper that syncs your working directory on exit.

Your Setup

Yazi is launched via a shell wrapper function defined in `~/.aliases.zsh`:

```
function y() {  
    local tmp="$(mktemp -t "yazi-cwd.XXXXXX")" cwd  
    command yazi "$@" --cwd-file="$tmp"  
    IFS= read -r -d ' ' cwd < "$tmp"  
    [ "$cwd" != "$PWD" ] && [ -d "$cwd" ] && builtin cd -- "$cwd"  
    rm -f -- "$tmp"  
}
```

This wrapper writes the last working directory to a temp file on exit. When you quit Yazi with `q`, your shell cds to wherever you navigated. Quitting with `Q` exits without syncing the directory.

Configuration lives in `~/.config/yazi/`:

- `yazi.toml` – General settings (layout ratio 2:4:3, hidden files shown, alphabetical sort, directories first)
- `keymap.toml` – All keybindings (Vim-style, heavily customized)
- `theme.toml` – Color theme
- `package.toml` – Plugin packages

The layout uses a three-pane view: parent directory (left, 2 parts), current directory (center, 4 parts), preview (right, 3 parts). Hidden files are shown by default. Sort is alphabetical, case-insensitive, with directories listed first.

Core Concepts

The Three-Pane Layout

Yazi displays three columns: the parent directory on the left, the current directory in the center, and a preview of the hovered file on the right. You navigate by moving the cursor in the center pane. Moving into a child directory shifts all panes left; moving to the parent shifts them right.

Vim-Style Modes

Yazi uses modal navigation like Vim:

- **Normal mode:** Default state. Navigate, select, and operate on files.
- **Visual mode:** Enter with `v`. Move the cursor to extend a selection range.
- **Input mode:** Activated by commands like `a` (create), `r` (rename), `;` (shell). Has its own Vim-like normal/insert sub-modes.

The Two Quit Commands

Key	Action	Shell CWD
<code>q</code>	Quit Yazi	Shell changes to Yazi's last directory
<code>Q</code>	Quit without syncing	Shell stays in the directory it was before

This distinction is critical. Use `q` when you navigated to a directory and want your shell to follow. Use `Q` when you were just browsing.

Essential Commands

Movement

Key	Action
<code>j</code>	Next file (down)
<code>k</code>	Previous file (up)
<code>h</code>	Parent directory (left)
<code>l</code>	Enter directory / open file (right)
<code>Ctrl+U</code>	Half page up
<code>Ctrl+D</code>	Half page down
<code>Ctrl+B</code>	Full page up
<code>Ctrl+F</code>	Full page down
<code>gg</code>	Go to first file
<code>G</code>	Go to last file
<code>H</code>	Go back (directory history)
<code>L</code>	Go forward (directory history)
<code>K</code>	Seek preview up 5 lines
<code>J</code>	Seek preview down 5 lines

Selection

Key	Action
<code>Space</code>	Toggle selection on current file, move down
<code>Ctrl+A</code>	Select all files
<code>Ctrl+R</code>	Invert selection
<code>v</code>	Enter visual mode (extend selection by moving)

Key	Action
V	Enter visual mode (unset mode)
Esc	Clear selection / exit visual mode

File Operations

Key	Action
o	Open selected files
O	Open selected files (choose program interactively)
Enter	Open selected files
y	Yank (copy) selected files
x	Cut selected files
p	Paste yanked/cut files
P	Paste (overwrite if exists)
Y	Cancel yank
X	Cancel yank
d	Move to trash
D	Permanently delete
a	Create file or directory (append / for directory)
r	Rename (cursor before extension)
—	Create symlink (absolute path)
—	Create symlink (relative path)

Search and Find

Key	Action
s	Search by filename via fd
S	Search by content via ripgrep
Ctrl+S	Cancel ongoing search
z	Jump to file/directory via fzf
Z	Jump to directory via zoxide
f	Filter current directory (live, narrows visible files)
/	Find next (highlight matches in current directory)
?	Find previous
n	Go to next match
N	Go to previous match

The distinction between search, find, and filter:

- **Search** (s/S): Recursively searches the entire subtree using fd or rg. Results replace the file list.
- **Filter** (f): Narrows the current directory view in real time as you type. Does not recurse.
- **Find** (/?): Highlights matching files in the current directory and jumps between them with n/N. Does not hide non-matches.

Goto Shortcuts

Key Sequence	Destination
gh	Home directory (~)
gc	Config directory (~/.config)
gd	Downloads directory (~/.Downloads)
g Space	Interactive directory jump (cd prompt)
gf	Follow hovered symlink to its target

Tabs

Key	Action
t	Create new tab at current directory
1-9	Switch to tab by number
[	Switch to previous tab
]	Switch to next tab
{	Swap current tab with previous
}	Swap current tab with next
Ctrl+C	Close current tab (or quit if last)

Copy Paths

Key Sequence	What Is Copied
cc	Full file path
cd	Directory path (dirname)
cf	Filename with extension
cn	Filename without extension

Sorting

Key Sequence	Sort By	Reverse
,m	Modified time	,M
,b	Birth/creation time	,B
,s	File size	,S
,a	Alphabetical	,A
,n	Natural order	,N
,e	Extension	,E
,r	Random	—

Sorting by time or size also sets the linemode to display that metadata alongside filenames.

Linemode (Metadata Display)

Key Sequence	Shows
ms	File size
mp	Permissions
mb	Birth/creation time
mm	Modified time
mo	Owner
mn	None (reset)

Other

Key	Action
.	Toggle hidden files
;	Run shell command (interactive)
:	Run shell command (blocking, waits for exit)
Tab	Spot (preview) the hovered file
w	Show task manager
~ or F1	Open help
Ctrl+Z	Suspend Yazi (return to shell, fg to resume)

Practical Recipes

Browse and land in a directory

```
y                                # Launch Yazi
# Navigate with h/j/k/l to the target directory
# Press q to quit — your shell is now in that directory
```

Quick file preview without opening

Navigate to any file. The right pane shows a preview automatically. For a larger preview, press Tab to open the spotter view. Yazi previews text files with syntax highlighting, images inline (in supported terminals like iTerm2), PDFs, archives, and more.

Bulk rename files

1. Select files with Space (one by one) or v (visual range) or Ctrl+A (all).
2. Press r to rename. For a single file, this opens an inline rename prompt with the cursor positioned before the extension.

Move files between directories using tabs

1. Press t to open a new tab.
2. Navigate to the destination directory in the new tab.
3. Press [to go back to the source tab.

4. Select files with Space.
5. Press x to cut (or y to copy).
6. Press] to go to the destination tab.
7. Press p to paste.

Search for a file deep in the tree

Press s to search by filename (uses fd). Type part of the filename. The results replace the file list. Navigate to the desired file and press Enter to open it or Ctrl+S to cancel and return to the normal view.

Search file contents

Press S to search by content (uses rg). Type a search pattern. Yazi shows files containing that pattern. This is useful when you know what a file contains but not its name.

Jump to a frequently visited directory

Press Z to invoke zoxide. Type part of the directory name. Zoxide uses its frequency database to find the best match and jumps there.

Jump using fzf

Press z to invoke the fzf plugin. This presents a fuzzy finder over the directory tree, letting you type to narrow down and jump to any file or directory.

Filter the current directory

Press f and start typing. The file list narrows in real time to show only matching entries. This is useful in large directories when you know roughly what you are looking for. Press Esc to clear the filter.

Run a shell command on selected files

1. Select files with Space.
2. Press ; for an interactive shell prompt.
3. Type your command. Use \$@ to refer to selected files.

Or press : for a blocking command that shows output and waits for you to press Enter.

Copy a file path to clipboard

Navigate to the target file, then press cc. The full path is now in your system clipboard, ready to paste into a terminal, editor, or chat.

Check file sizes in a directory

Press ,s to sort by size (largest first shows at bottom; use ,S for reverse). The linemode automatically switches to show sizes next to each filename.

Advanced Usage

The Shell Wrapper in Detail

The `y()` function is essential for Yazi integration with the shell. Without it, Yazi runs as a subprocess and any directory changes inside it are lost when it exits. The wrapper:

1. Creates a temporary file for CWD communication.
2. Launches Yazi with `--cwd=file` pointing to that temp file.
3. On exit, reads the temp file to get Yazi's last directory.
4. If the directory is valid and different from the current one, `cds` to it.
5. Cleans up the temp file.

The builtin `cd` call (not the `zoxide`-enhanced `cd`) is used intentionally to avoid adding the intermediate navigation to `zoxide`'s database.

Yazi's Search Tools

Yazi delegates search to external tools rather than implementing its own:

Yazi Key	External Tool	What It Searches
s	fd	Filenames, recursively
S	rg (ripgrep)	File contents, recursively
z	fzf	Filenames, interactively
Z	zoxide	Directory frequency database
f	Built-in	Current directory only (live filter)
/	Built-in	Current directory only (highlight matches)

This means Yazi benefits from the same tools you use on the command line, with the same speed and `.gitignore` awareness.

Task Manager

Press `w` to open the task manager. Yazi runs file operations (copy, move, delete, search) as background tasks. In the task manager:

Key	Action
j/k	Navigate tasks
Enter	Inspect task details
x	Cancel a task
w	Close task manager

Opener Configuration

Yazi opens files based on MIME type rules defined in `yazi.toml`. The default opener for text files uses `$EDITOR` (Neovim in this setup). Images open with the system default (open on macOS), media files play with the default player, and archives can be extracted.

You can press O (capital) to choose from available openers interactively rather than using the default.

Spot Mode

Press Tab to enter spot mode for the hovered file. This opens an expanded preview where you can scroll through the file content. In spot mode:

Key	Action
j/k	Scroll content up/down
h/l	Swipe to previous/next file
Tab	Close spot mode
cc	Copy selected cell content

Input Mode (Rename, Create, Shell)

When you press a (create), r (rename), or ;/: (shell), Yazi enters input mode with Vim keybindings:

Key	Action
i	Enter insert mode
a	Enter append mode
Esc	Back to normal mode or cancel
Enter	Submit input
Ctrl+C	Cancel
h/l	Move cursor left/right
w/b	Move by word forward/backward
0/\$	Move to beginning/end of line
d	Cut selection
y	Copy selection
p/P	Paste after/before cursor
u	Undo
Ctrl+R	Redo

Configuration Highlights

From `yazi.toml`, notable settings in this setup:

Setting	Value	Meaning
ratio	[2, 4, 3]	Column widths: parent 2, current 4, preview 3
show_hidden	true	Hidden files visible by default
sort_by	alphabetical	Default sort order
sort_dir_first	true	Directories appear before files
scrolloff	5	Keep 5 lines visible above/below cursor

Setting	Value	Meaning
title_format	Yazi: {cwd}	Window title shows current directory

Neovim Fundamentals

The language of vim is a grammar: verbs act on nouns, and fluency comes from combining a small set of each.

Your Setup

zero.dev001 uses Neovim (aliased as vim and v) with the LazyVim distribution. But the power of Neovim does not come from plugins – it comes from the modal editing model that has existed since the 1970s. Every plugin, every custom mapping, and every workflow in the following chapters builds on the fundamentals covered here. Learn these, and every vim-based tool (lazygit, Yazi, man pages, even your shell with vi mode) becomes immediately more productive.

Neovim is launched from the shell:

```
$ nvim .           # Open Neovim in current directory
$ nvim file.ts     # Open a specific file
$ v file.ts        # Same thing, via alias
```

Multiple Neovim configurations are available through NVIM_APPNAME:

```
$ vtest           # nvim-test config
$ vkick           # nvim-kickstart config
$ vlazy           # nvim-lazyvim config
$ vmin            # nvim-minimal config
```

Core Concepts

Modal Editing

Vim is a modal editor. The same key does different things depending on which mode you are in. This is the single most important concept to internalize.

Mode	Purpose	How to Enter	How to Leave
Normal	Navigate and manipulate text	Esc or Ctrl+[	Enter another mode
Insert	Type text	i, a, o, I, A, O	Esc

Mode	Purpose	How to Enter	How to Leave
Visual	Select text	v, V, Ctrl+V	Esc
Command	Execute commands	:	Enter or Esc
Replace	Overwrite text character by character	R	Esc

Normal mode is the home mode. You should spend most of your time there. If you are ever lost, press Esc until you are back in Normal mode.

The Insert Mode Entry Points

Each key that enters Insert mode places your cursor in a different position:

Key	Cursor Position
i	Before the character under the cursor
a	After the character under the cursor
I	At the first non-blank character of the line
A	At the end of the line
o	New line below, in insert mode
O	New line above, in insert mode
s	Delete character under cursor, enter insert
S	Delete entire line, enter insert

Essential Commands

Movement – Characters and Lines

Key	Motion
h	One character left
j	One line down
k	One line up
l	One character right
0	Beginning of line (column 0)
^	First non-blank character of line
\$	End of line
g_	Last non-blank character of line

Movement – Words

Key	Motion
w	Start of next word
b	Start of previous word

Key	Motion
e	End of current/next word
ge	End of previous word
W	Start of next WORD (whitespace-delimited)
B	Start of previous WORD
E	End of current/next WORD

A “word” is a sequence of letters, digits, and underscores. A “WORD” is any sequence of non-blank characters. In `my—variable`, `w` stops at `—` and `v`, while `W` jumps over the whole thing.

Movement – Vertical

Key	Motion
gg	First line of file
G	Last line of file
{count}G	Go to line number
Ctrl+D	Half-page down
Ctrl+U	Half-page up
Ctrl+F	Full page forward
Ctrl+B	Full page backward
{	Previous blank line (paragraph boundary)
}	Next blank line
%	Matching bracket/paren/brace

Movement – Find on Line

Key	Motion
f{char}	Jump forward to {char} on current line
F{char}	Jump backward to {char} on current line
t{char}	Jump forward to one before {char}
T{char}	Jump backward to one after {char}
;	Repeat last f/F/t/T forward
,	Repeat last f/F/t/T backward

The `f` and `t` commands are among the fastest ways to move within a line. To delete everything up to a comma: `dt,.` To change a function argument: `ct)`.

Operators

Operators are the verbs of the vim grammar. They act on a motion or text object.

Operator	Action
d	Delete (cut into register)

Operator	Action
c	Change (delete and enter Insert mode)
y	Yank (copy into register)
>	Indent right
<	Indent left (dedent)
=	Auto-format/indent
gu	Convert to lowercase
gU	Convert to uppercase
g~	Toggle case
!	Filter through external program

Double an operator to act on the entire current line: dd deletes a line, yy yanks a line, cc changes a line, >> indents a line, == auto-indents a line.

Text Objects

Text objects are the nouns of the vim grammar. They define a region of text. Every text object comes in two flavors: **inner** (i) excludes delimiters, **a/around** (a) includes them.

Text Object	Inner (i)	Around (a)
w	Inner word	Word plus trailing space
W	Inner WORD	WORD plus trailing space
s	Inner sentence	Sentence plus trailing space
p	Inner paragraph	Paragraph plus surrounding blanks
"	Inside double quotes	Quotes included
'	Inside single quotes	Quotes included
`	Inside backticks	Backticks included
(/)/b	Inside parentheses	Parentheses included
[/]	Inside square brackets	Brackets included
{/}/B	Inside curly braces	Braces included
</>	Inside angle brackets	Brackets included
t	Inside HTML/XML tags	Tags included

Practical Recipes

The Vim Grammar in Action

Every editing command is a sentence: **operator** + **motion** or **operator** + **text object**. Once you learn the grammar, you can construct commands you have never seen before.

Command	Reads As	Effect
dw	Delete word (from cursor)	Deletes to start of next word
diw	Delete inner word	Deletes the word under cursor

Command	Reads As	Effect
daw	Delete a word	Deletes word plus surrounding space
ci "	Change inside quotes	Clears quoted string, enters Insert
ca(	Change around parens	Clears parens and contents
yap	Yank around paragraph	Copies paragraph to register
dit	Delete inside tag	Clears HTML tag contents
>ip	Indent inner paragraph	Indents the current paragraph
guiw	Lowercase inner word	Converts word to lowercase
gUaw	Uppercase a word	Converts word to uppercase
=aB	Format around block	Auto-indents the current {} block
da[	Delete around brackets	Removes [...] entirely
yi(	Yank inside parens	Copies contents of parentheses

Counts as Multipliers

Prefix any motion or command with a number to repeat it:

Command	Effect
3w	Move forward 3 words
5j	Move down 5 lines
2dd	Delete 2 lines
3yy	Yank 3 lines
4>>	Indent 4 lines
50G	Go to line 50

Registers

Every delete and yank goes into a register. Registers are vim's clipboard system.

Register	Contents
"	Default register (last delete/yank)
0	Last yank (not affected by deletes)
1-9	Delete history (1 is most recent)
a-z	Named registers (you choose)
A-Z	Append to named register (lowercase letter)
+	System clipboard
*	Primary selection (X11) / same as + on macOS
—	Black hole register (delete without saving)
/	Last search pattern
.	Last inserted text
:	Last command-line command

Using registers:

Command	Effect
"ayy	Yank current line into register a
"ap	Paste from register a
"+y	Yank to system clipboard
"+p	Paste from system clipboard
"_dd	Delete line without saving to any register
:reg	Show all register contents
:reg a	Show contents of register a

Marks

Marks let you bookmark positions in a file and jump back to them.

Command	Effect
m{a–z}	Set a mark at cursor position (file-local)
m{A–Z}	Set a global mark (across files)
'{ letter }	Jump to the line of a mark
`{ letter }	Jump to the exact position of a mark
`0	Position where you last exited Neovim
`.`	Position of last change
	List all marks
`  Position before last jump    :marks`	

Macros

Macros record a sequence of keystrokes and replay them. They are the most powerful repetition tool in vim.

Command	Effect
q{a–z}	Start recording macro into register
q	Stop recording
@{a–z}	Play macro from register
@@	Repeat the last played macro
5@a	Play macro a five times

A practical macro workflow:

1. Move to the first line you want to change
2. Press qa to start recording into register a
3. Make your edit on the current line
4. Press j to move to the next line (so the macro is repeatable)
5. Press q to stop recording
6. Press @a to replay once, then @@ or 10@@ to repeat

Advanced Usage

Search and Substitution

Command	Effect
/ {pattern}	Search forward for pattern
? {pattern}	Search backward for pattern
n	Next match (same direction)
N	Next match (opposite direction)
*	Search forward for word under cursor
#	Search backward for word under cursor
:%s/old/new/g	Replace all occurrences in file
:%s/old/new/gc	Replace all with confirmation
:%s/old/new/gi	Replace all, case insensitive
:5,20s/old/new/g	Replace in lines 5-20
: '<,'>s/old/new/g	Replace in visual selection
:s/old/new/g	Replace on current line

Substitution flags:

Flag	Meaning
g	Global – all occurrences on each line
c	Confirm each substitution
i	Case insensitive
I	Case sensitive (overrides ignorecase)
n	Count matches without substituting

Visual Mode

Visual mode lets you select text and then act on the selection.

Key	Mode
v	Character-wise visual
V	Line-wise visual
Ctrl+V	Block-wise visual (column selection)
gv	Re-select the last visual selection
o	Jump to other end of selection

Once a visual selection is active, any operator acts on it: d deletes, c changes, y yanks, > indents, u lowercases, U uppercases.

Block visual mode (Ctrl+V) is uniquely powerful for columnar edits:

1. Ctrl+V to enter block visual mode
2. Select a column of text with j/k and l/h
3. I to insert at the start of each line, or A to append
4. Type your text, then press Esc – the edit applies to every line

The Dot Command

The `.` key repeats the last change. A “change” is anything from entering Insert mode to returning to Normal mode, or any Normal mode command that modifies text.

Effective use of `.`:

- `ciw` + type replacement + Esc, then `n.n.n.` to search and replace interactively
- `dd` to delete a line, then `.` to delete the next one
- `>>` to indent a line, then `j.j.j.` to indent several more
- `A;` + Esc to append a semicolon, then `j.j.` to do the same on following lines

The dot command is the reason vim users structure their edits as repeatable atomic operations. A single `ciw` followed by dots is faster than four separate manual edits.

Undo and Redo

Command	Effect
<code>u</code>	Undo last change
<code>Ctrl+R</code>	Redo (undo the undo)
<code>U</code>	Undo all changes on the current line

Neovim supports undo branches – if you undo several times and then make a new change, the undone changes are not lost. Use `:earlier 5m` to go back to the state 5 minutes ago, or `:later 5m` to go forward.

Window Splits

Command	Effect
<code>Ctrl+W s</code>	Horizontal split
<code>Ctrl+W v</code>	Vertical split
<code>Ctrl+W q</code>	Close current split
<code>Ctrl+W o</code>	Close all other splits
<code>Ctrl+W =</code>	Equalize split sizes
<code>Ctrl+H/J/K/L</code>	Navigate between splits (LazyVim default)

Command-Line Mode Essentials

Command	Effect
<code>:w</code>	Save
<code>:q</code>	Quit (fails if unsaved changes)
<code>:wq / :x</code>	Save and quit
<code>:q!</code>	Quit without saving
<code>:e { file }</code>	Open file
<code>:{ number }</code>	Go to line number
<code>!{ cmd }</code>	Run shell command

Command	Effect
<code>:r !{cmd}</code>	Insert shell command output
<code>:%!{cmd}</code>	Filter entire file through command

This chapter covers the timeless vim fundamentals that apply regardless of your Neovim configuration. Master the grammar – operators, motions, text objects – and the rest of this book’s editor chapters become intuitive extensions of what you already know.

LazyVim Distribution

A batteries-included Neovim configuration that provides IDE features out of the box while staying fast and extensible.

Your Setup

zero.dev001's Neovim runs the LazyVim distribution – a pre-configured Neovim setup built on the lazy.nvim plugin manager. LazyVim provides sensible defaults for dozens of plugins, a consistent keymap system organized around the Space leader key, and an “extras” system for opting into language-specific tooling.

The configuration lives at:

Source: `~/.local/share/chezmoi/dot_config/nvim/`
Deployed: `~/.config/nvim/`

The directory structure follows LazyVim conventions:

```
~/.config/nvim/  
  init.lua           — Entry point , loads lazy.nvim  
  lua/  
    config/  
      autocmds.lua   — Custom autocommands  
      keymaps.lua    — Custom key mappings  
      lazy.lua        — lazy.nvim bootstrap and setup  
      options.lua     — Neovim options  
    plugins/  
      99.lua          — AI plugin (99.nvim)  
      auto-save.lua   — Auto-save on buffer changes  
      conform.lua     — Formatter configuration  
      dadbod.lua      — Database UI  
      hardtime.lua    — Vim habit trainer  
      img-clip.lua    — Image paste from clipboard  
      stay-centered.lua — Keep cursor centered  
      vim-visual-multi.lua — Multiple cursors  
      example.lua     — LazyVim example (disabled)
```

The bootstrap in lazy.lua loads LazyVim first, then imports custom plugins:

```
require("lazy").setup({
  spec = {
    { "LazyVim/LazyVim", import = "lazyvim.plugins" },
    { import = "plugins" },
  },
})
```

Plugin updates are checked automatically but silently – the checker runs periodically with `notify = false`.

Core Concepts

The Leader Key

The leader key is **Space**. Nearly every LazyVim command starts with it. When you press Space and pause, **which-key** pops up a floating window showing every available mapping and its description. This means you never need to memorize keybindings from scratch – press Space, read the menu, and pick the action you want.

The which-key groups are organized mnemonically:

Prefix	Group
<leader>f	Find/File (telescope)
<leader>s	Search (grep, symbols)
<leader>b	Buffer
<leader>c	Code (LSP actions)
<leader>g	Git
<leader>u	UI (toggles)
<leader>x	Diagnostics (trouble)
<leader>w	Window
<leader>l	Lazy (plugin manager)
<leader>q	Quit/Session

lazy.nvim Package Manager

lazy.nvim is the plugin manager underneath LazyVim. It handles downloading, updating, lazy-loading, and profiling plugins.

Key features: - **Lazy-loading**: Plugins load on demand – on specific events, commands, keymaps, or filetypes. This keeps startup fast. - **Lockfile**: `lazy-lock.json` pins every plugin to a specific commit, ensuring reproducible installs. - **Profiling**: The Lazy UI shows exactly how long each plugin takes to load.

The Extras System

LazyVim provides “extras” – optional plugin bundles for languages, tools, and UI features. Extras are imported in the plugin spec:

```
{ import = "lazyvim.plugins.extras.lang.typescript" }
{ import = "lazyvim.plugins.extras.lang.json" }
{ import = "lazyvim.plugins.extras.ui.mini-starter" }
```

You can browse and toggle extras from the Lazy UI with `<leader>l` then navigating to the Extras tab.

Essential Commands

File Finding (Telescope)

Key	Action
<code><leader>ff</code>	Find files in project
<code><leader>fr</code>	Find recent files
<code><leader>fg</code>	Live grep across project
<code><leader>fb</code>	Find open buffers
<code><leader>fc</code>	Find Neovim config files
<code><leader>fp</code>	Find plugin files
<code><leader>f/</code>	Search in current buffer
<code><leader><space></code>	Find files (alias for <code><leader>ff</code>)

Inside a Telescope picker:

Key	Action
<code>Ctrl+J</code> / <code>Ctrl+K</code>	Move up/down in results
<code>Enter</code>	Open selected file
<code>Ctrl+X</code>	Open in horizontal split
<code>Ctrl+V</code>	Open in vertical split
<code>Ctrl+T</code>	Open in new tab
<code>Esc</code>	Close picker (in Normal mode)

File Explorer (neo-tree)

Key	Action
<code><leader>e</code>	Toggle file explorer (focus)
<code><leader>E</code>	Toggle file explorer (root dir)

Inside neo-tree:

Key	Action
<code>Enter</code>	Open file/expand directory

Key	Action
a	Add file (end with / for directory)
d	Delete
r	Rename
c	Copy
m	Move
y	Copy file name
Y	Copy relative path
H	Toggle hidden files
/	Filter/search
q	Close

Buffer Management

Key	Action
<leader>bb	Switch buffer (picker)
<leader>bd	Delete current buffer
<leader>bo	Delete other buffers
H	Previous buffer
L	Next buffer
<leader>,	Switch buffer (alternative)

Search and Replace

Key	Action
<leader>sg	Grep across project (Telescope)
<leader>sw	Search word under cursor
<leader>ss	Search document symbols
<leader>sS	Search workspace symbols
<leader>sr	Search and replace (grug-far)
<leader>sd	Search diagnostics
<leader>sh	Search help tags
<leader>sk	Search keymaps
<leader>sm	Search marks

Navigation

Key	Action
s	Flash jump (type 2 chars to jump)
S	Flash treesitter selection
Ctrl+H	Navigate to left split/tmux pane
Ctrl+J	Navigate to split/tmux pane below
Ctrl+K	Navigate to split/tmux pane above

Key	Action
Ctrl+L	Navigate to right split/tmux pane

The Ctrl+H/J/K/L bindings work seamlessly between Neovim splits and tmux panes via **vim-tmux-navigator**. You never need to think about whether you are crossing a Neovim split boundary or a tmux pane boundary – the same keys handle both.

Git Integration

Key	Action
<leader>gg	Open lazygit
<leader>gf	Lazygit current file history
<leader>gb	Git blame line
<leader>gB	Git browse (open in browser)
]h	Next git hunk
[h	Previous git hunk
<leader>ghp	Preview hunk
<leader>ghs	Stage hunk
<leader>ghr	Reset hunk

UI Toggles

Key	Action
<leader>uf	Toggle auto-format
<leader>uw	Toggle word wrap
<leader>ul	Toggle line numbers
<leader>uL	Toggle relative line numbers
<leader>us	Toggle spell check
<leader>ud	Toggle diagnostics
<leader>uc	Toggle conceal
<leader>uh	Toggle inlay hints

Practical Recipes

Quick File Navigation Workflow

The fastest way to open files depends on context:

1. **You know the filename:** <leader>ff, type part of the name, Enter.
2. **You were just editing it:** <leader>fr for recent files.
3. **You need to find text:** <leader>sg to grep, then open the result.
4. **You want to browse the tree:** <leader>e to open neo-tree.
5. **You are switching between two files:** H and L to bounce between buffers.

Project-Wide Search and Replace

Using grug-far (<leader>sr):

1. Press <leader>sr to open the search and replace panel.
2. Type the search pattern in the first field.
3. Type the replacement in the second field.
4. Preview results in the buffer below.
5. Apply replacements selectively or globally.

This is significantly more ergonomic than :s commands for multi-file replacements.

Flash Jump

Flash replaces the traditional f/t motions with a two-character search:

1. Press s in Normal mode.
2. Type the first character of your target – labels appear on all matches.
3. Type the label character to jump directly there.

Flash also works as an operator motion: ds{char}{label} deletes from cursor to the labeled position. ys{char}{label} yanks from cursor to the label.

Managing the Plugin System

Command	Effect
<leader>l	Open Lazy plugin manager UI
:Lazy sync	Update all plugins
:Lazy profile	Show startup profiling
:Lazy health	Check plugin health

Inside the Lazy UI: - **S** to sync (install/update/clean) - **U** to update all - **X** to clean removed plugins - **p** to show profiling - **i** to install missing plugins

Overriding LazyVim Defaults

To override a LazyVim plugin, create a file in lua/plugins/ that returns a spec for the same plugin. The opts tables are deep-merged:

```
— lua/plugins/my-override.lua
return {
  "nvim-telescope/telescope.nvim",
  opts = {
    defaults = {
      layout_strategy = "horizontal",
    },
  },
}
```

To disable a LazyVim plugin entirely:

```

return {
  { "some/plugin.nvim", enabled = false },
}

```

Advanced Usage

The Plugin Spec Format

Every plugin in lazy.nvim is defined by a spec table. The most common fields:

Field	Purpose
[1] (string)	Plugin repository ("user/repo")
dir	Local directory path (for local plugins)
opts	Options table passed to plugin.setup()
config	Function called when plugin loads
keys	Keymaps (also triggers lazy-loading)
cmd	Commands (triggers lazy-loading)
event	Events (triggers lazy-loading)
ft	Filetypes (triggers lazy-loading)
dependencies	Plugins loaded before this one
lazy	Boolean to force lazy-load behavior
enabled	Boolean or function to enable/disable
branch	Git branch to use

Performance Notes

LazyVim with all of zero.dev001's custom plugins starts in under 100ms on modern hardware. The key performance strategies:

- Most custom plugins are lazy-loaded via event, cmd, or keys
- The checker runs in the background with notifications disabled
- Disabled runtime plugins: gzip, tarPlugin, tohtml, tutor, zipPlugin
- lazy.nvim caches module resolution for faster subsequent loads

Session Management

Key	Action
<leader>qs	Restore session for current directory
<leader>ql	Restore last session
<leader>qd	Quit and delete session
<leader>qq	Quit all

LazyVim auto-saves sessions so you can close Neovim and reopen exactly where you left off.

Startup Sequence

When Neovim launches, the following happens in order:

1. `init.lua` runs, loading `config/lazy.lua`
2. `lazy.nvim` bootstraps itself if not installed
3. LazyVim core plugins load (`which-key`, `telescope`, `neo-tree`, etc.)
4. Custom plugins from `lua/plugins/` are merged and loaded
5. `config/options.lua`, `config/keymaps.lua`, and `config/autocmds.lua` execute on `VeryLazy` event
6. Lazy-loaded plugins wait for their trigger conditions

Understanding this sequence matters when debugging load order issues or when a keymap needs to override a plugin's default.

Custom Plugins

Beyond LazyVim defaults: eight handpicked plugins that fill specific gaps in the editing workflow.

Your Setup

LazyVim ships with a curated set of plugins – Telescope, neo-tree, Flash, which-key, and many more. zero.dev001 adds eight additional plugins on top of the LazyVim base to handle AI assistance, auto-saving, formatting, database work, habit training, image pasting, scroll behavior, and multiple cursors.

Each plugin is defined in its own file under lua/plugins/. This keeps the configuration modular – you can disable any plugin by deleting its file or adding enabled = false.

Plugin source files:

```
~/ .config/nvim/lua/plugins/  
  99.lua                — AI integration  
  auto-save.lua         — Automatic saving  
  conform.lua           — Code formatting  
  dadbod.lua            — Database UI  
  hardtime.lua          — Vim habit trainer  
  img-clip.lua          — Image clipboard paste  
  stay-centered.lua     — Cursor centering  
  vim-visual-multi.lua  — Multiple cursors
```

Core Concepts

Plugin Loading Strategies

Each plugin uses a different lazy-loading strategy suited to when it is needed:

Plugin	Loading Strategy	Rationale
99.nvim	On keymap (keys)	Only load when AI is invoked
auto-save.nvim	On event (InsertLeave, TextChanged)	Active whenever editing

Plugin	Loading Strategy	Rationale
conform.nvim	LazyVim default + custom autocmd	Formatting on buffer leave
dadbod	On command (DBUI, DBUIToggle)	Only when database work begins
hardtime.nvim	Immediate (no lazy config)	Must be active from start
img-clip.nvim	On VeryLazy event	Available after startup
stay-centered.nvim	Immediate	Must be active from start
vim-visual-multi	On VeryLazy event	Available after startup

Local Plugins

The 99.nvim plugin is loaded from a local directory rather than a Git repository. Instead of specifying a "user/repo" string, it uses the dir field:

```
{
  dir = vim.fn.expand("~/Developer/zero/workspace/99"),
}
```

This means the plugin code lives alongside other development projects and can be edited and tested in real time without publishing to GitHub.

Essential Commands

99.nvim – AI Integration

Key	Mode	Action
<leader>mm	Normal	Open model selection picker
<leader>9v	Visual	Send selection to AI for processing
<leader>9s	Normal	Stop all active AI requests

Configuration details:

Setting	Value
Provider	ClaudeCodeProvider
Model	opus-4.6
Source	~/Developer/zero/workspace/99

The plugin uses the ClaudeCodeProvider from the 99.providers module, connecting to the Claude API. The model can be changed at runtime with <leader>mm, which presents a picker of available models.

Workflow: 1. Select code in Visual mode (v or V) 2. Press <leader>9v to invoke AI on the selection 3. The AI processes the selection and returns a result 4. If a request is taking too long, press <leader>9s to cancel

auto-save.nvim – Automatic Saving

Trigger	Behavior
InsertLeave	Save when exiting Insert mode
TextChanged	Save when text changes in Normal mode

This plugin requires zero configuration – it is a single spec with empty opts:

```
{
  "okuuuu/auto-save.nvim",
  event = { "InsertLeave", "TextChanged" },
  opts = {},
}
```

The effect is that you never need to manually :w. The moment you stop typing or leave Insert mode, the buffer is saved. This pairs naturally with the conform.nvim formatting-on-BufLeave strategy: you edit, auto-save fires, you switch buffers, conform formats the buffer you just left.

conform.nvim – Code Formatting

Trigger	Behavior
BufLeave	Format when switching away from a buffer

Key configuration choices:

Setting	Value	Rationale
format_on_save	false	No formatting on :w
format_after_save	false	No formatting after :w
async	true	Non-blocking formatting
lsp_fallback	true	Use LSP if no formatter matches

The custom autocmd only triggers formatting when the buffer has been modified (`vim.bo[args.buf].modified`), avoiding unnecessary formatter invocations on clean buffers.

This approach means formatting never interrupts your typing flow. You type, auto-save preserves your changes, and when you move to another buffer or file, conform quietly formats what you left behind.

dadbod – Database UI

Key	Mode	Action
<leader>D	Normal	Toggle the database UI panel

Supporting commands:

Command	Action
:DBUI	Open the database UI
:DBUIToggle	Toggle the database UI
:DBUIAddConnection	Add a new database connection
:DBUIFindBuffer	Find the DBUI buffer

The dadbod stack consists of three plugins working together:

Plugin	Role
tpope/vim-dadbod	Core database interface
kristijanhusak/vim-dadbod-ui	Visual UI for browsing/querying
kristijanhusak/vim-dadbod-completion	SQL autocompletion

Supported filetypes for completion: sql, mysql, plsql.

The UI uses Nerd Font icons (`vim.g.db_ui_use_nerd_fonts = 1`) for a clean visual presentation of database objects.

Workflow: 1. Press `<leader>D` to open the database UI 2. Add a connection string (e.g., `postgres://user:pass@localhost/dbname`) 3. Browse tables, views, and schemas in the left panel 4. Write and execute queries in the right panel 5. Results appear inline below your query

hardtime.nvim – Vim Habit Trainer

This plugin has no keybindings of its own. Instead, it monitors your editing behavior and discourages inefficient patterns.

What hardtime restricts:

Bad Habit	What Happens
Holding j or k	Key is blocked after repeated presses
Using arrow keys	Keys are disabled
Repeated h/l	Blocked – use w, b, f instead
Using x repeatedly	Suggests dw or diw instead

The plugin uses `nui.nvim` to display hints about better alternatives. The goal is to build muscle memory for efficient vim motions. Over time, the restrictions train you to reach for word motions (w, b, e), find motions (f, t), and text objects (ciw, da()) instead of character-by-character navigation.

img-clip.nvim – Image Clipboard Paste

Key	Mode	Action
<code><leader>p</code>	Normal	Paste image from system clipboard

Configuration:

Setting	Value	Effect
<code>embed_image_as_base64</code>	false	Save as file, not inline data
<code>prompt_for_file_name</code>	false	Auto-generate filename
<code>drag_and_drop.insert_mode</code>	true	Support drag-and-drop in Insert
<code>use_absolute_path</code>	false	Use relative paths in links

Workflow: 1. Copy an image to the system clipboard (screenshot, browser image, etc.) 2. Position the cursor where you want the image reference 3. Press `<leader>p` 4. The image is saved to disk and a markdown/markup reference is inserted

This is particularly useful when writing documentation or markdown files – screenshot something, switch to Neovim, and paste.

stay-centered.nvim – Cursor Centering

This plugin has no keybindings. It automatically keeps the cursor line vertically centered in the window as you scroll. This is equivalent to having `scrolloff` set to a very high value, but implemented more smoothly.

The effect: as you move through a file with `j`, `k`, `Ctrl+D`, `Ctrl+U`, or any other motion, the screen scrolls to keep your cursor in the middle of the viewport. You always see equal amounts of context above and below your current position.

```
{
  "arnamak/stay-centered.nvim",
  opts = {},
}
```

vim-visual-multi – Multiple Cursors

Key	Mode	Action
<code>Ctrl+N</code>	Normal	Add cursor on current word
<code>n</code>	Multi	Add next occurrence
<code>N</code>	Multi	Add previous occurrence
<code>q</code>	Multi	Skip current and move to next
<code>[/]</code>	Multi	Navigate between cursors
<code>Ctrl+Down</code>	Normal	Add cursor below
<code>Ctrl+Up</code>	Normal	Add cursor above
<code>Tab</code>	Multi	Switch between cursor and extend mode

Workflow for renaming a variable: 1. Place cursor on the variable name 2. Press `Ctrl+N` to select the word and enter multi-cursor mode 3. Press `n` repeatedly to select additional occurrences 4. Press `q` to skip any occurrence you do not want to change 5. Press `c` to change all selected occurrences simultaneously 6. Type the new name 7. Press `Esc` to exit

This is faster than search-and-replace for targeted, visual renaming where you want to confirm each occurrence.

Practical Recipes

The Auto-Save and Format Pipeline

The interaction between auto-save and conform creates a seamless editing pipeline:

Type code → Leave Insert mode → auto-save fires → buffer saved
Switch to another buffer → BufLeave fires → conform formats async
Return to original buffer → see formatted code

This means you never think about saving or formatting. Both happen automatically as a side effect of normal editing behavior.

Database Exploration with Dadbod

A typical database exploration session:

1. Open the UI: <leader>D
2. Add connection: type the connection string in the prompt
3. Expand the connection in the tree to see schemas and tables
4. Press Enter on a table to see its structure
5. Open a new query buffer and write SQL
6. Execute the query with the dadbod execute command
7. Results render in a split below

The completion plugin provides table names, column names, and SQL keywords as you type in query buffers.

Multi-Cursor Editing Patterns

Add the same text to multiple lines:

1. Move to the first target line
2. Ctrl+Down repeatedly to add cursors on consecutive lines
3. I to enter Insert mode at line start (or A for line end)
4. Type your addition
5. Esc to apply to all lines

Change a function signature across calls:

1. Place cursor on the function name
 2. Ctrl+N to start multi-cursor on the word
 3. n to find each additional call site
 4. q to skip any site you want to leave unchanged
 5. Use any vim editing command – all cursors execute it simultaneously
-

Advanced Usage

Disabling Plugins Temporarily

To disable hardtime for a session when you need to hold j to scan a long file:

```
:Hardtime disable
```

Re-enable with:

```
:Hardtime enable
```

Conform Formatter Selection

Conform picks formatters based on filetype. If no specific formatter is configured for a filetype, the `lsp_fallback = true` setting means it will ask the active LSP server to format instead. You can check which formatter will be used:

```
:ConformInfo
```

This shows the active formatters for the current buffer, their availability, and whether they are installed.

Plugin Update Workflow

All custom plugins (except the local 99.nvim) are managed by lazy.nvim:

1. Press <leader>l to open the Lazy UI
2. Press C to check for updates
3. Press U to update all plugins
4. Review changes in the UI – each plugin shows its recent commits
5. If something breaks, the lockfile (lazy-lock.json) lets you revert

The 99.nvim plugin, being local, is updated by pulling changes in its source directory (~/.Developer/zero/workspace/99) with git.

Writing Your Own Plugin Spec

To add a new plugin to this configuration:

1. Create a new file in lua/plugins/ (e.g., lua/plugins/my-plugin.lua)
2. Return a spec table:

```
return {
  "author/plugin-name",
  event = "VeryLazy",      — or keys, cmd, ft
  opts = {
    — plugin options here
  },
}
```

3. Restart Neovim or run :Lazy sync
4. The plugin installs and loads automatically

For a local plugin under development:

```
return {  
  dir = vim.fn.expand("~/Developer/my-plugin"),  
  config = function()  
    require("my-plugin").setup({})  
  end,  
}
```

The `dir` field tells `lazy.nvim` to load from a local path instead of cloning a repository.

LSP and Treesitter

Two technologies that transform Neovim from a text editor into a language-aware development environment.

Your Setup

LazyVim pre-configures both LSP (Language Server Protocol) and Treesitter with sensible defaults. zero.dev001's setup uses the LazyVim defaults for both – no custom overrides in `options.lua`, `keymaps.lua`, or `autocmds.lua` for these features. The LSP and Treesitter configuration comes entirely from LazyVim's built-in specs and any language extras that are imported.

The relevant LazyVim plugins:

Plugin	Role
neovim/nvim-lspconfig	LSP client configuration
williamboman/mason.nvim	Package manager for LSP servers, formatters, linters
williamboman/mason-lspconfig.nvim	Bridge between Mason and lspconfig
nvim-treesitter/nvim-treesitter	Treesitter integration
folke/trouble.nvim	Pretty diagnostics list
stevearc/conform.nvim	Formatting (configured in <code>plugins/conform.lua</code>)

Core Concepts

What Is LSP?

The Language Server Protocol is a standard that lets editors communicate with language-specific servers. Instead of every editor implementing its own Go analyzer, TypeScript checker, or Python linter, a single language server provides:

- **Completions** – context-aware suggestions as you type
- **Diagnostics** – errors and warnings in real time
- **Go to definition** – jump to where a symbol is defined
- **Find references** – list everywhere a symbol is used
- **Hover documentation** – show type signatures and docs

- **Code actions** – automated refactors, quick fixes, import organization
- **Rename** – rename a symbol across the entire project
- **Formatting** – language-standard code formatting

The editor sends requests to the server; the server responds with structured data. Neovim’s built-in LSP client handles the protocol. LazyVim and Mason handle the configuration and installation.

What Is Treesitter?

Treesitter is a parser generator that builds concrete syntax trees for source code. Traditional syntax highlighting in vim uses regular expressions – patterns like “a word after function is a function name.” These patterns are fragile, slow, and frequently wrong.

Treesitter parses the actual grammar of each language and produces an AST (Abstract Syntax Tree). This gives Neovim:

- **Accurate highlighting** – every token is highlighted based on its grammatical role, not a regex guess
- **Structural navigation** – move between functions, classes, parameters, and other language constructs
- **Incremental selection** – expand or shrink selections along AST boundaries
- **Text objects** – function, class, parameter, block, and other language-aware selections
- **Folding** – fold code based on syntax structure, not indentation
- **Indentation** – auto-indent based on the syntax tree

How They Work Together

LSP and Treesitter complement each other:

Capability	LSP	Treesitter
Syntax highlighting	No	Yes (AST-based)
Completions	Yes (semantic)	No
Diagnostics	Yes (type errors, lint)	No
Go to definition	Yes (cross-file)	No
Text objects	No	Yes (function, class)
Incremental selection	No	Yes
Code actions	Yes (refactoring)	No
Folding	No	Yes (syntax-based)
Indentation	No	Yes (syntax-based)
Rename symbol	Yes (project-wide)	No

You want both. Treesitter handles the visual and structural aspects; LSP handles the semantic and project-wide intelligence.

Essential Commands

LSP Navigation

Key	Action
gd	Go to definition
gD	Go to declaration
gr	Go to references (list all usages)
gI	Go to implementation
gy	Go to type definition
K	Hover documentation (show type/docs popup)
gK	Signature help

LSP Code Actions

Key	Action
<leader>ca	Code action (quick fixes, refactors)
<leader>cA	Source action (organize imports, etc.)
<leader>cr	Rename symbol (project-wide)
<leader>cf	Format document
<leader>cl	LSP info (show active servers)

Diagnostics

Key	Action
<leader>cd	Line diagnostics (show error popup)
]d	Next diagnostic
[d	Previous diagnostic
]e	Next error
[e	Previous error
]w	Next warning
[w	Previous warning
<leader>xx	Toggle Trouble (diagnostics panel)
<leader>xX	Buffer diagnostics (Trouble)
<leader>xL	Location list (Trouble)
<leader>xQ	Quickfix list (Trouble)

Mason Package Manager

Key / Command	Action
<leader>cm	Open Mason UI
:Mason	Open Mason UI (same)
:MasonInstall {name}	Install a specific package
:MasonUninstall {name}	Uninstall a package
:MasonUpdate	Update all packages

Inside the Mason UI:

Key	Action
i	Install package under cursor
u	Update package
X	Uninstall package
Ctrl+F	Filter by category
1–5	Filter by category number

Treesitter

Key / Command	Action
Enter	Initialize incremental selection
Enter (again)	Expand selection to next AST node
Backspace	Shrink selection to previous node
:TSInstall {lang}	Install a language parser
:TSUpdate	Update all installed parsers
:TSInstallInfo	Show installed/available parsers
:InspectTree	Show the syntax tree for current buffer

Practical Recipes

Navigating a Codebase with LSP

A typical code exploration workflow:

1. Open a file and place cursor on a function call
2. Press `gd` to jump to its definition (may open another file)
3. Read the implementation, press `K` to see type info
4. Press `gr` to see everywhere this function is called
5. Select a reference from the list and jump to it
6. Press `Ctrl+O` to jump back through the navigation stack
7. Press `Ctrl+I` to jump forward again

The jump list (`Ctrl+O` / `Ctrl+I`) is your history of positions. Every `gd`, `gr`, or search creates a jump entry, so you can always retrace your steps.

Rename a Symbol Safely

1. Place cursor on the variable, function, or type name
2. Press `<leader>cr` to trigger rename
3. Type the new name in the popup
4. Press `Enter` to confirm
5. The LSP renames every occurrence across every file in the project

This is semantically aware – it renames the symbol, not the string. A variable named `count` inside a function will not rename the word “count” in a comment or in an unrelated scope.

Understanding Diagnostics

When the LSP reports an error, you will see:

- A red (or yellow for warnings) marker in the sign column on the left
- Underlined text at the error location
- A virtual text hint at the end of the line (if enabled)

To read the full diagnostic message:

1. Move cursor to the error line
2. Press <leader>cd to open the diagnostic float
3. Read the full error message and any suggested fixes
4. Press <leader>ca to see available code actions (auto-fixes)

To cycle through all diagnostics:]d moves to the next, [d moves to the previous. To jump only between errors (skipping warnings):]e and [e.

Incremental Selection with Treesitter

Treesitter incremental selection lets you expand a selection along AST boundaries:

1. Place cursor inside a string: "hello world"
2. Press Enter to start selection – selects hello world
3. Press Enter again – selects "hello world" (including quotes)
4. Press Enter again – selects the entire function argument list
5. Press Enter again – selects the entire function call
6. Press Backspace to shrink back one level

This is far more precise than visual mode because it understands the syntax structure. You never accidentally select half a bracket pair or miss a closing tag.

Inspecting the Syntax Tree

To understand how Treesitter sees your code:

1. Open any source file
2. Run :InspectTree
3. A split opens showing the full AST
4. Move your cursor in the source – the corresponding AST node highlights
5. Move your cursor in the tree – the corresponding source highlights

This is invaluable for debugging highlighting issues, writing custom queries, or understanding why a text object selects a certain range.

Advanced Usage

Common Language Servers

LazyVim auto-installs servers for languages you use. Here are the most common ones in zero.dev001's workflow:

Language	Server	Provides
TypeScript	<code>typescript-language-server</code>	Types, completions, diagnostics
Lua	<code>lua-language-server</code>	Neovim API awareness
JSON	<code>json-lsp</code>	Schema validation, completions
CSS	<code>css-lsp</code>	Properties, values, diagnostics
HTML	<code>html-lsp</code>	Tags, attributes, completions
ESLint	<code>eslint</code>	Linting, auto-fix on save
Tailwind	<code>tailwindcss-language-server</code>	Class completions, hover preview
Go	<code>gopls</code>	Full Go intelligence
Python	<code>pyright</code>	Type checking, completions
Bash	<code>bash-language-server</code>	Shell script intelligence
YAML	<code>yaml-language-server</code>	Schema validation

Mason-Managed Tools

Mason manages more than just LSP servers. It also handles:

Category	Examples
LSP servers	All servers listed above
Formatters	<code>stylua</code> , <code>prettier</code> , <code>shfmt</code> , <code>black</code>
Linters	<code>shellcheck</code> , <code>flake8</code> , <code>eslint_d</code>
DAP servers	Debug adapters for various languages

The `ensure_installed` list in the LazyVim mason config guarantees these tools are available:

`stylua`, `shellcheck`, `shfmt`, `flake8`

Treesitter Text Objects

With Treesitter-powered text objects (provided by `nvim-treesitter-textobjects`), you get language-aware selections:

Text Object	Selects
<code>af</code>	Around function (including signature)
<code>if</code>	Inside function (body only)
<code>ac</code>	Around class
<code>ic</code>	Inside class
<code>aa</code>	Around parameter/argument
<code>ia</code>	Inside parameter/argument

These work with all operators: `daf` deletes an entire function, `yif` yanks a function body, `caa` changes a function argument.

Treesitter Navigation

Jump between language constructs:

Key	Action
]f	Next function start
[f	Previous function start
]c	Next class start
[c	Previous class start
]a	Next parameter
[a	Previous parameter

Diagnostics Severity Configuration

LazyVim configures diagnostic display with virtual text hints. The severity levels and their visual indicators:

Severity	Sign	Meaning
Error	Red	Code will not compile or run
Warning	Yellow	Potential issue, code still works
Info	Blue	Informational hint
Hint	Teal	Suggestion for improvement

Troubleshooting LSP Issues

When an LSP server is not working as expected:

1. Check server status: <leader>cl shows attached servers and their status
2. Check health: :checkhealth lsp runs diagnostics
3. Check Mason: <leader>cm shows installed/available packages
4. View logs: :LspLog opens the LSP log file
5. Restart server: :LspRestart restarts all attached servers

Common issues: - **No completions:** The server may still be indexing. Large projects take time. - **Wrong server attached:** Check :LspInfo for which server is active. - **Missing server:** Run :MasonInstall {server-name} to install it. - **Stale diagnostics:** :LspRestart clears the slate.

Performance Considerations

LSP servers run as separate processes. For large projects:

- TypeScript projects with many files may take several seconds to fully index
- Multiple LSP servers can attach to the same buffer (e.g., TypeScript + ESLint + Tailwind)
- Treesitter parsing is incremental – it only re-parses what changed, keeping highlighting fast even in large files
- If Neovim feels slow, check :Lazy profile and :LspLog to identify the bottleneck

AI Integration: Claude and 99.nvim

AI as a resident collaborator – embedded in the editor, running beside it in the terminal, and woven into every coding session.

Your Setup

zero.dev001's AI workflow has two layers: **99.nvim** inside Neovim for inline AI assistance, and **Claude Code** running as a CLI tool in a dedicated tmux pane. Together they create an environment where AI is always one keystroke or one pane switch away.

The 99.nvim plugin:

Source: ~/Developer/zero/workspace/99

Provider: ClaudeCodeProvider

Model: opus-4.6

The IDE tmux layout:

+-----+	
20%	Neovim (80%)
Claude	-----+
	Terminal (20%)
+-----+	

This layout is created by the `ide()` shell function, which uses `tmuxinator` to launch three panes with precise sizing.

Core Concepts

99.nvim Architecture

99.nvim is a custom Neovim plugin developed locally. It is not published to a package registry – it lives in zero.dev001's development workspace at ~/Developer/zero/workspace/99 and is loaded by lazy.nvim via the `dir` field.

The plugin follows a provider pattern:

- **Providers** abstract the AI backend. The ClaudeCodeProvider connects to the Claude API.
- **Models** are selectable at runtime. The default is opus-4.6, but any model the provider supports can be chosen via the picker.
- **Visual mode integration** is the primary interaction: select code, invoke AI, receive a response.

The plugin configuration in lua/plugins/99.lua:

```
return {
  {
    dir = vim.fn.expand("~/Developer/zero/workspace/99"),
    config = function()
      require("99").setup({
        provider = require("99.providers").ClaudeCodeProvider,
        model = "opus-4.6",
      })
    end,
    keys = {
      { "<leader>mm", function() require("99").select_model() end,
        desc = "Select AI model" },
      { "<leader>9v", function() require("99").visual() end,
        mode = "v", desc = "99 AI visual" },
      { "<leader>9s", function() require("99").stop_all_requests() end,
        desc = "99 Stop all requests" },
    },
  },
}
```

The keys field serves double duty: it defines the keybindings and tells lazy.nvim to lazy-load the plugin only when one of these keys is pressed.

Claude Code in the Terminal

Claude Code is a CLI tool that provides a conversational AI interface directly in the terminal. It runs as an interactive process in its own tmux pane – not as a Neovim plugin, but as a separate application sharing the same project context.

Key characteristics: - Runs in the left 20% pane of the IDE layout - Has full access to the project filesystem - Can read files, suggest changes, run commands - Maintains conversation history within a session - Understands project context from the working directory

The IDE Layout

The ide() function creates the complete development environment:

```
ide() {
  local dir="${1:-.}"
  local name=$(basename "$(cd "$dir" 2>/dev/null && pwd)")
  tmuxinator start ide "$dir" --no-attach
  tmux resize-pane -t "$name:1.0" -x '20%'
  tmux resize-pane -t "$name:1.2" -y '20%'
}
```

```

tmux select-pane -t "$name:1.1 "
tmux rename-window -t "$name:1 " "$name"
tmux attach-session -t "$name"
}

```

The tmuxinator template (`~/config/tmuxinator/ide.yml`) defines:

Pane	Content	Position	Size
Pane 0	claude	Left	20%
Pane 1	nvim .	Right top	80%
Pane 2	Terminal	Right bottom	20%

After tmuxinator creates the layout, the `ide()` function resizes panes to exact proportions, selects the Neovim pane as active, and renames the tmux window to the project name.

Essential Commands

99.nvim Key Mappings

Key	Mode	Action
<leader>mm	Normal	Open model selection picker
<leader>9v	Visual	Send visual selection to AI
<leader>9s	Normal	Stop all active AI requests

IDE Layout Commands

Command	Action
<code>ide</code>	Start IDE layout in current directory
<code>ide ~/Projects/x</code>	Start IDE layout in specified directory
<code>Ctrl+H</code>	Move to left pane (Claude Code)
<code>Ctrl+L</code>	Move to right pane (Neovim)
<code>Ctrl+J</code>	Move to pane below (Terminal)
<code>Ctrl+K</code>	Move to pane above

The `Ctrl+H/J/K/L` navigation works seamlessly between Neovim splits and tmux panes thanks to `vim-tmux-navigator`. Pressing `Ctrl+H` from Neovim will cross the tmux pane boundary into the Claude Code pane on the left.

Claude Code Terminal Commands

These are run directly in the Claude Code pane or any terminal:

Command	Action
claude	Start a new Claude Code session
claude "question"	Ask a one-off question
/help	Show available commands
/clear	Clear conversation history
Ctrl+C	Cancel current response

Practical Recipes

Using 99.nvim for Code Transformation

The primary workflow for in-editor AI:

1. Navigate to the code you want to transform
2. Enter Visual mode with `v` (character), `V` (line), or use text objects
3. Select the relevant code block
4. Press `<leader>9v` to send the selection to the AI
5. The AI processes the selection and returns a result
6. Review the output

Example scenarios: - Select a function and ask for a refactored version - Select a block of code and request an explanation - Select a data structure and request a type definition - Select a test and request additional test cases

Switching AI Models

Different tasks may benefit from different models:

1. Press `<leader>mm` in Normal mode
2. A picker appears showing available models
3. Select the model best suited for your current task
4. The model change takes effect immediately for subsequent requests

The default opus-4.6 is the most capable model, but switching to a faster model can be useful for simple transformations where speed matters more than depth.

Stopping a Runaway Request

If an AI request is taking too long or you realize you sent the wrong selection:

1. Press `<leader>9s` in Normal mode
2. All active requests are cancelled immediately
3. You can make a new selection and try again

The Complete IDE Workflow

A full development session with AI integration:

1. **Start the environment:** `ide ~/Projects/myapp`

2. **Three panes appear:** Claude on the left, Neovim in the center-right, terminal below
3. **Code in Neovim:** Edit files, navigate with LSP, use all the tools from previous chapters
4. **Consult Claude Code:** Press Ctrl+H to switch to the Claude pane, ask a question about architecture or debugging, read the response
5. **Return to Neovim:** Press Ctrl+L to switch back, apply what you learned
6. **Use 99.nvim for inline tasks:** Select code, press <leader>9v, get an AI-assisted transformation without leaving the editor
7. **Run commands:** Press Ctrl+J to drop to the terminal pane, run tests or build commands, press Ctrl+K to return

This three-pane setup means AI is always visible in your peripheral vision. You can glance left to reference a previous Claude response while editing, or switch to the terminal to verify a suggestion.

AI-Assisted Debugging

When you encounter a bug:

1. In Neovim, select the problematic code with Visual mode
2. Press <leader>9v for a quick inline analysis
3. For deeper investigation, switch to the Claude pane with Ctrl+H
4. Paste error messages, stack traces, or describe the behavior
5. Claude Code can read the project files to understand context
6. Switch back to Neovim with Ctrl+L and apply the fix
7. Drop to the terminal with Ctrl+J to run tests

AI-Assisted Code Review

When reviewing code changes:

1. Open lazygit with <leader>gg in Neovim
2. Review diffs in the lazygit interface
3. For complex changes, switch to the Claude pane
4. Ask Claude to analyze the diff or explain the implications
5. Return to lazygit to approve, comment, or request changes

Advanced Usage

Developing 99.nvim

Since 99.nvim is a local plugin, development happens in real time:

1. Open the plugin source: nvim ~/Developer/zero/workspace/99
2. Make changes to the plugin code
3. In your project Neovim instance, reload with :Lazy reload 99.nvim or restart Neovim
4. Test the changes immediately

The lazy.nvim dir field means there is no install or build step – Neovim reads the plugin source directly from disk. Changes to the plugin take effect on the next load.

Provider System

The `99.providers` module abstracts AI backends. The `ClaudeCodeProvider` is the active provider, but the architecture supports swapping in different providers:

```
require ("99").setup({
  provider = require ("99.providers").ClaudeCodeProvider ,
  model = "opus-4.6" ,
})
```

To switch providers, you would change the provider field to a different provider module. The `select_model()` function presents models available from the currently configured provider.

Optimizing the IDE Layout

The default layout gives Claude 20% width, which is roughly 40 columns on a standard screen. This is enough for reading responses and short queries. Adjustments:

- For a wider Claude pane: modify the `tmux resize-pane` percentage in the `ide()` function
- For a taller terminal: change the `-y '20%'` value for pane 2
- For no terminal pane: modify the `tmuxinator` template to only create two panes

The `tmuxinator` template at `~/.config/tmuxinator/ide.yml`:

```
name: <%= File.basename( File.expand_path( @args[0] || "." )) %>
root: <%= @args[0] || "." %>
```

```
windows:
- code:
  layout: main-vertical
  panes:
    - claude
    - nvim .
    - # terminal
```

The template accepts a directory argument, names the session after the directory, and sets the root for all panes to that directory. This means Claude Code, Neovim, and the terminal all start in the same project root.

Contextual AI Usage Patterns

Different situations call for different AI interaction patterns:

Situation	Best Tool	Approach
Refactor a single function	99.nvim	Select + <leader>9v
Understand a codebase	Claude Code	Conversational exploration
Debug an error	Claude Code	Paste error, discuss context
Generate boilerplate	Either	Describe what you need
Review a diff	Claude Code	Paste or describe the changes
Quick type annotation	99.nvim	Select code, request types

Situation	Best Tool	Approach
Architecture decisions	Claude Code	Extended conversation

The rule of thumb: use 99.nvim for targeted, code-level transformations where you can select exactly what you want processed. Use Claude Code for exploratory conversations, debugging sessions, and architectural discussions where context builds over multiple exchanges.

Security Considerations

Both AI tools interact with your code:

- **99.nvim** sends selected code to the Claude API via the `ClaudeCodeProvider`
- **Claude Code** can read files in the project directory

Be aware of what you send to AI services. Avoid selecting and sending sensitive credentials, API keys, or proprietary algorithms that should not leave your machine. The AI tools are powerful collaborators, but they communicate over the network.

Tmux Core

A terminal multiplexer that lets you run persistent sessions with multiple windows and panes – the backbone of every terminal workflow.

Your Setup

The tmux configuration lives in `~/.tmux.conf`, managed by chezmoi as `dot_tmux.conf`. It is tuned for speed, modern terminal features, and Vim-style navigation.

Terminal and display settings:

```
set-option -sa terminal-overrides " ,xterm*:Tc"    # True color support
set -g mouse on                                     # Full mouse support
set -g escape-time 0                               # No delay after Escape
set -g history-limit 1000000                       # 1 million lines of scrollbar
set -g detach-on-destroy off                       # Stay in tmux when session closes
set -g set-clipboard on                            # OSC 52 clipboard integration
set -g status-position top                         # Status bar at the top
set -g set-titles on                               # Terminal title updates
set -g set-titles-string "#S"                     # Title shows session name
```

Indexing starts at 1, not 0, and windows renumber automatically when one is closed:

```
set -g base-index 1
set -g pane-base-index 1
set-window-option -g pane-base-index 1
set-option -g renumber-windows on
```

The prefix key is the default `Ctrl+b`.

Passthrough is enabled for tools like Neovim image protocols and terminal features that need to send escape sequences through tmux:

```
set -g allow-passthrough on
set -ga update-environment TERM
set -ga update-environment TERM_PROGRAM
```

The theme is Catppuccin Mocha, providing a consistent dark palette that matches the Neovim and terminal themes.

Core Concepts

The Tmux Hierarchy: Sessions, Windows, and Panes

Tmux organizes your work in three levels:

- **Session:** A named collection of windows. Each project or context gets its own session. Sessions persist even when you disconnect, so you can resume exactly where you left off.
- **Window:** A full-screen tab within a session. Think of these as tabs in a browser. Each window has its own set of panes.
- **Pane:** A split within a window. Panes divide a window into rectangular regions, each running an independent shell or process.

A typical structure looks like this:

```
Session: "webapp"
  Window 1: "editor"      — nvim in a single pane
  Window 2: "servers"     — two panes: API server | log tail
  Window 3: "database"    — psql in a single pane

Session: "dotfiles"
  Window 1: "config"      — nvim editing config files
```

The Prefix Key

Most tmux commands start with the prefix key. The default is Ctrl+b. The prefix is a chord – press and release it, then press the command key. It is not held down during the command key.

Example: to split horizontally, the sequence is:

1. Press Ctrl+b (prefix fires)
2. Release
3. Tap " (split command)

Detach and Attach

The defining feature of tmux is that sessions survive disconnection. When you close your terminal or SSH connection, your tmux sessions continue running. You can reattach at any time and find everything exactly as you left it.

- **Detach:** prefix + d disconnects from the current session without stopping it.
- **Attach:** `tmux attach -t session-name` reconnects to a running session.
- **detach-on-destroy off:** When you close the last window in a session, tmux switches you to another session instead of dumping you back to the bare terminal. This keeps you inside tmux.

Naming Conventions

Name sessions after projects or contexts: `webapp`, `dotfiles`, `api`. Name windows after their purpose: `editor`, `servers`, `tests`. This makes the status bar immediately readable and session switching intuitive.

Essential Commands

Starting and Managing Sessions

Command	Action
<code>tmux</code>	Start a new unnamed session
<code>tmux new -s name</code>	Start a new session named “name”
<code>tmux ls</code>	List all running sessions
<code>tmux attach -t name</code>	Attach to session “name”
<code>tmux kill-session -t name</code>	Kill session “name”
<code>tmux kill-server</code>	Kill tmux entirely
<code>prefix + d</code>	Detach from current session
<code>prefix + \$</code>	Rename current session

Window Management

Keybinding	Action
<code>prefix + c</code>	Create new window
<code>prefix + ,</code>	Rename current window
<code>prefix + &</code>	Close current window (confirm)
<code>prefix + n</code>	Next window
<code>prefix + p</code>	Previous window
<code>prefix + 1–9</code>	Switch to window by number
<code>Shift+Left</code>	Previous window (no prefix)
<code>Shift+Right</code>	Next window (no prefix)
<code>Alt+H</code>	Previous window (no prefix)
<code>Alt+L</code>	Next window (no prefix)

Note: `Shift+Left/Right` and `Alt+H/L` do not require the prefix key. They are bound with `-n` (no prefix) for rapid window switching.

Pane Management

Keybinding	Action
<code>prefix + "</code>	Split horizontally (current path)
<code>prefix + %</code>	Split vertically (current path)
<code>prefix + x</code>	Close current pane (confirm)
<code>prefix + z</code>	Toggle pane zoom (fullscreen)
<code>prefix + !</code>	Break pane into its own window
<code>prefix + q</code>	Show pane numbers (tap number to switch)

Pane Navigation

Keybinding	Action
prefix + h	Select pane left
prefix + j	Select pane below
prefix + k	Select pane above
prefix + l	Select pane right
Alt+Left	Select pane left (no prefix)
Alt+Right	Select pane right (no prefix)
Alt+Up	Select pane above (no prefix)
Alt+Down	Select pane below (no prefix)
Ctrl+h/j/k/l	Navigate panes and Vim splits (vim-tmux-navigator)

The vim-tmux-navigator plugin (covered in the next chapter) overrides Ctrl+h/j/k/l to move seamlessly between tmux panes and Neovim splits. The same keys work whether you are in a tmux pane or a Neovim split.

Copy Mode

Copy mode uses vi keybindings. Enter it with prefix + [.

Keybinding	Action
prefix + [	Enter copy mode
v	Start selection (in copy mode)
Ctrl+v	Toggle rectangle selection
y	Copy selection and exit copy mode
q	Exit copy mode
/	Search forward (in copy mode)
?	Search backward (in copy mode)
n / N	Next / previous search match

In copy mode, standard Vim motions work: h/j/k/l, w/b/e, 0/\$, gg/G, Ctrl+u/d for half-page scrolling.

Practical Recipes

Start a named session and split into a working layout

```
tmux new -s webapp
# Inside tmux:
# prefix + " to split horizontally
# prefix + % to split vertically
```

Quickly switch between two windows

Use Shift+Left and Shift+Right to flip between adjacent windows without touching the prefix key. For Vim-style switching, Alt+H and Alt+L do the same thing.

Zoom a pane for full-screen focus

When a pane is too small to read comfortably, prefix + z toggles it to full screen. Press prefix + z again to restore the original layout. The zoomed pane is indicated in the status bar.

Reorder windows after closing one

Windows renumber automatically (renumber—windows on). If you close window 2 out of windows 1, 2, 3, the remaining windows become 1, 2 – no gaps.

Copy text from terminal output

1. Enter copy mode: prefix + [
2. Navigate to the start of the text you want (Vim motions)
3. Press v to begin selection
4. Move to the end of the desired text
5. Press y to copy and exit copy mode
6. Paste with prefix +] or your system paste shortcut

The tmux-yank plugin ensures the copied text also lands in your system clipboard.

Move a pane to a different window

To move the current pane to window 3: prefix + : to open the command prompt, then type join—pane —t :3. To break a pane out into its own new window: prefix + !.

Resize panes

Hold the prefix and use arrow keys: prefix + Up/Down/Left/Right resizes in that direction. Alternatively, use the command prompt: prefix + : then resize—pane —D 10 to shrink down by 10 rows.

Advanced Usage

Why escape-time Is Zero

The escape—time setting controls how long tmux waits after receiving an Escape character before passing it through. The default is 500ms, which creates a noticeable lag when pressing Escape in Neovim to leave insert mode. Setting it to 0 eliminates this delay entirely. If you use a slow SSH connection where key sequences might arrive in fragments, you might need to raise this slightly, but for local use, 0 is ideal.

The 1-Million-Line Scrollback

The history—limit 1000000 setting gives each pane a scrollback buffer of one million lines. This is generous – a typical terminal session with active build logs might generate tens of thousands of lines per hour. With a million lines, you can scroll back through an entire day of work. The memory cost is negligible on modern machines because tmux only allocates memory for lines that actually exist.

Status Bar Position

The status bar sits at the top (`status-position top`), which pairs well with the Catppuccin theme and keeps the bottom of the terminal clear for command output. This is a matter of preference, but top positioning avoids conflicts with shell prompts and keeps session/window information at eye level.

Set-Titles for Terminal Tab Names

With `set-titles on` and `set-titles-string "#S"`, tmux updates the terminal's title to show the current session name. In iTerm2, this means the tab title reflects which tmux session is active, making it easy to find the right tab when you have multiple terminal windows open.

Base Index 1

Windows and panes start at 1 instead of 0. This matches the physical keyboard layout – the number keys 1 through 9 are easier to reach than 0, and prefix `+ 1` being the first window feels natural. The `renumber-windows on` setting ensures that closing a window does not leave gaps in the numbering.

Detach-on-Destroy Off

The default behavior when closing the last window in a session is to detach from tmux entirely, dropping you to the bare terminal. With `detach-on-destroy off`, tmux switches to the next available session instead. This keeps you inside tmux at all times, which is important when tmux is the foundation of your workflow.

Mouse Support

Mouse mode is on, which allows:

- Clicking to select panes
- Clicking to select windows in the status bar
- Dragging pane borders to resize
- Scrolling to enter copy mode and scroll through history
- Selecting text (though keyboard-based copy mode is usually faster)

Mouse support does not interfere with terminal applications that handle mouse events themselves – tmux passes mouse events through to the active pane's program.

Clipboard Integration

The `set-clipboard on` setting enables OSC 52 clipboard integration, which allows tmux to set the system clipboard directly through the terminal's escape sequences. This works across SSH connections, remote machines, and inside nested tmux sessions. Combined with the `tmux-yank` plugin, copying text in tmux copy mode places it directly on the system clipboard.

Tmux Plugins

Twelve plugins that transform tmux from a terminal multiplexer into a session-persistent, clipboard-aware, fzf-powered development environment.

Your Setup

Plugins are managed by TPM (Tmux Plugin Manager) and declared at the bottom of `~/.tmux.conf`. The full plugin list:

```
set -g @plugin 'tmux-plugins/tpm '  
set -g @plugin 'tmux-plugins/tmux-sensible '  
set -g @plugin 'christoomey/vim-tmux-navigator '  
set -g @plugin 'dreamsofcode-io/catppuccin-tmux '  
set -g @plugin 'tmux-plugins/tmux-yank '  
set -g @plugin 'tmux-plugins/tmux-resurrect '  
set -g @plugin 'tmux-plugins/tmux-continuum '  
set -g @plugin 'omerxx/tmux-sessionx '  
set -g @plugin 'omerxx/tmux-floax '  
set -g @plugin 'wfxr/tmux-fzf-url '  
set -g @plugin 'sainnhe/tmux-fzf '  
set -g @plugin 'fcsonline/tmux-thumbs '
```

TPM is loaded at the very end of the config:

```
run '~/.tmux/plugins/tpm/tpm '
```

Plugin-specific settings are grouped together:

```
set -g @continuum-restore 'on '  
set -g @resurrect-strategy-nvim 'session '  
set -g @sessionx-bind 'o '  
set -g @sessionx-zoxide-mode 'on '  
set -g @sessionx-filter-current 'false '  
set -g @floax-width '80% '  
set -g @floax-height '80% '  
set -g @floax-border-color 'magenta '  
set -g @floax-text-color 'blue '  
set -g @floax-bind 'p '  
set -g @floax-change-path 'true '
```

```
set -g @fzf-url-fzf-options '-p 60%,30% --prompt="  " --border-label="  OpenURL  "
set -g @fzf-url-history-limit '2000'
```

Core Concepts

TPM: The Plugin Manager

TPM clones plugin repositories into `~/tmux/plugins/` and sources them when tmux starts. After adding a new plugin line to `tmux.conf`, press prefix + I (capital I) to install it. TPM fetches the repository and activates the plugin immediately.

Plugin Categories

The twelve plugins fall into four functional groups:

Foundation: `tpm`, `tmux-sensible` – provide the plugin infrastructure and a set of universally accepted defaults.

Navigation: `vim-tmux-navigator` – the single most important plugin, enabling seamless movement between tmux panes and Neovim splits.

Persistence: `tmux-resurrect`, `tmux-continuum` – save and restore your entire tmux environment across reboots.

Productivity: `sessionx`, `floax`, `tmux-fzf-url`, `tmux-fzf`, `tmux-thumbs`, `tmux-yank`, `catppuccin` – enhance session management, clipboard, search, and visual appearance.

Why vim-tmux-navigator Is Critical

Without this plugin, moving between a Neovim split and an adjacent tmux pane requires completely different keybindings. With it, `Ctrl+h/j/k/l` moves in the expected direction regardless of whether the boundary is a Vim split or a tmux pane. The plugin detects which program is running in the active pane and routes the key event accordingly.

Essential Commands

TPM Management

Keybinding	Action
prefix + I	Install new plugins
prefix + U	Update all plugins
prefix + Alt+u	Uninstall removed plugins

vim-tmux-navigator

Keybinding	Action
Ctrl+h	Move left (pane or Vim split)
Ctrl+j	Move down (pane or Vim split)

Keybinding	Action
Ctrl+k	Move up (pane or Vim split)
Ctrl+l	Move right (pane or Vim split)
Ctrl+\	Move to previous pane/split

These bindings work identically whether you are in a tmux pane, a Neovim split, or crossing the boundary between them. Neovim must also have the vim-tmux-navigator plugin installed for this to work.

Session Persistence (resurrect + continuum)

Keybinding	Action
prefix + Ctrl+s	Save session state
prefix + Ctrl+r	Restore saved session

Continuum automatically saves every 15 minutes and restores on tmux server start. No manual action needed for day-to-day use.

Session Picker (sessionx)

Keybinding	Action
prefix + o	Open session picker

Inside the session picker, zoxide mode is on, meaning you can search for directories by frequency as well as existing tmux sessions. filter —current is off, so the current session appears in the list.

Floating Terminal (floax)

Keybinding	Action
prefix + p	Toggle floating terminal

The floating terminal occupies 80% of the screen width and height, with a magenta border and blue text. It opens in the current pane's working directory (change—path true), making it ideal for quick commands without disrupting your layout.

URL and Pattern Extraction

Keybinding	Action
prefix + u (fzf-url default)	Open URLs from terminal output in fzf popup

Keybinding	Action
prefix + Space (thumbs default)	Highlight copyable patterns in terminal output

Clipboard (tmux-yank)

Action	Behavior
y in copy mode	Copies selection to system clipboard and cancels copy mode
Mouse selection	Copies to system clipboard (when mouse mode is on)

Practical Recipes

First-time plugin installation

After cloning your dotfiles to a new machine:

```
# Clone TPM if not present
git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm

# Start tmux, then install all plugins
tmux
# Press prefix + I
```

TPM reads the plugin declarations from `tmux.conf` and clones each repository into `~/.tmux/plugins/`.

Seamless Vim-Tmux navigation workflow

With the IDE layout (covered in a later chapter), you have three panes: Claude Code on the left, Neovim top-right, terminal bottom-right. Moving between them:

- From Neovim to terminal below: `Ctrl+j`
- From terminal to Claude on the left: `Ctrl+h`
- From Claude to Neovim on the right: `Ctrl+l`

The key insight is that you never think about whether the boundary is a Vim split or a tmux pane. The same four keys navigate everywhere.

Surviving a reboot with resurrect and continuum

The typical flow:

1. Work normally in tmux with multiple sessions, windows, and panes.
2. Continuum saves state every 15 minutes automatically.
3. Reboot the machine.
4. Start tmux. Continuum detects saved state and restores automatically (`continuum—restore on`).
5. All sessions, windows, and pane layouts are restored. Neovim sessions are restored via the `resurrect—strategy—nvim` session setting, which tells resurrect to save and restore Neovim's session files.

To force a manual save before a planned reboot: `prefix + Ctrl+s`.

Quick session switching with sessionx

Press `prefix + o` to open the session picker. Type to filter sessions by name. Because zoxide mode is on, you can also type a directory name you have visited before, and sessionx will offer to create a new session in that directory.

Pop open a floating terminal

Mid-task in Neovim, you need to run a quick command without leaving your editor or creating a new pane. Press `prefix + p`. A floating terminal appears at 80% screen size, already in the current directory. Run your command, then press `prefix + p` again to dismiss it.

Grab a URL from terminal output

A build log or command output contains a URL you need to open. Press the `fzf-url` trigger to scan the visible terminal output for URLs. An `fzf` popup (60% wide, 30% tall) appears with all detected URLs. Select one and it opens in your default browser.

Quick-copy a hash, path, or IP with thumbs

`tmux-thumbs` scans the visible terminal output for common patterns: file paths, git hashes, IP addresses, UUIDs, and more. When triggered, it highlights each match with a letter hint. Press the letter to copy that pattern to the clipboard instantly. This is faster than entering copy mode, navigating to the text, selecting it, and yanking.

Advanced Usage

How vim-tmux-navigator Detects the Active Program

The plugin works by checking the name of the process running in the current tmux pane. When the active process is `vim`, `nvim`, or another configured name, key presses are forwarded to Vim's own split navigation. Otherwise, tmux handles the navigation directly. This detection uses `pane_current_command` in a `tmux if-shell` conditional.

If you run Neovim inside a wrapper script or through a non-standard binary name, you may need to configure the detection pattern. The Neovim side of the plugin must also be installed – typically via `lazy.nvim` as `christoomey/vim-tmux-navigator`.

Resurrect Save Strategy for Neovim

The setting `@resurrect-strategy-nvim 'session'` tells `tmux-resurrect` to save Neovim's state using Neovim's built-in `:mksession` command. When restoring, resurrect launches Neovim with `nvim -S` to reload the session file. This restores open buffers, split layouts, and cursor positions within Neovim.

For this to work reliably, Neovim must have session support enabled and the session file must be written to a predictable location. If you use an auto-session plugin in Neovim, it coordinates with this mechanism.

Continuum Save Interval

Continuum saves every 15 minutes by default. This interval is a good balance between safety and disk activity. Save files are stored in `~/.tmux/resurrect/`. Each save creates a new file with a timestamp, so you can manually restore from an older state if needed by symlinking the last file to a previous save.

Catppuccin Theme Customization

The Catppuccin Mocha theme from the dreamsofcode fork provides status bar styling that matches the Neovim and terminal color schemes. The theme colors the status bar, pane borders, and mode indicators. The mocha flavor uses warm, muted tones with good contrast for readability.

sessionx vs. sesh

Both sessionx and sesh provide session management. They complement each other:

Feature	sessionx (prefix + o)	sesh (prefix + T)
Trigger	Single keybinding popup	fzf-tmux popup with mode switching
Zoxide integration	Yes (zoxide-mode on)	Yes (Ctrl+x for zoxide list)
Named config sessions	No	Yes (from sesh.toml)
Kill sessions	Via sessionx interface	Ctrl+d in sesh picker
Find new directories	No	Yes (Ctrl+f uses fd)

Use sessionx for quick switching between existing sessions. Use sesh when you want access to named config sessions, zoxide directories, or directory discovery via fd.

floax Working Directory

With `@floax—change—path true`, the floating terminal starts in the working directory of the current pane. This is especially useful when you are in a project-specific session and want to run a quick `git status` or `npm test` without navigating to the right directory.

fzf-url History Limit

The `@fzf—url—history—limit 2000` setting tells the plugin to scan the last 2000 lines of terminal output for URLs. This is generous enough to capture URLs from long build logs or verbose command output. Increase it if you work with tools that produce very long output with URLs scattered throughout.

Plugin Load Order

TPM loads plugins in the order they are declared. The order matters for plugins that modify the same keybindings or hooks. In this config, `tpm` and `tmux—sensible` are first (providing the

foundation), `vim-tmux-navigator` is early (setting up navigation before anything else), and the theme is loaded before plugins that display in the status bar.

Troubleshooting Plugins

If a plugin stops working after an update:

```
# Check that plugins are installed  
ls ~/.tmux/plugins/
```

```
# Reinstall all plugins  
prefix + I
```

```
# Check tmux for errors  
tmux show-messages
```

```
# Source the config manually  
tmux source-file ~/.tmux.conf
```

If `vim-tmux-navigator` stops working, verify that both the `tmux` plugin and the `Neovim` plugin are installed and that the process name detection is matching correctly.

Session Management: sesh

A smart tmux session manager that combines named configurations, zoxide frecency, and fzf fuzzy finding into a single session picker.

Your Setup

Sesh is configured in two places: `~/.config/sesh/sesh.toml` defines named sessions, and `~/.tmux.conf` defines the keybinding that launches the sesh picker.

The named sessions in `sesh.toml`:

```
[[ session ]]
name = " dotfiles "
path = "~/.local/share/chezmoi "
startup_command = "nvim "
preview_command = "bat —color=always ~/.local/share/chezmoi/dot_zshrc.tmpl "

[[ session ]]
name = "tmux config "
path = "~/.local/share/chezmoi "
startup_command = "nvim dot_tmux.conf "
preview_command = "bat —color=always ~/.local/share/chezmoi/dot_tmux.conf "

[[ session ]]
name = "nvim config "
path = "~/.config/nvim "
startup_command = "nvim lua/config/lazy.lua "
preview_command = "eza —tree —level=2 —icons ~/.config/nvim "

[[ session ]]
name = "Downloads "
path = "~/Downloads "
startup_command = "ls "
```

The tmux keybinding that launches the picker (prefix + T):

```
bind-key "T" run-shell "sesh connect \ "$(
    sesh list | fzf-tmux -p 55%,60% \
    —no-sort —ansi —border-label ' sesh ' —prompt '  ' \
```

```

—header '␣^a␣all␣t␣tmux␣^g␣configs␣^x␣zoxide␣^d␣tmux␣kill␣^f␣find' \
—bind 'tab:down,btab:up' \
—bind 'ctrl-a:change-prompt(␣␣)+reload(sesh␣list)' \
—bind 'ctrl-t:change-prompt(␣␣)+reload(sesh␣list␣-t)' \
—bind 'ctrl-g:change-prompt(␣␣)+reload(sesh␣list␣-c)' \
—bind 'ctrl-x:change-prompt(␣␣)+reload(sesh␣list␣-z)' \
—bind 'ctrl-f:change-prompt(␣␣)+reload(fd␣-H␣-d␣2␣-t␣d␣.␣~)' \
—bind 'ctrl-d:execute(tmux␣kill-session␣-t␣{ })+change-prompt(␣␣)+reload(sesh␣
)\ " "

```

Core Concepts

What sesh Does

Sesh sits between you and tmux session management. Instead of manually creating sessions with `tmux new -s name`, setting the working directory, and running startup commands, sesh handles all of this from a single picker interface. It knows about three sources of sessions:

1. **Named config sessions** – defined in `sesh.toml`, these are persistent bookmarks for your most important contexts. They always appear in the picker, whether a tmux session exists for them or not.
2. **Running tmux sessions** – any tmux session that is currently active.
3. **Zoxide directories** – directories ranked by frequency from your zoxide database, available as potential new sessions.

When you select a named session that is not currently running, sesh creates the tmux session, sets the working directory, and runs the startup command. When you select a running session, sesh switches to it. When you select a zoxide directory, sesh creates a new session named after the directory.

Named Session Anatomy

Each `[[ session ]]` entry in `sesh.toml` has four fields:

- **name:** The display name and tmux session name. This is what appears in the picker and the tmux status bar.
- **path:** The working directory for the session. Tilde expansion works.
- **startup_command:** The command to run when the session is first created. Typically `nvim` or `nvim <file>` to open directly into the editor.
- **preview_command:** What to show in the fzf preview pane when the session is highlighted. This gives you a quick look at the session's content before switching.

The Picker Modes

The sesh picker is an fzf-tmux popup with multiple modes, switchable via Ctrl keybindings:

Keybinding	Mode	What It Lists
Ctrl+a	All	Everything: config sessions + tmux sessions + zoxide

Keybinding	Mode	What It Lists
Ctrl+t	Tmux	Only running tmux sessions
Ctrl+g	Configs	Only named sessions from sesh.toml
Ctrl+x	Zoxide	Directories from zoxide database
Ctrl+f	Find	Directories found by fd (depth 2 from home)
Ctrl+d	Kill	Kills the selected tmux session and reloads

sesh vs. sessionx

Both tools manage tmux sessions, but they serve different purposes:

sesh (prefix + T) is the full-featured session manager. It understands named configurations, can create sessions from scratch with startup commands, integrates with zoxide for directory discovery, and can find new directories with fd. Use sesh when you want to jump to a project that may or may not have a tmux session yet.

sessionx (prefix + o) is a lighter session picker. It lists existing tmux sessions and zoxide directories in a simpler interface. Use sessionx for quick switching between sessions that are already running.

In practice, prefix + T is the primary session management keybinding, and prefix + o is the quick-switch alternative.

Essential Commands

Launching the Picker

Keybinding	Action
prefix + T	Open sesh picker
prefix + o	Open sessionx picker (alternative)

Inside the sesh Picker

Keybinding	Action
Type text	Filter sessions by name
Enter	Connect to selected session
Tab / Shift+Tab	Move down / up in list
Ctrl+a	Show all sources
Ctrl+t	Show tmux sessions only
Ctrl+g	Show config sessions only
Ctrl+x	Show zoxide directories
Ctrl+f	Find directories with fd
Ctrl+d	Kill selected session
Escape	Cancel and close picker

ssh CLI Commands

Command	Action
ssh list	List all sessions (config + tmux + zoxide)
ssh list -t	List running tmux sessions only
ssh list -c	List config sessions only
ssh list -z	List zoxide directories
ssh connect <name>	Connect to or create a session
ssh clone <repo> [name]	Clone a git repo and create a session for it

Practical Recipes

Jump to your dotfiles from anywhere

1. Press prefix + T
2. Type dot to filter
3. Select “dotfiles” and press Enter

Sesh creates the session (if not running) in ~/.local/share/chezmoi, runs nvim, and switches you there. The preview pane shows the contents of dot_zshrc.tmpl via bat with syntax highlighting.

Edit tmux config quickly

1. Open the ssh picker with prefix + T
2. Press Ctrl+g to filter to config sessions only
3. Select “tmux config” and press Enter

This opens Neovim directly on dot_tmux.conf in the chezmoi directory. After making changes, you can source the config with tmux source-file ~/.tmux.conf from within Neovim’s terminal or a separate pane.

Start a session for a project you recently visited

1. Open the ssh picker with prefix + T
2. Press Ctrl+x to switch to zoxide mode
3. Type the project name to filter
4. Select the directory and press Enter

Sesh creates a new tmux session named after the directory and sets the working directory. Because zoxide tracks frequency, your most-used projects appear near the top.

Discover and open a new project

1. Open the ssh picker with prefix + T
2. Press Ctrl+f to switch to find mode
3. The picker now shows directories found by fd -H -d 2 -t d . ~ - all directories up to 2 levels deep from your home directory, including hidden ones
4. Select a directory and press Enter

This is useful for projects you have not visited yet (so they are not in zoxide’s database) but that exist in a standard location like `~/Projects/` or `~/Work/`.

Kill a session you no longer need

1. Open the sesh picker with prefix `+ T`
2. Navigate to the session you want to remove
3. Press `Ctrl+d` to kill it

The session is killed and the list reloads. This is faster than running `tmux kill-session -t name` manually.

Use sesh from the command line

You do not have to use the tmux keybinding. Sesh works as a standalone CLI tool:

```
# List all available sessions
sesh list
```

```
# Connect directly to a named session
sesh connect dotfiles
```

```
# Connect to a directory as a session
sesh connect ~/Projects/webapp
```

This is useful in scripts or when you are outside tmux and want to start a specific session.

Advanced Usage

How sesh connect Works

When you run `sesh connect <name>`:

1. Sesh checks if a tmux session with that name already exists. If yes, it switches to it.
2. If not, sesh checks if the name matches a `[[session]]` entry in `sesh.toml`. If yes, it creates a new tmux session with the configured path and runs the startup command.
3. If not, sesh treats the name as a directory path. If the directory exists, it creates a session named after the directory basename with that directory as the working directory.
4. If none of the above match, the command fails.

This cascading logic means `sesh connect` does the right thing regardless of whether you pass a session name, a config name, or a path.

Preview Commands

The `preview_command` field in `sesh.toml` controls what `fzf` shows in the preview pane when a session is highlighted. Choosing good preview commands makes the picker much more useful:

- **bat** for file previews: Shows syntax-highlighted file contents. Good for sessions focused on a specific config file, like the “tmux config” session previewing `dot_tmux.conf`.
- **eza -tree** for directory overviews: Shows the directory structure with icons. Good for sessions focused on a project directory, like the “nvim config” session showing the Neovim config tree.

- **ls** or other simple commands: For sessions where a quick listing is sufficient.

The preview command runs every time you move the cursor in the picker, so keep it fast. Avoid commands that take more than a second to execute.

The fd-Based Find Mode

The find mode (Ctrl+f) runs `fd -H -d 2 -t d . ~`, which finds:

- `-H`: Hidden directories (like `.config`, `.local`)
- `-d 2`: Up to 2 levels deep from home
- `-t d`: Only directories (not files)
- `. ~`: Starting from the home directory

This gives you a broad but manageable list of project directories. The depth limit of 2 means you see `~/Projects/webapp` but not `~/Projects/webapp/src/components`. Adjust the depth if your project layout is deeper.

Session Naming from Zoxide Directories

When `sesh` creates a session from a zoxide directory, it uses the directory's basename as the session name. If you select `~/Projects/my-webapp`, the session is named `my-webapp`. This means the tmux status bar shows the project name, which is exactly what you want.

Be aware that tmux session names cannot contain dots or colons. If a directory name contains these characters, `sesh` may modify the name or the session creation may fail. Stick to alphanumeric names with hyphens and underscores for project directories.

Combining sesh with the IDE Layout

The `sesh` picker and the `ide()` function (covered in the IDE Layout chapter) serve different purposes but complement each other:

- **sesh** creates simple sessions with a single window and a startup command. Use it for quick access to config files, downloads, or lightweight tasks.
- **ide()** creates a full three-pane layout with Claude Code, Neovim, and a terminal. Use it for development work where you want the complete IDE experience.

A common workflow: use `sesh` to jump between project sessions for quick checks, and use `ide` when you are sitting down for focused development on a project.

Startup Command Limitations

The `startup_command` runs once when the session is first created. If you switch away from the session and come back, the startup command does not re-run – you return to whatever state the session is in. This is by design: `sesh` creates the session, and `tmux-resurrect/continuum` handle persistence from there.

If the startup command fails (for example, if `nvim` is not installed), the session is still created but the command's error output appears in the terminal. The session's working directory is still set correctly.

Customizing the Picker Appearance

The `fzf-tmux` popup is configured with `-p 55%,60%`, which creates a centered floating window at 55% width and 60% height. The `--no-sort` flag preserves `sesh`'s native ordering (config sessions first, then `tmux` sessions, then `zoxide`). The `--ansi` flag enables color output in the session list.

The header line shows the available mode-switching keybindings as a quick reference. The border label shows “`sesh`” to identify the picker.

When to Use `sesh` vs. `tmux` Directly

Use `sesh` for:

- Switching between known projects and config contexts
- Creating sessions with specific working directories and startup commands
- Discovering directories via `zoxide` or `fd`
- Killing sessions from a unified interface

Use `tmux` directly for:

- Creating ad-hoc sessions for one-off tasks: `tmux new -s scratch`
- Renaming sessions: `prefix + $`
- Moving windows between sessions: `prefix + :` then `move-window`
- Advanced session manipulation that `sesh` does not cover

`Sesh` is the 90% case – most session management flows through the picker. Fall back to raw `tmux` commands for the remaining 10%.

IDE Layout

- A three-pane tmux layout that puts Claude Code, Neovim, and a terminal side by side
- the complete AI-assisted development environment in a single window.

Your Setup

The IDE layout is defined in two places: a tmuxinator template at `~/.config/tmuxinator/ide.yml` and a shell function in `~/.aliases.zsh`.

The tmuxinator template:

```
name: <%= File.basename(File.expand_path(@args[0] || ".")) %>
root: <%= @args[0] || "." %>
```

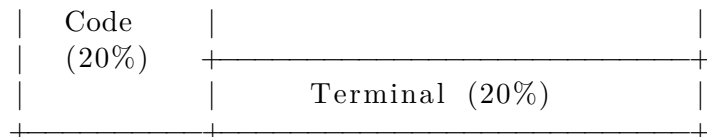
```
windows:
  - code:
      layout: main-vertical
      panes:
        - claude
        - nvim .
        - # terminal
```

The `ide()` shell function that orchestrates the layout:

```
ide() {
  local dir="${1:-.}"
  local name=$(basename "$(cd "$dir" 2>/dev/null;&&pwd)")
  tmuxinator start ide "$dir" --no-attach
  tmux resize-pane -t "$name:1.0" -x '20%'
  tmux resize-pane -t "$name:1.2" -y '20%'
  tmux select-pane -t "$name:1.1"
  tmux rename-window -t "$name:1" "$name"
  tmux attach-session -t "$name"
}
```

The resulting layout:

```
+-----+
| Claude | Neovim (80%) |
+-----+
```



- **Pane 0** (left, 20% width): Claude Code – an AI assistant in the terminal
- **Pane 1** (top-right, 80% width, 80% height): Neovim – the primary editor
- **Pane 2** (bottom-right, 80% width, 20% height): A bare terminal for running commands

Core Concepts

Why This Layout

The three-pane layout is designed around a specific workflow: AI-assisted development. Each pane has a clear role:

- **Claude Code** on the left is the conversational interface. You describe what you want to build, ask questions about code, or request reviews. It occupies 20% of the width because conversation text does not need wide columns.
- **Neovim** in the top-right is where you write and edit code. It gets the largest share of screen space (80% width, 80% height) because reading and writing code demands the most visual real estate.
- **Terminal** in the bottom-right is for running commands: builds, tests, git operations, server processes. It occupies 20% of the height on the right side – enough to see output without stealing space from the editor.

This mirrors the layout of modern IDEs like VS Code (sidebar, editor, terminal panel) but uses terminal-native tools that are faster, more configurable, and work over SSH.

How tmuxinator Works

Tmuxinator is a tmux session manager that creates sessions from YAML templates. The `ide.yml` template uses ERB (embedded Ruby) to accept a directory argument:

- `<%= File.basename(File.expand_path(@args[0] || ".") %>` extracts the directory name for the session name. If you run `ide ~/Projects/webapp`, the session is named `webapp`.
- `<%= @args[0] || "." %>` sets the root directory. All panes start in this directory.
- The layout: `main-vertical` arranges panes in tmux's `main-vertical` layout, which places one pane on the left and stacks the rest on the right.
- The three panes are defined in order: `claude` (runs the `claude` command), `nvim .` (opens Neovim in the project directory), and a blank pane (bare terminal).

The `ide()` Function: Why Not Just tmuxinator?

Tmuxinator creates the session and panes, but it does not give precise control over pane sizes. The `main-vertical` layout splits space evenly among the stacked panes and uses a default main pane size. The `ide()` function fixes this:

1. **tmuxinator start ide "\$dir" --no-attach** – Creates the session in the background without attaching. This is critical because the `resize` commands need to run after the session exists but before you see it.

2. **tmux resize-pane -t "\$name:1.0" -x '20%'** – Shrinks the Claude pane (pane 0) to 20% width, giving the right side 80%.
3. **tmux resize-pane -t "\$name:1.2" -y '20%'** – Shrinks the terminal pane (pane 2) to 20% height, giving Neovim 80% of the right column.
4. **tmux select-pane -t "\$name:1.1"** – Sets focus on the Neovim pane. When you attach, your cursor is in the editor, ready to code.
5. **tmux rename-window -t "\$name:1" "\$name"** – Renames the window from “code” (the tmuxinator template name) to the project name.
6. **tmux attach-session -t "\$name"** – Finally attaches to the session, showing the finished layout.

Pane Addressing

Tmux addresses panes with the format `session:window.pane`:

- `$name:1.0` – Session named after the directory, window 1 (base-index is 1), pane 0
- `$name:1.1` – Same session and window, pane 1 (Neovim)
- `$name:1.2` – Same session and window, pane 2 (terminal)

Panes within a window are numbered starting from 0 regardless of the `pane-base-index` setting in this context because tmuxinator creates them in order.

Essential Commands

Launching the IDE

Command	Action
<code>ide</code>	Open IDE layout in current directory
<code>ide ~/Projects/webapp</code>	Open IDE layout for a specific project
<code>ide .</code>	Explicit current directory (same as <code>ide</code>)

Navigating Between Panes

Keybinding	Direction
<code>Ctrl+h</code>	Move to the pane on the left
<code>Ctrl+l</code>	Move to the pane on the right
<code>Ctrl+j</code>	Move to the pane below
<code>Ctrl+k</code>	Move to the pane above

These keybindings work via `vim-tmux-navigator`, meaning they cross both tmux pane boundaries and Neovim split boundaries seamlessly. From Neovim, `Ctrl+h` moves to Claude Code. From the terminal, `Ctrl+k` moves up to Neovim. From Claude, `Ctrl+l` moves to Neovim.

Managing the Layout

Keybinding	Action
prefix + z	Zoom current pane to full screen
prefix + z (again)	Restore zoomed pane
prefix + x	Close current pane
prefix + !	Break pane into new window

Window Utilities

Keybinding/Command	Action
tab name	Rename current tmux window
prefix + c	Create a new window
Shift+Left/Right	Switch between windows
Alt+H/L	Switch windows (vim-style)

Practical Recipes

Start a new project session

```
ide ~/Projects/new-api
```

This creates a tmux session named `new-api` with the three-pane layout. Claude Code starts on the left, Neovim opens the project directory on the right, and a terminal sits below the editor. Your cursor starts in Neovim.

The AI-assisted development loop

This is the primary workflow the IDE layout is built for:

1. **Discuss in Claude** (left pane): Describe the feature you want to implement, paste error messages, or ask for code review. Navigate to Claude with `Ctrl+h`.
2. **Code in Neovim** (top-right pane): Write the implementation based on Claude's suggestions. Navigate to Neovim with `Ctrl+l` from Claude, or `Ctrl+k` from the terminal.
3. **Test in terminal** (bottom-right pane): Run builds, tests, or the development server. Navigate to the terminal with `Ctrl+j` from Neovim.
4. **Iterate**: If tests fail, move back to Claude to discuss the error, then to Neovim to fix it, then to the terminal to test again. The three keybindings (`Ctrl+h`, `Ctrl+l`, `Ctrl+j`) become muscle memory.

Zoom a pane for focused work

Sometimes you need full-screen focus. Press `prefix + z` to zoom the current pane:

- Zoom Neovim when editing a complex file and you need maximum screen space
- Zoom Claude when reading a long explanation
- Zoom the terminal when debugging verbose output

Press `prefix + z` again to snap back to the three-pane layout.

Run a long process in the terminal pane

Navigate to the terminal pane (Ctrl+j from Neovim) and start a development server or test watcher:

```
npm run dev
# or
cargo watch -x test
# or
python manage.py runserver
```

The process runs continuously in the terminal pane. Switch back to Neovim with Ctrl+k to keep coding. The terminal pane is visible at the bottom, so you can glance at server output or test results without switching panes.

Use the floating terminal for quick commands

If you need to run a one-off command without leaving your current pane, use floax: prefix + p. A floating terminal appears at 80% screen size, runs your command, and prefix + p again dismisses it. This is ideal for:

- Quick git status or git diff checks
- Installing a package
- Looking up a file path
- Running a single test

Add a fourth pane

The IDE layout starts with three panes, but you can add more. To split the terminal pane horizontally and run two processes side by side:

```
prefix + "    # Split the terminal pane
```

Now the bottom-right has two stacked panes. Navigate between all panes with Ctrl+h/j/k/l. To return to the standard three-pane layout, close the extra pane with prefix + x.

Open multiple IDE windows

You can create IDE sessions for multiple projects simultaneously:

```
ide ~/Projects/frontend
# Detach with prefix + d
ide ~/Projects/backend
```

Now you have two IDE sessions. Switch between them with the sesh picker (prefix + T) or sessionx (prefix + o). Each session maintains its own three-pane layout independently.

Rename the window for clarity

The ide() function names the window after the project directory. If you want a different name:

```
tab api-server
```

The tab() function (defined in ~/.aliases.zsh) renames the current tmux window. This updates the status bar display.

Advanced Usage

Why `--no-attach` Is Essential

The `ide()` function uses `tmuxinator start ide "$dir" --no-attach` to create the session without attaching to it. This is not just a convenience – it is required for the resize commands to work correctly.

If `tmuxinator` attaches immediately, you see the default layout for a split second before the resize commands run (they cannot run because you are already inside the session and the function has not continued). By creating the session detached, the function runs all resize and focus commands against the session from outside, then attaches to the finished layout. The user sees only the final, correctly sized layout.

Pane Size Percentages

The percentage-based resize commands (`-x '20%'` and `-y '20%'`) calculate sizes relative to the terminal window. This means the layout adapts to different screen sizes:

- On a 1920-wide terminal: Claude gets ~384 columns, Neovim gets ~1536
- On a 2560-wide ultrawide: Claude gets ~512 columns, Neovim gets ~2048
- On a smaller laptop screen: Claude gets ~256 columns, Neovim gets ~1024

The 20/80 split works well across these sizes. Claude Code renders conversation text well in narrow columns, while Neovim benefits from the extra width for code, file trees, and split views.

The main-vertical Layout

`Tmux`’s main-vertical layout places the first pane on the left as the “main” pane and stacks all remaining panes vertically on the right. This is why the order of panes in `ide.yml` matters:

1. First pane (claude) becomes the left column
2. Second pane (nvim .) becomes the top of the right column
3. Third pane (terminal) becomes the bottom of the right column

The `ide()` function then overrides the default sizing with explicit percentages.

Customizing the Layout for Different Workflows

You can create variations of the IDE layout for different needs. For example, a documentation writing layout:

```
# ~/.config/tmuxinator/docs.yml
name: <%= File.basename(File.expand_path(@args[0] || ".")) %>
root: <%= @args[0] || "." %>
```

windows:

- writing:
 - layout: main-vertical
 - panes:
 - nvim .
 - # preview terminal

And a corresponding shell function with different proportions.

Session Persistence with the IDE Layout

The `tmux-resurrect` and `tmux-continuum` plugins (covered in the Tmux Plugins chapter) save and restore the IDE layout automatically. When tmux restores after a reboot:

- The three-pane layout is recreated
- Neovim restarts with its saved session (via `resurrect--strategy--nvim session`)
- The terminal pane returns to the project directory
- Claude Code needs to be restarted manually (AI sessions are not persistent across reboots)

After restoration, navigate to the Claude pane (Ctrl+h) and run `claude` to restart the AI assistant.

Integration with sesh

The IDE layout and `sesh` serve complementary purposes. `Sesh` creates simple single-window sessions from config bookmarks and `zoxide` directories. The IDE layout creates a structured multi-pane workspace. You might use `sesh` for quick config edits and the IDE layout for sustained development work.

A workflow pattern:

1. Start your day by running `ide ~/Projects/main--project` for your primary work
2. Use prefix + T (`sesh`) to quickly check dotfiles or another config
3. Use Shift+Left/Right or Alt+H/L to switch windows if you add more to the IDE session
4. At the end of the day, detach (prefix + d) – `tmux-continuum` saves everything

Troubleshooting the Layout

Panes are the wrong size: The resize commands depend on the session name matching the directory basename. If the directory name contains special characters, the `$name` variable may not match the tmux session name. Stick to alphanumeric directory names with hyphens and underscores.

Claude pane shows an error: If the `claude` command is not installed or not in your `PATH`, the pane opens a shell instead. Install Claude Code or replace `claude` in the `tmuxinator` template with another command.

Neovim does not open: If `nvim` is not in `PATH`, the pane shows an error. The `tmuxinator` template runs `nvim .` which opens Neovim in the project directory.

Layout looks different on a small screen: The percentage-based resizing adapts to screen size, but on very small terminals (under 120 columns), the 20% Claude pane may be too narrow to be useful. Consider increasing it to 25% or 30% by editing the `ide()` function.

Extending the ide() Function

You can add steps to the `ide()` function for project-specific setup:

```
ide() {
    local dir="${1:-.}"
    local name=$(basename "$(cd "$dir" && pwd)")
}
```

```

tmuxinator start ide "$dir" --no-attach
tmux resize-pane -t "$name:1.0" -x '20%'
tmux resize-pane -t "$name:1.2" -y '20%'
tmux select-pane -t "$name:1.1"
tmux rename-window -t "$name:1" "$name"

# Optional: send a command to the terminal pane
# tmux send-keys -t "$name:1.2" "npm run dev" Enter

tmux attach-session -t "$name"
}

```

Uncommenting the `send-keys` line would start a dev server in the terminal pane automatically. You could also add conditional logic to detect project types (check for `package.json`, `Cargo.toml`, etc.) and run appropriate commands.

The `tab()` Helper

The `tab()` function from `~/aliases.zsh` provides a quick way to rename tmux windows:

```

tab() {
  if [ -n "$TMUX" ]; then
    tmux rename-window "$1"
  else
    echo -ne "\033]1;$1\007"
  fi
}

```

Inside tmux, it renames the window. Outside tmux, it sets the terminal tab title using an escape sequence. This dual behavior means `tab myname` always does the right thing regardless of context.

Git Workflow

A rebase-centric workflow backed by delta diffs, Oh My Zsh aliases, and an automatic rsync backup on every commit.

Your Setup

Git is configured in `~/.gitconfig` (chezmoi source: `dot_gitconfig`):

```
[user]
  name = zero-dev001
  email = 316405743+zero-dev001@users.noreply.github.com
[core]
  editor = vi
  hooksPath = ~/.githooks
  excludesfile = ~/.gitignore
  pager = delta
[init]
  defaultBranch = main
[interactive]
  diffFilter = delta --color-only
[delta]
  navigate = true
  dark = true
  side-by-side = true
  line-numbers = true
  syntax-theme = Catppuccin Mocha
[merge]
  conflictstyle = diff3
[diff]
  colorMoved = default
```

Key decisions encoded in this configuration:

- **vi as editor** – commit messages open in vi, not Neovim, keeping git operations lightweight and fast.
- **Custom hooks path** – all git hooks live in `~/.githooks/`, managed by chezmoi, so every repository on the machine shares the same post-commit backup behavior.
- **Global excludes** – `~/.gitignore` contains patterns for macOS (`.DS_Store`), editors (`.swp`, `.swo`), and environment files (`.env`) so they never appear in any repository's untracked files.

- **Delta as pager** – every git command that pages output (diff, log, show, blame) renders through delta with side-by-side Catppuccin-colored syntax highlighting.
- **diff3 conflict style** – merge conflicts show three sections (ours, base, theirs) instead of two, making it clear what the original code looked like before either branch changed it.
- **colorMoved** – when lines are moved rather than changed, git highlights them differently so you can distinguish a genuine edit from a reorganization.

Oh My Zsh's git plugin is loaded in ~/.zshrc, providing a comprehensive set of aliases that form the muscle memory for daily git operations.

Core Concepts

The Commit Cycle

Every change follows the same four-step loop: check status, stage changes, commit, push. The Oh My Zsh aliases compress this into single commands.

Branching Model

This setup uses a simple branching model:

- main is the default branch and the source of truth.
- Feature branches are created from main, named descriptively (e.g., feat/add-search, fix/login-timeout).
- Branches are rebased onto main before merging to keep a linear history.
- Branches are deleted after merge.

The diff3 Conflict Style

Standard git conflicts show two sections:

```
<<<<<<< HEAD
our changes
=====
their changes
>>>>>>> feature-branch
```

With `conflictstyle = diff3`, you see three:

```
<<<<<<< HEAD
our changes
||||||| parent of abc1234
the original code before either branch touched it
=====
their changes
>>>>>>> feature-branch
```

The middle section is the common ancestor. This tells you what each side intended to change, making conflict resolution dramatically easier. Without it, you are guessing what the original looked like.

The Post-Commit Backup Hook

Every repository on the machine triggers a post-commit hook defined in `~/.git/hooks/post-commit`. This hook runs an rsync backup of the project source directory to a configurable destination:

```
RSYNC_FLAGS=(-a -h --exclude=.git/ '--filter=:-.gitignore'
              "--exclude-from=$HOME/.gitignore" --delete)
```

The hook respects both the repository's `.gitignore` and the global `~/.gitignore`, so only tracked and meaningful files are backed up. It can run synchronously (default) or asynchronously (`ASYNC=1`). The backup log is written to `~/.githook_backup.log`.

This means every commit creates a point-in-time snapshot of your project in a separate location, independent of git. If the git repository is corrupted or deleted, you have a clean rsync copy.

Essential Commands

Daily Workflow (Oh My Zsh Aliases)

Alias	Expands To	Action
gst	git status	Show working tree status
ga	git add	Stage specific files
gaa	git add --all	Stage all changes
gc	git commit --verbose	Commit with diff shown in editor
gcam	git commit -a -m	Stage all and commit with message
gp	git push	Push to remote
gl	git pull	Pull from remote
gf	git fetch	Fetch from remote

Branching

Alias	Expands To	Action
gco	git checkout	Switch branches
gcb	git checkout -b	Create and switch to new branch
gb	git branch	List branches
gbd	git branch -d	Delete a branch (safe)
gm	git merge	Merge a branch

Viewing History

Alias	Expands To	Action
glo	git log --oneline --decorate	Compact log
glg	git log --stat	Log with file change stats
gd	git diff	Diff unstaged changes
gds	git diff --staged	Diff staged changes

Stashing

Alias	Expands To	Action
gsta	git stash push	Stash current changes
gstp	git stash pop	Apply and remove top stash

Practical Recipes

Start a feature branch

```
gco main          # Switch to main
gl               # Pull latest changes
gcb feat/new-thing # Create and switch to feature branch
```

The full commit cycle

```
gst             # Check what changed
gd             # Review unstaged diffs (renders in delta, side-by-side)
ga src/app.ts  # Stage specific files
gds           # Review staged diffs before committing
gc            # Commit (opens vi for message)
gp           # Push to remote
```

Quick commit everything

```
gcam "fix: resolve timeout in auth flow"
gp
```

This stages all changes and commits with a message in one command. Use this for small, self-contained fixes. Avoid it for large changes where you want to review what you are staging.

Stash work in progress

```
gsta          # Stash current changes
gco main      # Switch to main for a hotfix
# ... do the hotfix ...
gco feat/my-thing # Return to feature branch
gstp         # Pop the stash
```

Rebase onto main

```
gco main      # Switch to main
gl           # Pull latest
gco feat/my-thing # Back to feature branch
git rebase main # Rebase onto latest main
```

If conflicts arise during rebase, the diff3 style shows three-way context. Fix each conflict, then:

```
ga .                                # Stage resolved files
git rebase —continue              # Continue the rebase
```

To abort a rebase that has gone wrong:

```
git rebase —abort
```

Resolve a merge conflict with diff3

When you see a diff3 conflict block:

1. Read the middle section (||||| parent) to understand the original code.
2. Read the top section (<<<<<< HEAD) to understand your changes relative to the original.
3. Read the bottom section (>>>>>> branch) to understand their changes relative to the original.
4. Write the correct resolution that incorporates both intents.
5. Remove all conflict markers.
6. Stage and continue.

This is categorically easier than two-way conflicts because you can see what each side was trying to do.

Interactive rebase to clean up commits

Before merging a feature branch, clean up the commit history:

```
git rebase -i main
```

This opens vi with a list of commits. Common operations:

- pick – keep the commit as-is
- squash – combine with the previous commit
- reword – change the commit message
- drop – remove the commit entirely

View a specific file's history

```
git log —follow -p — src/utils.ts
```

This shows every commit that touched the file, with full diffs rendered through delta. The —follow flag tracks the file across renames.

Blame with delta coloring

```
git blame src/app.ts
```

Because delta is the pager, blame output is syntax-highlighted and easier to read than default git blame.

Advanced Usage

The Post-Commit Hook in Detail

The post-commit hook at `~/.githooks/post-commit` is a chezmoi template (`executable_post-commit.tpl`). It uses chezmoi template variables to set the source and destination paths:

```
SRC="{SRC:~/.projectSrcPath}"
DEST="{DEST:~/.backupDestPath}"
```

These paths are resolved from chezmoi's data file at apply time, so different machines can back up to different locations. The hook:

1. Creates the destination directory if it does not exist.
2. Runs `rsync` with archive mode, excluding `.git/`, respecting `.gitignore` filters, and deleting files from the destination that no longer exist in the source.
3. Logs all output to `~/.githook_backup.log`.

To run asynchronously (for large repos where you do not want to wait):

```
ASYNC=1 git commit -m "large change"
```

Overriding the Hook Per-Repository

If a specific repository should not trigger the backup, override the hooks path:

```
git config --local core.hooksPath .githooks
```

This points that repository to its own hooks directory, bypassing the global one.

colorMoved and Code Refactoring

The `diff.colorMoved = default` setting changes how git renders diffs when lines are moved rather than modified. Moved lines appear in a distinct color (dimmed text) rather than showing as a deletion and an addition. This is invaluable during refactoring – when you move a function from one file to another, the diff clearly shows it was moved, not rewritten.

Combining Aliases for Complex Workflows

Oh My Zsh aliases compose naturally with standard git arguments:

```
glo --all --graph      # Decorated graph log across all branches
gd --stat              # Diff summary showing files changed
gb -a                 # List all branches including remotes
gbd -D feature/old     # Force-delete a branch (uppercase D)
```

Excluding Files Globally

The global `~/.gitignore` referenced by `core.excludesfile` contains patterns that should never be tracked in any repository:

```
.DS_Store
*.swp
*.swp
.env
.env.local
node_modules/
.idea/
.vscode/
```

This keeps repository-level `.gitignore` files focused on project-specific patterns.

Safe Directory for Homebrew

The `safe.directory = /opt/homebrew` setting tells git to trust the Homebrew installation directory. Without this, git operations in `/opt/homebrew` fail with an ownership warning because the directory is often owned by a different user or group.

Beautiful Diffs: delta

A syntax-highlighting pager that replaces git's default diff output with side-by-side, line-numbered, Catppuccin-colored diffs you can navigate like a document.

Your Setup

Delta is configured in `~/.gitconfig` as both the core pager and the interactive diff filter:

```
[core]
  pager = delta
[interactive]
  diffFilter = delta --color-only
[delta]
  navigate = true
  dark = true
  side-by-side = true
  line-numbers = true
  syntax-theme = Catppuccin Mocha
```

This configuration means:

- **Every git command that produces paged output** – `git diff`, `git log -p`, `git show`, `git blame`, `git stash show -p` – renders through delta automatically.
- **Interactive staging** (`git add -p`) uses delta's color-only mode for syntax highlighting without the side-by-side layout (which would break the interactive interface).
- **Navigation mode** is enabled, allowing you to jump between file diffs with `n` and `N` keys.
- **Side-by-side** layout shows old and new versions of each file in parallel columns.
- **Line numbers** appear in the gutter of both sides.
- **Catppuccin Mocha** theme ensures diffs match the color palette used in iTerm2, Neovim, lazygit, and every other tool in the environment.

Delta is installed via Homebrew:

```
brew install git-delta
```

The companion tool `diffnav` is also installed, providing an alternative navigation interface for delta output.

Core Concepts

What Delta Replaces

Without delta, git diff output looks like this:

```
diff --git a/src/app.ts b/src/app.ts
index abc1234..def5678 100644
--- a/src/app.ts
+++ b/src/app.ts
@@ -10,7 +10,7 @@ function main() {
- const timeout = 5000;
+ const timeout = 10000;
```

Plain text, no syntax highlighting, no line numbers, unified format only. For small changes this is adequate. For large diffs across multiple files, it becomes difficult to read.

With delta, the same diff renders as:

- Two columns side by side: the left column shows the old version, the right column shows the new version.
- Full syntax highlighting using Treesitter-quality parsing for the file's language.
- Line numbers in the gutter of both columns.
- Additions highlighted in green, deletions in red, using the Catppuccin Mocha palette.
- Moved lines highlighted differently from changed lines (because `diff.colorMoved = default is set`).
- File headers clearly delineated with the filename and change summary.

Side-by-Side Mode

Side-by-side diffs show the before and after states in parallel columns. This layout has several advantages:

- **Context is visible on both sides.** You do not need to mentally reconstruct what the old code looked like while reading the new code – both are right there.
- **Line alignment.** Corresponding lines are vertically aligned, making it easy to see exactly what changed on each line.
- **Large diffs are scannable.** You can quickly scroll through a multi-file diff and spot the important changes because the layout is denser than unified format.

The trade-off is horizontal space. On a typical terminal (120+ columns), side-by-side works well. On very narrow terminals, lines may wrap. You can temporarily disable it:

```
git diff --no-ext-diff
```

Or pass delta flags directly:

```
git diff | delta --side-by-side=false
```

Navigation with n/N

When `navigate = true`, delta inserts special markers at file boundaries. Inside the pager (less), you can press:

Key	Action
n	Jump to the next file in the diff
N	Jump to the previous file in the diff

This transforms large diffs from a wall of text into a navigable document. Instead of scrolling through hundreds of lines, you jump directly to the file you care about.

Syntax Themes

Delta supports any syntax theme that bat supports, because both tools use the same syntect highlighting engine. The Catppuccin Mocha theme is specified to match the rest of the environment:

- Additions use Catppuccin green (#a6e3a1)
- Deletions use Catppuccin red (#f38ba8)
- Syntax colors (keywords, strings, functions) match what you see in Neovim and bat
- The dark background matches the terminal's base color (#1e1e2e)

Essential Commands

Commands That Automatically Use Delta

Command	What Delta Renders
git diff (alias: gd)	Unstaged changes, side-by-side
git diff --staged (alias: gds)	Staged changes, side-by-side
git log -p	Commit history with full diffs
git show <commit>	Single commit with diff
git blame <file>	Annotated file with syntax highlighting
git stash show -p	Stash contents with diff
git diff branch1..branch2	Differences between two branches
git diff HEAD~3..HEAD	Last three commits as a diff

Delta-Specific Navigation (Inside the Pager)

Key	Action
n	Next file in diff
N	Previous file in diff
q	Quit the pager
/pattern	Search forward for text
?pattern	Search backward for text
g	Go to top of output
G	Go to bottom of output
Space	Page down
b	Page up
j / k	Scroll down / up one line

Overriding Delta Temporarily

Command	Effect
<code>git diff --no-ext-diff</code>	Bypass delta entirely, use default git diff
<code>git diff \ delta --side-by-side=false</code>	Pipe through delta without side-by-side
<code>git diff \ delta --line-numbers=false</code>	Pipe through delta without line numbers
<code>GIT_PAGER=cat git diff</code>	Use cat as pager (plain text, no delta)

Practical Recipes

Review a feature branch diff

```
git diff main..feat/my-branch
```

Delta renders the entire diff between main and the feature branch in side-by-side format. Press `n` to jump between files, focusing on the ones you care about.

Review a single commit

```
git show abc1234
```

Shows the commit message followed by the full diff, syntax-highlighted and side-by-side.

Compare two commits

```
git diff abc1234..def5678
```

Delta renders the difference between any two commits with full highlighting.

Review what you are about to commit

```
gds          # git diff --staged
```

Before running `gc` (git commit), always review staged changes. Delta's side-by-side view makes it easy to catch mistakes, debug output, or unintended changes.

View a file's history with diffs

```
git log -p --follow -- src/utils.ts
```

Every commit that touched the file is shown with its full diff. Press `n` to jump between commits (each commit's diff is treated as a separate file section by delta's navigator).

Blame with syntax highlighting

```
git blame src/app.ts
```

Delta syntax-highlights the blame output, making it much easier to read than the default monochrome output. Each line shows the commit hash, author, date, and the code itself in full color.

View stash contents before applying

```
git stash show -p stash@{0}
```

See exactly what a stash contains before deciding to apply it. The side-by-side view shows what would change.

Diff only specific file types

```
git diff -- '*.ts' '*.tsx'
```

Limit the diff to TypeScript files. Delta still renders everything in side-by-side mode with correct syntax highlighting for the file type.

Advanced Usage

How Delta Integrates with the Pager

Delta produces output with ANSI color codes and passes it to `less` (the default pager for `less -R`). The `navigate = true` option inserts special section markers that `less` can jump between. This is why `n/N` navigation works – delta is using `less`'s built-in section-jump feature.

The Interactive Diff Filter

The `interactive.diffFilter = delta --color-only` setting is specifically for `git add -p` (interactive staging). In this mode, `git` shows you hunks one at a time and asks whether to stage each one. The `--color-only` flag tells delta to add syntax highlighting but not reformat the output into side-by-side mode, which would break the interactive interface.

This means when you run:

```
git add -p src/app.ts
```

Each hunk is syntax-highlighted with Catppuccin colors, but displayed in the standard unified format that the interactive staging interface expects.

Delta and colorMoved

The `diff.colorMoved = default` setting in `gitconfig` works in concert with delta. When `git` detects that lines were moved rather than changed (for example, moving a function from the top of a file to the bottom), it marks those lines with a special attribute. Delta then renders moved lines in a distinct color, separate from additions and deletions.

This is especially useful during refactoring. A diff that appears to delete 50 lines and add 50 identical lines elsewhere is actually just a move – and delta makes that visually obvious.

The diffnav Tool

The `diffnav` tool provides an alternative way to navigate delta output. Instead of using `n/N` within `less`, `diffnav` presents a file picker at the top of the screen, letting you select which file's diff to view. It is installed alongside delta and can be used as:

```
git diff | diffnav
```

This is particularly useful for large diffs spanning many files, where the file picker gives you a better overview than sequential navigation.

Performance Considerations

Delta adds overhead to every git diff operation because it performs syntax highlighting on the fly. For most diffs, this is imperceptible. For extremely large diffs (thousands of lines across dozens of files), you may notice a slight delay. In those cases:

```
git diff --stat                # See which files changed (fast, minimal delta work)
git diff -- path/to/specific.ts # Narrow the diff to one file
```

Using Delta Outside of Git

Delta can highlight any unified diff, not just git diffs:

```
diff -u old_file.ts new_file.ts | delta
```

This is useful for comparing files that are not in a git repository, or for reviewing diff output from other tools.

Delta Configuration Options Not in This Setup

Delta supports many options beyond what is configured here. Some worth knowing:

Option	What It Does
<code>hyperlinks = true</code>	Make file paths clickable in terminals that support OSC 8
<code>file --modified-label = "modified:"</code>	Customize the label shown for modified files
<code>hunk-header-style = file line-number</code>	Show file name and line number in hunk headers
<code>line-numbers-left-format = "{nm:>Q}"</code>	Customize left-side line number format
<code>line-numbers-right-format = "{np:>Q}"</code>	Customize right-side line number format
<code>max-line-length = 512</code>	Truncate very long lines (default: 512)
<code>wrap-max-lines = 2</code>	Maximum lines to use when wrapping long lines

Catppuccin Consistency

The `syntax-theme = Catppuccin Mocha` setting is not arbitrary. It is the same theme used across the entire environment:

Tool	Catppuccin Integration
iTerm2	Catppuccin Mocha color profile
Neovim	Catppuccin Mocha colorscheme
bat	<code>--theme="Catppuccin Mocha"</code>
lazygit	Inherits terminal colors
Starship	Catppuccin Mocha palette in prompt
Delta	<code>syntax-theme = Catppuccin Mocha</code>

When you see a string literal highlighted in peach in Neovim, it is the same peach in a delta diff. When a keyword is mauve in your editor, it is mauve in your diffs. This consistency eliminates the cognitive cost of switching contexts between editing and reviewing code.

Git TUI: lazygit

A full terminal interface for git that makes staging hunks, interactive rebasing, and conflict resolution visual and fast.

Your Setup

Lazygit is installed from the jesseduffield/lazygit Homebrew tap:

```
brew install jesseduffield/lazygit/lazygit
```

Launch it from any git repository:

```
l a z y g i t
```

Lazygit renders a multi-panel terminal interface that replaces the need to memorize dozens of git commands. It integrates with delta for diff rendering, inheriting the Catppuccin Mocha color palette from the terminal.

There is no separate lazygit configuration file in this setup – it uses sensible defaults and picks up delta as the diff pager from gitconfig. The diff3 merge conflict style configured in gitconfig carries through to lazygit’s conflict resolution view.

Core Concepts

The Five Panels

Lazygit divides the terminal into five panels, each accessible by number key or by tabbing between them:

Panel	Key	Contents
Status	1	Current branch, repo name, recent commit
Files	2	Working tree changes (staged and unstaged)
Branches	3	Local and remote branches
Commits	4	Commit history for the current branch
Stash	5	Stashed changes

The active panel is highlighted. The right side of the screen shows a context-sensitive detail view – if you are in the Files panel, it shows the diff for the selected file. If you are in the Commits panel, it shows the diff for the selected commit.

Staging Model

Lazygit distinguishes between three states for changes:

- **Unstaged** – the file appears in red in the Files panel.
- **Staged** – the file appears in green.
- **Partially staged** – some hunks are staged, others are not. The file appears with a mixed indicator.

You can stage entire files, individual hunks, or even individual lines within a hunk. This granular control is one of lazygit's greatest strengths – it replaces the cumbersome `git add -p` workflow with a visual interface.

Navigation Philosophy

Lazygit follows vim-inspired navigation:

- j and k move up and down within a panel.
- h and l (or panel numbers) switch between panels.
- Enter opens a detail view or expands the current item.
- q quits or goes back one level.
- ? shows context-sensitive help for the current panel.

Essential Commands

Global Keys

Key	Action
1-5	Switch to panel by number
q	Quit lazygit (or go back one level)
?	Show help for current panel
/	Filter the current panel's list
Enter	View detail / expand
[/]	Switch tabs within a panel
@	Open command log (see every git command lazygit runs)
+	Toggle view (expand/collapse detail pane)
R	Refresh

Files Panel

Key	Action
Space	Stage or unstage the selected file
a	Stage all files / unstage all files (toggle)
c	Commit staged changes
A	Amend the last commit
d	Discard changes in the selected file
D	Open discard options (unstaged, all, etc.)
e	Open file in editor

Key	Action
o	Open file in default application
i	Add to .gitignore
S	Stash all changes
Enter	View individual hunks for staging

Inside a File (Hunk View)

Key	Action
Space	Stage or unstage the selected hunk
v	Toggle line-by-line selection mode
a	Stage or unstage all hunks in the file
j / k	Move between hunks
Esc	Return to the files list

Branches Panel

Key	Action
n	Create a new branch
Space	Checkout the selected branch
d	Delete the selected branch
r	Rebase current branch onto selected
M	Merge selected branch into current
f	Fetch
Enter	View commits on the selected branch

Commits Panel

Key	Action
s	Squash the selected commit into the one above it
r	Reword the selected commit message
d	Drop the selected commit
e	Edit the selected commit (interactive rebase)
p	Pick (keep) during rebase
f	Fixup (squash without changing message)
Enter	View the full diff of the selected commit
y	Copy commit hash to clipboard
g	Reset options (soft, mixed, hard)
t	Create a tag on the selected commit

Push and Pull

Key	Action
p	Push to remote
P	Pull from remote
shift +p	Push with options (force push shows a confirmation)

Stash Panel

Key	Action
Space	Apply the selected stash
g	Pop the selected stash (apply and delete)
d	Drop the selected stash
n	Create a new stash with a custom name
Enter	View stash contents

Practical Recipes

Stage specific hunks from a file

1. Navigate to the Files panel (2).
2. Select the file with j/k.
3. Press Enter to view hunks.
4. Move between hunks with j/k.
5. Press Space to stage individual hunks.
6. Press Esc to return to the file list.
7. Press c to commit only the staged hunks.

This replaces `git add -p` with a visual interface that shows you exactly what you are staging.

Stage specific lines within a hunk

1. Enter hunk view as above.
2. Press v to enter line-by-line selection mode.
3. Use j/k to select individual lines.
4. Press Space to stage the selected lines.

This level of granularity is nearly impossible to achieve with command-line git.

Interactive rebase to clean history

1. Navigate to the Commits panel (4).
2. Move to the oldest commit you want to modify.
3. Press e to start an interactive rebase from that point.
4. For each commit, use the appropriate key:
 - s to squash into the commit above
 - r to reword the message
 - d to drop
 - f to fixup (squash without message)

- p to pick (keep as-is)
5. Lazygit applies the rebase in real time.

This is dramatically easier than the text editor-based `git rebase -i` because you see the result of each operation immediately.

Resolve merge conflicts

1. After a merge or rebase that produces conflicts, lazygit shows conflicted files in the Files panel.
2. Select a conflicted file and press Enter.
3. The conflict view shows the diff3 sections (ours, base, theirs) when `conflictstyle = diff3` is set.
4. Navigate between conflict markers with j/k.
5. Choose a resolution:
 - b to choose the “base” version
 - j/k to select specific sections
 - Press the displayed key to pick a side
6. After resolving all conflicts in a file, it is automatically staged.
7. Continue the rebase/merge from the Status panel.

Cherry-pick a commit from another branch

1. Navigate to the Branches panel (3).
2. Select the source branch and press Enter to view its commits.
3. Select the commit you want and press c to copy it (cherry-pick).
4. Return to your branch.
5. Press v to paste (apply the cherry-picked commit).

Compare two branches visually

1. Go to the Branches panel (3).
2. Select the branch you want to compare against.
3. Press Enter to see its commits.
4. The diff view on the right shows each commit’s changes, rendered through delta.

Undo a commit (soft reset)

1. Navigate to the Commits panel (4).
2. Select the commit before the one you want to undo.
3. Press g to open reset options.
4. Choose “soft” to undo the commit but keep changes staged.

Force push safely

After an interactive rebase or amend, you need to force push:

1. Press p to push.
2. Lazygit detects the divergence and asks for confirmation.
3. Confirm to force push with lease (which prevents overwriting someone else’s commits).

Advanced Usage

The Command Log

Press @ to open the command log. This shows every git command that lazygit executed on your behalf. This is invaluable for:

- **Learning git** – see the actual commands behind the UI actions.
- **Debugging** – if something went wrong, check what commands ran.
- **Reproducibility** – copy commands to use in scripts or documentation.

Delta Integration

Lazygit automatically uses delta when it is configured as the git pager. This means all diffs shown in the detail pane on the right side of the screen render with:

- Syntax highlighting in Catppuccin Mocha colors.
- Line numbers.
- Side-by-side layout (when the terminal is wide enough).

The diff3 conflict style also carries through, showing three-way conflicts in the merge resolution view.

Lazygit in Neovim

Many developers launch lazygit from within Neovim using a floating terminal plugin. In this setup, you can open a terminal pane in tmux (or use Neovim's built-in terminal) and run lazygit from there. The vim-style keybindings mean the muscle memory transfers seamlessly.

Custom Keybindings

Lazygit's keybindings can be customized in ~/.config/lazygit/config.yml. While this setup uses defaults, common customizations include:

```
keybinding:
  universal:
    quit: 'q'
    return: '<esc>'
  files:
    commitChanges: 'c'
```

Filtering and Searching

Press / in any panel to filter the list:

- In the Files panel, filter by filename.
- In the Branches panel, filter by branch name.
- In the Commits panel, filter by commit message.

This is essential in large repositories with many branches or long commit histories.

Performance on Large Repositories

Lazygit loads git data incrementally, so it starts fast even in repositories with thousands of commits. However, the initial load of the commit graph can take a moment. The Files panel loads almost instantly because it only reads the working tree status.

GitHub CLI and gh-dash

Manage pull requests, issues, and CI runs without leaving the terminal, plus a TUI dashboard that brings your GitHub world into a single view.

Your Setup

The GitHub CLI is installed via Homebrew:

```
brew install gh
```

After installation, authentication is handled with:

```
gh auth login
```

This stores credentials in the system keychain. Once authenticated, every gh command operates against the GitHub API without requiring personal access tokens in environment variables.

The gh-dash extension is installed as a TUI dashboard:

```
gh extension install dlvdhr/gh-dash
```

Both tools work with the git identity configured in ~/.gitconfig:

```
[user]
  name = zero-dev001
  email = 316405743+zero-dev001@users.noreply.github.com
```

Core Concepts

What gh Replaces

Before the GitHub CLI, interacting with GitHub required switching to a browser. Creating a pull request meant opening github.com, navigating to the repository, clicking “New pull request,” filling out a form, and clicking “Create.” Reviewing CI status meant refreshing a browser tab.

With gh, all of these operations happen in the terminal:

- Create, view, and merge pull requests.
- Create and manage issues.
- Monitor GitHub Actions runs.
- Browse repository pages.
- Clone repositories by name (no URL needed).

The gh Extension System

GitHub CLI supports extensions – community-built plugins that add new subcommands. Extensions are installed with `gh extension install` and invoked as `gh <extension-name>`. The `gh-dash` extension adds a `gh dash` command that launches a full TUI dashboard.

gh-dash: The Dashboard

`gh dash` renders a terminal user interface with multiple sections:

- **Pull Requests** – PRs you authored, PRs assigned to you, PRs requesting your review.
- **Issues** – Issues you created, issues assigned to you.
- **Configurable sections** – custom queries for any GitHub search filter.

The dashboard refreshes periodically, giving you a live view of your GitHub activity without opening a browser.

Essential Commands

Pull Request Workflow

Command	Action
<code>gh pr create</code>	Create a PR from the current branch
<code>gh pr create --fill</code>	Create a PR, auto-filling title and body from commits
<code>gh pr create --draft</code>	Create a draft PR
<code>gh pr create --title "..." --body "..."</code>	Create with explicit title and body
<code>gh pr list</code>	List open PRs in the current repo
<code>gh pr list --author @me</code>	List your open PRs
<code>gh pr list --state merged</code>	List merged PRs
<code>gh pr view</code>	View the current branch's PR
<code>gh pr view 42</code>	View PR number 42
<code>gh pr view --web</code>	Open the PR in the browser
<code>gh pr checkout 42</code>	Check out PR number 42 locally
<code>gh pr merge</code>	Merge the current branch's PR
<code>gh pr merge --squash</code>	Squash merge
<code>gh pr merge --rebase</code>	Rebase merge
<code>gh pr merge --delete-branch</code>	Merge and delete the branch
<code>gh pr review</code>	Start a review on the current PR
<code>gh pr review --approve</code>	Approve the PR
<code>gh pr review --request-changes -b "..."</code>	Request changes with a comment
<code>gh pr diff</code>	View the PR diff (rendered through delta)
<code>gh pr checks</code>	View CI check status for the PR

Issue Workflow

Command	Action
gh issue create	Create an issue interactively
gh issue create --title "..." --body "..."	Create with explicit title and body
gh issue create --label bug	Create with a label
gh issue list	List open issues
gh issue list --assignee @me	List your assigned issues
gh issue list --label "priority:high"	Filter by label
gh issue view 42	View issue number 42
gh issue view 42 --web	Open in browser
gh issue close 42	Close an issue
gh issue reopen 42	Reopen a closed issue
gh issue comment 42 --body "..."	Add a comment

Repository Operations

Command	Action
gh repo clone owner/repo	Clone a repository by name
gh repo clone repo	Clone from your own account
gh repo view	View current repo info
gh repo view --web	Open repo in browser
gh repo fork	Fork the current repo
gh browse	Open the repo in the browser
gh browse --settings	Open repo settings
gh browse path/to/file	Open a specific file on GitHub

GitHub Actions

Command	Action
gh run list	List recent workflow runs
gh run view	View the most recent run
gh run view 12345	View a specific run
gh run view --log	View full run logs
gh run watch	Watch a run in progress (live updates)
gh run rerun 12345	Re-run a failed workflow
gh run rerun 12345 --failed	Re-run only the failed jobs

gh-dash Keys

Key	Action
j / k	Move down / up
Enter	View the selected PR or issue
o	Open in browser
d	View diff

Key	Action
c	Checkout the PR
m	Merge the PR
/	Search / filter
s	Switch section (PRs, Issues, custom)
r	Refresh the dashboard
?	Show help
q	Quit

Practical Recipes

Create a PR from a feature branch

```
gco main && gl           # Update main
gcb feat/add-search      # Create feature branch
# ... make changes, commit ...
gp -u origin feat/add-search # Push with upstream tracking
gh pr create --fill       # Create PR with auto-filled title/body
```

The `--fill` flag uses the commit messages from the branch to populate the PR title and body. For a single-commit branch, this is usually perfect. For multi-commit branches, you may want to write a custom body:

```
gh pr create --title "Add_search_functionality" --body "Implements_full-text_search"

## Changes
- Added_search_endpoint
- Added_search_component
- Added_tests

## Testing
Run `npm test` to verify."
```

Review and merge a teammate's PR

```
gh pr list                # See open PRs
gh pr checkout 42         # Check out PR 42 locally
gh pr diff 42             # View the diff (via delta)
# ... test locally ...
gh pr review --approve    # Approve the PR
gh pr merge --squash --delete-branch # Squash merge and clean up
```

Monitor a CI run

```
gh pr checks              # Quick status of checks
gh run watch              # Live output of the running workflow
```

The watch command refreshes every few seconds, showing job progress in real time. When the run completes, it exits with the appropriate exit code (0 for success, 1 for failure).

Create an issue from the terminal

```
gh issue create --title "Login_timeout_too_short" \  
  --body "Users_with_slow_connections_hit_the_5s_timeout." \  
  --label bug --label "priority:high" \  
  --assignee zero-dev001
```

Quick open in browser

When you need the browser for something gh cannot do (like configuring branch protection rules):

```
gh browse                # Opens repo  
gh browse --settings     # Opens settings  
gh pr view --web         # Opens current PR  
gh issue view 42 --web   # Opens issue 42
```

Clone a repo by name

```
gh repo clone zero-dev001/my-project # Full path  
gh repo clone my-project             # From your own account
```

No need to construct a full git URL. gh resolves the repository and clones it.

Search for a PR

```
gh pr list --search "fix_auth"    # Search by text  
gh pr list --state all            # Include closed/merged  
gh pr list --head feat/my-branch  # Find PR for a specific branch
```

View another repo's PRs

```
gh pr list --repo owner/repo  
gh pr view 42 --repo owner/repo
```

You do not need to be inside the repo directory.

Advanced Usage

gh API for Custom Queries

The gh api command provides direct access to the GitHub REST and GraphQL APIs:

```
# Get PR comments  
gh api repos/owner/repo/pulls/42/comments
```

```
# Get your notifications  
gh api notifications
```

```
# GraphQL query  
gh api graphql -f query='
```

```

    query {
      viewer {
        pullRequests(first: 10, states: OPEN) {
          nodes {
            title
            url
          }
        }
      }
    }
  }
'

```

This is useful for automation and for accessing data that the higher-level commands do not expose.

gh-dash Configuration

gh-dash reads its configuration from `~/.config/gh-dash/config.yml`. You can customize:

- Which sections appear (PRs authored, PRs reviewing, custom searches).
- Refresh interval.
- Keybindings.
- Theme colors.

A typical configuration might add a section for PRs in repositories you maintain:

```

prSections:
- title: "My PRs"
  filters: "is:open author:@me"
- title: "Review Requested"
  filters: "is:open review-requested:@me"
- title: "Team PRs"
  filters: "is:open org:my-org"
issuesSections:
- title: "My Issues"
  filters: "is:open author:@me"
- title: "Assigned"
  filters: "is:open assignee:@me"

```

Aliases for Common gh Commands

gh supports custom aliases stored in its config:

```

gh alias set prc 'pr create --fill'
gh alias set prm 'pr merge --squash --delete-branch'
gh alias set prv 'pr view --web'

```

After defining these:

```

gh prc      # Create PR with auto-fill
gh prm      # Squash merge and delete branch
gh prv      # Open PR in browser

```

Using gh in Scripts

gh commands work well in CI/CD scripts and automation:

```
# Wait for checks to pass, then merge  
gh pr checks —watch && gh pr merge —squash —delete-branch
```

```
# Create a release  
gh release create v1.0.0 —generate-notes
```

```
# Download an artifact from a workflow run  
gh run download 12345 —name my-artifact
```

gh with Multiple GitHub Accounts

If you work with multiple GitHub accounts (personal and work), gh supports switching:

```
gh auth login —hostname github.com # Default  
gh auth login —hostname github.enterprise.com # Enterprise  
gh auth switch # Switch between accounts
```

Window Management: Aerospace

A tiling window manager for macOS that organizes windows into workspaces with vim-style keybindings, so you never drag, resize, or hunt for a window again.

Your Setup

Aerospace is configured in `~/.config/aerospace/aerospace.toml` (chezmoi source: `dot_config/aerospace/aerospace.t`

```
start-at-login = true
```

```
enable-normalization-flatten-containers = true
```

```
enable-normalization-opposite-orientation-for-nested-containers = true
```

```
accordion-padding = 0
```

```
default-root-container-layout = 'tiles '
```

```
default-root-container-orientation = 'auto '
```

```
key-mapping.preset = 'qwerty '
```

```
on-focused-monitor-changed = ['move-mouse monitor-lazy-center ']
```

```
on-focus-changed = ['move-mouse window-lazy-center ']
```

Gaps between windows:

```
[gaps]
```

```
inner.horizontal = 10
```

```
inner.vertical = 10
```

```
outer.left = 0
```

```
outer.right = 0
```

```
outer.bottom = 0
```

```
outer.top = [
```

```
  { monitor."built-in" = 10 },
```

```
  { monitor.main = 38 },
```

```
  20,
```

```
]
```

The outer top gap varies by monitor: 10px on the built-in MacBook display, 38px on the main external monitor (to leave room for a menu bar or status bar), and 20px as the default for any other monitors.

Window rules automatically assign applications to workspaces:

```
# WS1: Browsers
[[on-window-detected]]
if.app-name-regex-substring = 'Firefox '
run = 'move-node-to-workspace 1'

# WS2: Terminals
[[on-window-detected]]
if.app-id = 'com.googlecode.itym2 '
run = 'move-node-to-workspace 2'

# WS3: Communication
[[on-window-detected]]
if.app-id = 'com.tinyspeck.slackmacgap '
run = 'move-node-to-workspace 3'

[[on-window-detected]]
if.app-id = 'com.hnc.Discord '
run = 'move-node-to-workspace 3'

[[on-window-detected]]
if.app-id = 'ru.keepcoder.Telegram '
run = 'move-node-to-workspace 3'

# WS4: Other/Media
[[on-window-detected]]
if.app-id = 'com.spotify.client '
run = 'move-node-to-workspace 4'
```

Workspace-to-monitor assignments:

```
[workspace-to-monitor-force-assignment]
1 = 'main '
2 = 'main '
3 = 'main '
4 = 'secondary '
```

Workspaces 1 through 3 live on the main monitor. Workspace 4 (media and other apps) is sent to the secondary monitor, keeping Spotify and auxiliary apps off the primary screen.

Core Concepts

Tiling vs. Floating

Traditional macOS window management is floating: windows overlap, you drag them around, you resize them by grabbing corners, and you lose windows behind other windows. Tiling window managers eliminate all of this. Every window occupies a non-overlapping tile. When you open a new window, existing windows shrink to make room. When you close a window, remaining windows expand. You never manually position anything.

Aerospace defaults to tiling mode (`default-root-container-layout = 'tiles'`). Individual windows can be toggled to floating mode when needed (via service mode).

Workspaces

Workspaces are virtual desktops. This setup uses four:

Workspace	Purpose	Applications
1	Browsers	Firefox
2	Terminals	iTerm2
3	Communication	Slack, Discord, Telegram
4	Media / Other	Spotify

Switching workspaces is instant. Press `Alt+1` to see your browser, `Alt+2` to see your terminals, `Alt+3` to check messages. You never use `Cmd+Tab` to hunt through a list of windows. You know exactly where everything is.

Container Layouts

Aerospace supports two container layouts:

- **Tiles** – windows are placed side by side horizontally or vertically, each taking a proportional share of space. This is the default.
- **Accordion** – windows are stacked in the same space, and you focus through them one at a time. The focused window takes up the full container area.

Toggle between them with `Alt+/'` (tiles) or `Alt+,` (accordion).

Container Orientation

Within a tiles layout, the orientation determines how new windows split:

- **Horizontal** – new windows are placed to the right.
- **Vertical** – new windows are placed below.
- **Auto** – Aerospace chooses based on the available space (wider containers split horizontally, taller ones split vertically).

The default is auto, which produces intuitive layouts without manual intervention.

Normalizations

Two normalizations keep the window tree clean:

- **Flatten containers** – removes unnecessary nesting. If a container has only one child, it is collapsed.
- **Opposite orientation for nested containers** – when you nest a container inside another, the inner container automatically uses the opposite orientation. This prevents deeply nested same-direction splits.

Mouse Follows Focus

```
on-focused-monitor-changed = [ 'move-mouse monitor-lazy-center ' ]
on-focus-changed = [ 'move-mouse window-lazy-center ' ]
```

When you switch focus to a different window or monitor using the keyboard, the mouse cursor moves to the center of the newly focused area. The `lazy-center` variant means the mouse only moves if it is not already inside the target area, preventing jarring cursor jumps when you are already looking at the right place.

Essential Commands

Main Mode Bindings

Focus (Navigate Between Windows)

Binding	Action
Cmd+Shift+H	Focus the window to the left
Cmd+Shift+J	Focus the window below
Cmd+Shift+K	Focus the window above
Cmd+Shift+L	Focus the window to the right
Alt+O	Toggle focus between last two windows

Move Windows

Binding	Action
Ctrl+Shift+H	Move focused window left
Ctrl+Shift+J	Move focused window down
Ctrl+Shift+K	Move focused window up
Ctrl+Shift+L	Move focused window right

Layout

Binding	Action
Alt+/	Toggle tiles: horizontal / vertical
Alt+,	Toggle accordion: horizontal / vertical
Cmd+Shift+F	Toggle fullscreen for focused window

Resize

Binding	Action
Alt+Shift+-	Shrink window by 50px
Alt+Shift+=	Grow window by 50px

Workspaces

Binding	Action
Alt+1	Switch to workspace 1 (Browsers)
Alt+2	Switch to workspace 2 (Terminals)
Alt+3	Switch to workspace 3 (Communication)
Alt+4	Switch to workspace 4 (Media)

Move Window to Workspace

Binding	Action
Alt+Shift+1	Move window to workspace 1
Alt+Shift+2	Move window to workspace 2
Alt+Shift+3	Move window to workspace 3
Alt+Shift+4	Move window to workspace 4

Monitor Management

Binding	Action
Alt+Shift+Tab	Move workspace to next monitor

Mode Switch

Binding	Action
Alt+Shift+;	Enter service mode

Service Mode Bindings

Service mode is for less-common operations. Enter it with Alt+Shift+; and it automatically returns to main mode after each action.

Binding	Action
Esc	Reload config and return to main mode
r	Flatten the workspace tree
f	Toggle floating / tiling for focused window
Backspace	Close all windows except the current one
Alt+Shift+H	Join current window with the container to the left
Alt+Shift+J	Join with the container below
Alt+Shift+K	Join with the container above
Alt+Shift+L	Join with the container to the right

Practical Recipes

Typical development layout

1. Press Alt+2 to switch to workspace 2 (Terminals).
2. Open iTerm2 – it automatically tiles in workspace 2.
3. Open a second iTerm2 window – it splits the space with the first.
4. Use Cmd+Shift+L to focus the right terminal.
5. Use Cmd+Shift+H to focus the left terminal.

One terminal runs Neovim, the other runs tests or a dev server. Both are always visible, no overlapping.

Move a misplaced window

If an application opens on the wrong workspace:

1. Focus it with Cmd+Shift+H/J/K/L.
2. Press Alt+Shift+2 to send it to workspace 2.
3. Switch to workspace 2 with Alt+2 to confirm.

Toggle a window to floating

Some windows (system preferences, small dialogs) work better floating:

1. Focus the window.
2. Press Alt+Shift+; to enter service mode.
3. Press f to toggle it to floating.
4. Drag and resize it as needed.
5. Press Alt+Shift+; then f again to tile it back.

Reorganize a cluttered workspace

If windows are nested oddly:

1. Press Alt+Shift+; to enter service mode.
2. Press r to flatten the workspace tree.
3. All windows return to a single-level tiled layout.

Full-screen focus

When you need to concentrate on one window:

1. Focus the window.
2. Press Cmd+Shift+F to toggle fullscreen.
3. The window takes the entire workspace. Other windows in the workspace are hidden.
4. Press Cmd+Shift+F again to return to tiled layout.

Move workspace to a different monitor

When you plug in or unplug a monitor, or want to rearrange:

1. Switch to the workspace you want to move (Alt+1 through Alt+4).

2. Press Alt+Shift+Tab to move it to the next monitor.
3. Press again to cycle through monitors.

Quick switch between two windows

Press Alt+O to toggle focus between the last two focused windows. This is the window management equivalent of `cd -` in the shell. You can rapidly flip between your editor and your browser, or between two terminals.

Advanced Usage

Understanding the Workspace-to-Monitor Assignment

```
[ workspace-to-monitor-force-assignment ]
1 = 'main '
2 = 'main '
3 = 'main '
4 = 'secondary '
```

The `force-assignment` key means these assignments are strict. Workspace 4 always lives on the secondary monitor. If the secondary monitor is disconnected, workspace 4 moves to the main monitor but returns when the secondary is reconnected.

With a two-monitor setup, the typical arrangement is:

- **Main monitor** (the one you face directly): workspaces 1-3 (browser, terminals, communications).
- **Secondary monitor** (off to the side): workspace 4 (Spotify, documentation, monitoring dashboards).

The Outer Top Gap by Monitor

The variable `top gap` deserves explanation:

```
outer.top = [
  { monitor."built-in" = 10 },
  { monitor.main = 38 },
  20,
]
```

- **Built-in display** (MacBook screen): 10px top gap. The built-in display has a notch, and 10px provides minimal spacing.
- **Main external monitor**: 38px top gap. This leaves room for a menu bar or status bar application that occupies the top of the screen.
- **Default** (any other monitor): 20px top gap. A reasonable middle ground.

The inner gaps (10px horizontal and vertical) create visual separation between tiled windows without wasting significant screen space. Setting `outer left`, `right`, and `bottom` to 0 maximizes usable area.

Normalization Behavior

The two normalization flags prevent the window tree from becoming unnecessarily complex:

Flatten containers (`enable-normalization-flatten-containers = true`): If you move windows around and end up with a container that has only one child window, Aerospace removes the container and promotes the window. This prevents ghost containers from accumulating.

Opposite orientation for nested containers (`enable-normalization-opposite-orientation-for-nested-containers = true`): When you create a nested split, Aerospace automatically makes the inner split perpendicular to the outer split. If the outer container splits horizontally, the inner container splits vertically. This produces grid-like layouts naturally without manual orientation management.

Join Operations in Service Mode

The `join-with` commands (`Alt+Shift+H/J/K/L` in service mode) merge the focused window into the container in the specified direction, creating sub-splits for complex layouts. For example, to stack B and C vertically while keeping A on the left: focus C, enter service mode, press `Alt+Shift+K` to join C upward into B's container.

Aerospace CLI

Aerospace includes a command-line interface for scripting:

```
aerospace list-windows —all           # List all windows
aerospace list-workspaces —all         # List all workspaces
aerospace workspace 2                  # Switch to workspace 2
aerospace move-node-to-workspace 3     # Move focused window
aerospace reload-config                # Reload configuration
```

When Not to Tile

Some applications work poorly with tiling – system dialogs, Finder windows, video calls. Toggle floating with service mode (`Alt+Shift+;` then `f`) for these cases.

Aerospace vs. Alternatives

Aerospace uses Apple's accessibility APIs and does not require disabling System Integrity Protection (unlike `yabai`). It supports arbitrary container nesting (unlike `Amethyst`) and manages full layouts automatically (unlike `Rectangle`, which is snap-to-grid). The trade-off is that it is macOS-only and requires accessibility permissions in System Preferences > Privacy & Security > Accessibility.

Modern Replacements: bat and eza

Drop-in replacements for cat and ls that add syntax highlighting, icons, git awareness, and tree views to everyday file inspection.

Your Setup

Both tools are installed via Homebrew and aliased in `~/.aliases.zsh` to transparently replace their traditional counterparts:

```
# ~/.aliases.zsh
alias cat="bat"
alias ls="eza"
alias l="eza -l --icons --git -a"
alias lt="eza --tree --level=2 --long --icons --git"
alias ltree="eza --tree --level=2 --icons --git"
```

Because of these aliases, every time you type `cat` or `ls` you are already using the modern versions. The original binaries remain available as `/bin/cat` and `/bin/ls` if you ever need them.

`bat` is also used as:

- The previewer in **fzf** fuzzy finder windows
- The `preview_command` in **ses**h session definitions (e.g., `bat --color=always ~/.local/share/chezmoi/dot_zshrc`)
- An alternative **MANPAGER** (though your setup uses `nvim +Man!` instead)

`eza` icons require a Nerd Font. Your terminal is configured with JetBrainsMono Nerd Font, which provides the necessary glyphs. Ensure a Nerd Font is set in your terminal or the icon characters will render as boxes.

Core Concepts

bat: A cat Clone with Wings

`bat` is a drop-in replacement for `cat` that adds automatic syntax highlighting for over 200 languages, line numbers, git diff markers in the gutter, and pager integration. When piped to another command, `bat` detects the non-interactive context and behaves like plain `cat`, so it is safe to use in pipelines.

eza: A Modern ls

eza replaces ls with a tool that understands git status, renders file-type icons via Nerd Font glyphs, and offers a built-in tree view. It uses colour by default and supports long-format output with human-readable sizes.

Essential Commands – bat

Command	Purpose
<code>bat file.py</code>	Display file with syntax highlighting and line numbers
<code>bat -l json data.txt</code>	Force a specific language for highlighting
<code>bat --diff a.py b.py</code>	Show a side-by-side diff between two files
<code>bat -p file.py</code>	Plain mode – no line numbers, no header, no grid
<code>bat -pp file.py</code>	Plain-plain mode – also disables paging
<code>bat --list-themes</code>	Show all available colour themes with previews
<code>bat --list-languages</code>	Show all supported language syntaxes
<code>bat -A file.txt</code>	Show non-printable characters (tabs, newlines, etc.)
<code>bat --style=numbers file.py</code>	Show only line numbers, no header or grid
<code>bat --style=header,grid file.py</code>	Pick specific style components
<code>bat --paging=never file.py</code>	Disable the pager even in interactive mode
<code>bat --color=always file less -R</code>	Force colour when piping to a pager
<code>bat header.h src.c</code>	Concatenate and display multiple files
<code>bat --wrap=never file.log</code>	Disable line wrapping for wide content

Essential Commands – eza

Command	Purpose
<code>ls</code>	Basic file listing (aliased to eza)
<code>l</code>	Long listing with icons, git status, and hidden files
<code>lt</code>	Tree view, 2 levels deep, long format with icons and git
<code>ltree</code>	Tree view, 2 levels deep, icons and git (no long format)
<code>eza -l --sort=modified</code>	Long listing sorted by modification time
<code>eza -l --sort=size</code>	Long listing sorted by file size
<code>eza -l --sort=ext</code>	Long listing sorted by file extension
<code>eza --tree --level=3</code>	Tree view with deeper nesting
<code>eza -l --no-permissions</code>	Long listing without the permissions column
<code>eza -l --no-user</code>	Long listing without the user/group columns
<code>eza --git-ignore</code>	Respect .gitignore and hide ignored files
<code>eza -l --header</code>	Long listing with a column header row

Command	Purpose
<code>eza -l</code>	One file per line, no extra detail
<code>eza -l --time-style=relative</code>	Show times as “2 hours ago” instead of dates
<code>eza --group-directories-first</code>	List directories before files

Practical Recipes

Preview files with fzf using bat

```
# fzf with bat as the preview window
fzf --preview 'bat --color=always --style=numbers --line-range=:500 {}'
```

Quickly check git changes in a directory

```
# Long listing shows git status per file (M, N, -, etc.)
l
```

The git column in eza output shows single-letter codes: M for modified, N for new/untracked, – for unchanged. This gives you an instant overview without running `git status`.

Use bat as a MANPAGER

```
# Export to use bat for man pages (alternative to your nvim approach)
export MANPAGER="sh -c 'col -bx | bat -l man -p' "
```

Your current setup uses `nvim +Man!` as the MANPAGER, which provides its own syntax highlighting. If you prefer bat, the line above is the standard recipe.

Coloured diff between two files

```
bat --diff config.old config.new
```

This highlights additions and deletions with colour, similar to `git diff` but for arbitrary files.

Explore a project structure before diving in

```
# See the shape of a project, two levels deep
eza --tree --level=2 --icons --git ~/projects/my-app
```

This is exactly what the `sesh nvim config` session uses as its preview command to give a quick structural overview.

Tail a log file with syntax highlighting

```
tail -f /var/log/system.log | bat --paging=never -l log
```

bat detects piped input and disables paging automatically, but `--paging=never` makes the intent explicit.

Display all supported bat themes with a sample

```
bat —list—themes | head -20
# Preview a specific theme:
bat —theme="Catppuccin_Mocha" file.py
```

Advanced Usage

bat Configuration File

bat reads defaults from `~/.config/bat/config`. You can set your preferred theme and style globally:

```
# ~/.config/bat/config
—theme="Catppuccin Mocha"
—style="numbers,changes,header"
—italic—text=always
```

With a config file in place, every bat (and therefore every cat) invocation inherits your preferences without extra flags.

Building Custom bat Syntaxes

If you work with an uncommon file format, bat supports adding custom `.sublime-syntax` files:

```
mkdir -p "$(bat—config-dir)/syntaxes"
# Place .sublime-syntax files there, then rebuild the cache:
bat cache —build
```

eza Colour and Icon Customization

eza reads the `EXA_COLORS` environment variable (legacy name) or `LS_COLORS` for file-type colouring. You can override specific extensions:

```
export EXA_COLORS="*.md=38;5;141:*.json=38;5;178"
```

Combining bat and eza in Scripts

Because both tools detect non-interactive output and fall back to plain text, they are safe in shell scripts. To force colour output (for example, when piping through `less`), pass `—color=always`:

```
eza -l —color=always | less -R
bat —color=always file.py | head -50
```

Using bat with git

bat integrates with git to show change markers in the gutter. Lines added since the last commit get a green +, modified lines get a yellow ~, and deleted regions get a red marker. This works automatically in any git repository.

```
# See only the changes column in bat output
bat —diff —style=changes file.py
```

HTTP Tools: curl and wget

The two foundational command-line HTTP clients – curl for precision API work and scripting, wget for recursive downloads and mirroring.

Your Setup

Both tools are installed explicitly via Homebrew in the master Brewfile:

```
brew "curl"  
brew "wget"
```

The Homebrew versions are preferred over the macOS system curl because they include more recent TLS libraries, HTTP/2 and HTTP/3 support, and additional protocols. After installation, Homebrew's curl takes precedence in your PATH.

These tools pair naturally with **jq** (covered in the next chapter) for working with JSON APIs, and with **aria2** for high-speed parallel downloads.

Core Concepts

curl – Transfer Data with URLs

curl is a tool for transferring data using URL syntax. It supports dozens of protocols (HTTP, HTTPS, FTP, SFTP, SCP, and more) but is most commonly used for HTTP requests. curl sends a single request and prints the response to stdout, making it ideal for API exploration, webhook testing, and scripting.

Key design principles: - Everything goes to stdout by default (use `-o` to save to a file) - Silent about progress unless you ask for verbosity (or ask it to be silent with `-s`) - Follows the Unix philosophy – combine with pipes, redirects, and other tools

wget – Non-Interactive Network Downloader

wget is designed for downloading files. Unlike curl, it defaults to saving to a file (not stdout), supports recursive downloading, and can resume interrupted transfers. wget is the better choice when you want to mirror a website or download a large set of files.

Essential Commands – curl

Command	Purpose
<code>curl URL</code>	GET request, print response to stdout
<code>curl -s URL</code>	Silent mode – suppress progress bar
<code>curl -o file URL</code>	Save response to a named file
<code>curl -O URL</code>	Save with the remote filename
<code>curl -I URL</code>	Fetch HTTP headers only (HEAD request)
<code>curl -v URL</code>	Verbose output showing full request/response
<code>curl -L URL</code>	Follow HTTP redirects
<code>curl -X POST URL</code>	Send a POST request
<code>curl -X PUT URL</code>	Send a PUT request
<code>curl -X DELETE URL</code>	Send a DELETE request
<code>curl -d '{"key":"val"}' URL</code>	Send data in request body
<code>curl -d @file.json URL</code>	Send file contents as request body
<code>curl -H "Content-Type: application/json" URL</code>	Set a request header
<code>curl -H "Authorization: Bearer TOKEN" URL</code>	Authenticate with a bearer token
<code>curl -u user:pass URL</code>	HTTP basic authentication
<code>curl -b cookies.txt URL</code>	Send cookies from a file
<code>curl -c cookies.txt URL</code>	Save received cookies to a file
<code>curl -C - -O URL</code>	Resume an interrupted download
<code>curl --connect-timeout 5 --max-time 10 URL</code>	Set connection and total timeouts

Essential Commands – wget

Command	Purpose
<code>wget URL</code>	Download a file (saves with remote name)
<code>wget -O name URL</code>	Download and save with a custom filename
<code>wget -c URL</code>	Resume an interrupted download
<code>wget -b URL</code>	Download in the background
<code>wget -r URL</code>	Recursive download (follow links)
<code>wget -r -l 2 URL</code>	Recursive download, limit depth to 2 levels
<code>wget -m URL</code>	Mirror a website (recursive + timestamps + infinite depth)
<code>wget -i urls.txt</code>	Download all URLs listed in a file
<code>wget -q URL</code>	Quiet mode – no output
<code>wget --limit-rate=1m URL</code>	Limit download speed to 1 MB/s
<code>wget --no-check-certificate URL</code>	Skip SSL certificate verification
<code>wget -P /path/to/dir URL</code>	Save downloaded file to a specific directory

Practical Recipes

GET a JSON API and pretty-print with jq

```
curl -s https://api.github.com/users/octocat | jq '.'
```

The `-s` flag silences the progress bar so only clean JSON reaches `jq`.

POST JSON to an API

```
curl -s -X POST \
  -H "Content-Type: application/json" \
  -d '{"name": "new-repo", "private": true}' \
  -H "Authorization: Bearer $GITHUB_TOKEN" \
  https://api.github.com/user/repos | jq '.'
```

Upload a file with a multipart form

```
curl -X POST -F "file=@document.pdf" -F "name=upload-test" \
  https://example.com/upload
```

The `-F` flag creates a multipart/form-data request, which is how HTML file upload forms work.

Check if a URL is reachable

```
curl -s -o /dev/null -w "%{http_code}" https://example.com
```

This prints only the HTTP status code (e.g., 200, 404, 301). The `-o /dev/null` discards the body, and `-w` formats the output.

Measure response time

```
curl -s -o /dev/null -w "DNS:%{time_namelookup}s\nConnect:%{time_connect}s\nTLS:"
```

The `-w` flag supports many timing variables that break down where latency is spent.

Download with a progress bar

```
curl -# -O https://example.com/large-file.tar.gz
```

The `-#` flag replaces the default statistics with a simple progress bar.

Mirror a website for offline reading

```
wget -m -k -p https://example.com
```

The flags mean: `-m` mirror (recursive + timestamps), `-k` convert links for local viewing, `-p` download page requisites (CSS, images).

Download all files of a type from a page

```
wget -r -l 1 -A "*.pdf" https://example.com/papers/
```

The `-A` flag accepts only files matching the pattern, and `-l 1` limits recursion depth.

Retry a flaky download

```
wget -c --tries=10 --waitretry=30 https://example.com/large-file.iso
```

`wget` will retry up to 10 times, waiting 30 seconds between attempts, and resuming from where it left off each time.

Advanced Usage

curl with a Configuration File

For repeated API work, store common options in a file:

```
# ~/.curlrc
—silent
—show-error
—location
—connect-timeout 10
—max-time 30
```

curl reads ~/.curlrc automatically. Override any setting with explicit flags.

Sending Multiple curl Requests in Parallel

curl 7.66+ supports `--parallel` for sending multiple requests simultaneously:

```
curl —parallel —parallel-max 5 \
  -O https://example.com/file1.tar.gz \
  -O https://example.com/file2.tar.gz \
  -O https://example.com/file3.tar.gz
```

Inspecting and Debugging TLS

```
# Show full TLS handshake details
curl -v —trace-ascii /dev/stderr https://example.com 2>&1 | head -40

# Check certificate expiry
curl -s -v https://example.com 2>&1 | grep "expire_date"
```

Scripting Patterns with curl

A common pattern for API scripts: check the status code and handle errors.

```
response=$(curl -s -w "\n%{http_code}" https://api.example.com/data)
body=$(echo "$response" | head -n -1)
status=$(echo "$response" | tail -n 1)

if [ "$status" -ne 200 ]; then
  echo "Error: HTTP $status"
  echo "$body" | jq '.message'
  exit 1
fi

echo "$body" | jq '.'
```

When to Use Which Tool

Scenario	Preferred Tool
API requests (GET, POST, PUT, DELETE)	curl
Downloading a single file	Either (curl -O or wget)
Resuming a large download	wget -c
Recursive site download / mirroring	wget -m
Custom headers and authentication	curl
Scripting HTTP interactions	curl
Batch downloading from a URL list	wget -i
High-speed parallel downloads	aria2 (see pandoc/ffmpeg chapter)

JSON Processing: jq

A lightweight, flexible command-line JSON processor – the sed and awk of structured data.

Your Setup

jq is installed via Homebrew in the base Brewfile:

```
brew "jq"
```

It is most commonly used in combination with **curl** for API exploration and with shell scripts that consume JSON output from tools like gh, docker, and cloud CLIs. Any command that produces JSON can be piped directly to jq.

Core Concepts

jq operates on a stream of JSON values. You write a **filter expression** that transforms input JSON into output JSON. Filters can be chained with the pipe operator `|`, just like Unix commands. The simplest filter is `.` (the identity), which pretty-prints the input with colour.

Key mental model:

- `.` means “the current value”
- `.key` descends into an object
- `.[ ]` iterates over array elements or object values
- `|` passes output from one filter to the next
- Everything in jq is an expression that produces zero or more outputs

Type System

jq works with the same types as JSON: objects, arrays, strings, numbers, booleans, and null. It adds no new types but provides rich operators for transforming between them.

Essential Commands

Command	Purpose
<code>jq '.'</code>	Pretty-print JSON with syntax colouring
<code>jq '.key'</code>	Extract a top-level field

Command	Purpose
<code>jq '.key.nested'</code>	Extract a nested field
<code>jq '.array[0]'</code>	First element of an array
<code>jq '.array[-1]'</code>	Last element of an array
<code>jq '.array[2:5]'</code>	Array slice (indices 2, 3, 4)
<code>jq '.array[:3]'</code>	First three elements
<code>jq '.array[-2:]'</code>	Last two elements
<code>jq '[]'</code>	Iterate over all elements
<code>jq '.key1, .key2'</code>	Output multiple fields
<code>jq -r '.name'</code>	Raw output (no quotes around strings)
<code>jq -c '.'</code>	Compact output (one line per object)
<code>jq -e '.key'</code>	Exit with error if result is null or false
<code>jq -s '.'</code>	Slurp: read entire input as one array
<code>jq -n '{}'</code>	Null input: generate JSON from nothing
<code>jq --arg name "val" '.'</code>	Pass a string variable into the filter
<code>jq --argjson num 42 '.'</code>	Pass a JSON value variable into the filter

Filter Reference

Selection and Filtering

Filter	Purpose
<code>select (.key == "value")</code>	Keep only objects where condition is true
<code>select (.age > 30)</code>	Numeric comparison
<code>select (.name \ test("pattern"))</code>	Regex match
<code>select (.tags \ contains(["important"]))</code>	Array contains check
<code>select (has("key"))</code>	Object has a specific key
<code>select (.key != null)</code>	Filter out null values

Transformation

Filter	Purpose
<code>map(.field)</code>	Apply expression to each array element
<code>map(. + 1)</code>	Increment every element
<code>map(select(.active))</code>	Filter array, keep active items
<code>sort_by(.field)</code>	Sort array of objects by field
<code>group_by(.field)</code>	Group array elements by field value
<code>unique_by(.field)</code>	Deduplicate by field
<code>reverse</code>	Reverse an array
<code>flatten</code>	Flatten nested arrays one level
<code>limit(n; .[])</code>	Take first n elements from iterator
<code>keys</code>	Get object keys as an array
<code>values</code>	Get object values as an array
<code>length</code>	Count elements in array or keys in object

Filter	Purpose
<code>type</code>	Return the JSON type as a string
<code>to_entries</code>	Convert object to array of {key, value} pairs
<code>from_entries</code>	Convert {key, value} array back to object

Construction and String Handling

Filter	Purpose
<code>{newkey: .oldkey}</code>	Build a new object
<code>[. items [] . name]</code>	Build a new array
<code>"\(.key) is \(.value)"</code>	String interpolation
<code>.key // "default"</code>	Alternative operator (if null, use default)
<code>split(",")</code>	Split string into array
<code>join(",")</code>	Join array into string
<code>ascii_lowercase</code>	Lowercase a string
<code>ltrimstr(" prefix ")</code>	Remove a prefix
<code>rtrimstr(" suffix ")</code>	Remove a suffix
<code>gsub("old"; "new")</code>	Global string substitution

Conditionals and Reduction

Filter	Purpose
<code>if .x then .a else .b end</code>	Conditional expression
<code>reduce .[] as \$x (0; . + \$x)</code>	Fold/reduce over array
<code>.[] \ if .type == "a" then .val else empty end</code>	Conditional with empty to skip

Practical Recipes

Pretty-print a curl response

```
curl -s https://api.github.com/repos/neovim/neovim | jq '.'
```

Extract specific fields from API data

```
curl -s https://api.github.com/repos/neovim/neovim | \
jq '{name: .name, stars: .stargazers_count, language: .language}'
```

Filter an array of objects

```
echo ' [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}] ' | \
jq '[.[] | select(.age > 27)]'
```

Result: [{"name": "Alice", "age": 30}]

Transform a list of objects into a summary

```
curl -s https://api.github.com/users/octocat/repos | \
jq '[.[] | {name: .name, stars: .stargazers_count}] | sort_by(.stars) | reverse'
```

Create CSV output from JSON

```
curl -s https://api.github.com/users/octocat/repos | \
jq -r '[.[] | [.name, .stargazers_count, .language]] | @csv'
```

The @csv format filter handles quoting and escaping for proper CSV.

Count items by category

```
echo '[{"type": "bug"}, {"type": "feature"}, {"type": "bug"}]' | \
jq 'group_by(.type) | map({type: .[0].type, count: .length})'
```

Merge two JSON files

```
jq -s '[0] * [1]' base.json overrides.json
```

The * operator recursively merges objects. The -s (slurp) flag reads both files into a single array.

Update a nested value

```
jq '.config.timeout = 30' settings.json > settings-new.json
```

jq does not edit files in place. Write to a new file and rename, or use sponge from moreutils.

Process NDJSON (newline-delimited JSON)

```
# Each line is a separate JSON object — jq handles this natively
cat events.ndjson | jq 'select(.level == "error")'
```

Use shell variables inside jq

```
name="Alice"
jq --arg n "$name" '[.[] | select(.name == $n)]' data.json
```

Always use --arg to inject shell variables. Never interpolate shell variables directly into jq expressions – it breaks on special characters and is a security risk.

Advanced Usage**Building Complex Pipelines**

jq filters compose naturally. Think of each | as passing the current value(s) to the next stage:

```

curl -s https://api.github.com/orgs/github/repos?per_page=100 | \
jq '[.]
      |select(.language!=null)
      |[{name,.language,.stars:.stargazers_count}]
      |group_by(.language)
      |map({language:[0].language,count:length,total_stars:map(.stars)|add})
      |sort_by(.total_stars)
      |reverse'
```

This pipeline: filters out repos with no language, extracts fields, groups by language, counts repos and sums stars per language, then sorts by total stars descending.

Defining Functions

```

jq 'def double: . * 2; [.[] | double]' <<< '[1,2,3]'
# Output: [2,4,6]
```

The env Object

jq can access environment variables directly:

```

export API_KEY="secret"
jq -n 'env.API_KEY'
```

Error Handling with try-catch

```

echo '{"a":1}' | jq 'try .b.c catch "missing"'
# Output: "missing"
```

Working with gh CLI Output

The GitHub CLI outputs JSON with `--json`, which pairs perfectly with jq:

```

gh pr list --json number,title,author --jq '[.] | "\(.number): \(.title) by \(.author)'"
```

Note that gh has a built-in `--jq` flag, so you can avoid the pipe entirely.

Text Processing: grep, sed, and awk

The classic Unix text processing trinity – pattern matching, stream editing, and field-oriented data transformation.

Your Setup

GNU grep is installed via Homebrew to replace the BSD version shipped with macOS:

```
brew "grep"
```

The Homebrew GNU grep provides `-P` (Perl-compatible regex) and other features missing from the macOS default. After installation, it is available as `ggrep` or, if Homebrew's `gnubin` directory is in your `PATH`, as `grep`.

`sed` and `awk` on macOS are the BSD variants. Their behaviour differs from the GNU versions in small but important ways, particularly `sed`'s `-i` flag (macOS requires `sed -i ''`, while GNU uses `sed -i`).

Note: For most interactive code searching, **ripgrep** (`rg`) is the preferred tool in this environment. It is faster, respects `.gitignore`, and has a more intuitive interface. However, `grep`, `sed`, and `awk` remain essential for pipelines, scripts, and text transformations where `ripgrep` does not apply.

Core Concepts

grep – Global Regular Expression Print

`grep` searches files or `stdin` for lines matching a pattern and prints them. It is the most fundamental text search tool in Unix. Think of `grep` as a filter: input goes in, only matching lines come out.

sed – Stream Editor

`sed` processes text line by line, applying transformation rules. It does not load the entire file into memory, making it efficient for large files. The most common use is find-and-replace, but `sed` can also insert, delete, and rearrange lines.

awk – Pattern-Action Language

`awk` is a small programming language designed for processing columnar text data. It automatically splits each line into fields (`$1`, `$2`, ...) and lets you write pattern-action rules. `awk` bridges the gap between simple one-liners and full scripting languages.

Essential Commands – grep

Command	Purpose
grep pattern file	Search for pattern in a file
grep -r pattern dir	Recursive search through a directory
grep -i pattern file	Case-insensitive search
grep -n pattern file	Show line numbers with matches
grep -l pattern dir/*	Show only filenames containing matches
grep -c pattern file	Count matching lines
grep -v pattern file	Invert match (show non-matching lines)
grep -w pattern file	Match whole words only
grep -E 'pat1\ pat2' file	Extended regex (egrep) – alternation
grep -P '\d{3}-\d{4}' file	Perl regex (GNU grep only)
grep -A 3 pattern file	Show 3 lines after each match
grep -B 2 pattern file	Show 2 lines before each match
grep -C 2 pattern file	Show 2 lines of context (before and after)
grep -o pattern file	Print only the matched text, not the full line
grep -q pattern file	Quiet mode – exit code only, no output
grep --include='*.py' -r pattern dir	Recursive, limited to specific file types

Essential Commands – sed

Command	Purpose
sed 's/old/new/' file	Replace first occurrence per line
sed 's/old/new/g' file	Replace all occurrences per line
sed -i '' 's/old/new/g' file	In-place edit (macOS BSD sed)
sed -n '5p' file	Print only line 5
sed -n '2,5p' file	Print lines 2 through 5
sed -n '/pattern/p' file	Print lines matching pattern
sed '/pattern/d' file	Delete lines matching pattern
sed '5d' file	Delete line 5
sed '2,5d' file	Delete lines 2 through 5
sed '3i\new line' file	Insert text before line 3
sed '3a\new line' file	Append text after line 3
sed -e 's/a/b/' -e 's/c/d/' file	Multiple substitutions
sed 's/pattern/&-suffix/' file	Append to matched text (& = match)
sed 's/(group)/\1-extra/' file	Backreference to captured group
sed '/start/,end/d' file	Delete a range between two patterns
sed '1,3s/old/new/' file	Replace only on lines 1 through 3

Essential Commands – awk

Command	Purpose
<code>awk '{print \$1}' file</code>	Print the first field of each line
<code>awk '{print \$1, \$3}' file</code>	Print fields 1 and 3
<code>awk -F',' '{print \$2}' file</code>	Use comma as field separator
<code>awk -F: '{print \$1}' /etc/passwd</code>	Use colon as separator
<code>awk '/pattern/ {print}' file</code>	Print lines matching pattern
<code>awk '\$3 > 100 {print \$1, \$3}' file</code>	Conditional field printing
<code>awk '{print NR, \$0}' file</code>	Print line numbers
<code>awk 'NR==5' file</code>	Print only line 5
<code>awk 'NR>=5 && NR<=10' file</code>	Print lines 5 through 10
<code>awk '{print NF}' file</code>	Print number of fields per line
<code>awk '{print \$NF}' file</code>	Print last field of each line
<code>awk '{printf "%-20s %s\n", \$1, \$2}' file</code>	Formatted output
<code>awk 'BEGIN{sum=0} {sum+=\$1} END{print sum}' file</code>	Sum a column
<code>awk 'BEGIN{OFS=","} {print \$1,\$2}' file</code>	Set output field separator
<code>awk '!seen[\$0]++' file</code>	Remove duplicate lines (preserving order)
<code>awk '{sum+=\$1; count++} END{print sum/count}' file</code>	Calculate average

Practical Recipes

Find TODOs across a codebase

```
grep -rn 'TODO\|FIXME\|HACK' --include='*.py' src/
```

For interactive use, prefer ripgrep: `rg 'TODO|FIXME|HACK' -t py src/`

Replace a string across multiple files

Preview what will change (use grep first)

```
grep -rl 'old_function' src/
```

Apply the change

```
grep -rl 'old_function' src/ | xargs sed -i '' 's/old_function/new_function/g'
```

Extract email addresses from a file

```
grep -oE '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' file.txt
```

Remove blank lines from a file

```
sed '/^$/d' file.txt
```

Print a specific field from a CSV

```
awk -F',' '{print $2}' data.csv
```

For CSV files with quoted fields containing commas, awk's simple field splitting is insufficient. Use a dedicated CSV tool or `csvkit` instead.

Sum a column of numbers

```
awk ' {sum+= $1} END { print sum } ' numbers.txt
```

Extract text between two markers

```
sed -n '/BEGIN_SECTION/,/END_SECTION/p' config.txt
```

Add a header line to output

```
awk 'BEGIN { print "Name\tScore" } { print $1"\t"$2 } ' results.txt
```

Count word frequency

```
tr ' ' '\n' < file.txt | sort | uniq -c | sort -rn | head -20
```

This classic pipeline splits words, sorts, counts, and shows the top 20.

Replace only on lines matching a condition

```
# Only replace "foo" with "bar" on lines containing "target"
sed '/target/s/foo/bar/g' file.txt
```

Transpose rows and columns

```
awk ' { for (i=1; i<=NF; i++) a[NR][i]=$i }
      END { for (j=1; j<=NF; j++) { for (i=1; i<=NR; i++) printf "%s ", a[i][j]; print " } } '
```

Advanced Usage**Combining grep, sed, and awk in Pipelines**

The real power of these tools emerges when they are combined. Each tool handles what it does best:

```
# Find log errors, extract the timestamp and message, format as a table
grep 'ERROR' app.log | \
  sed 's/\[//;s/\]//' | \
  awk '{ printf "%-20s%s\n", $1" "$2, substr($0, index($0,$4)) } '
```

awk as a Mini Programming Language

awk supports arrays, functions, and control flow:

```
# Count HTTP status codes from an access log
awk ' { count[$9]++ } END { for (code in count) print code, count[code] } ' access.log |
```

sed with Regular Expression Groups

```
# Swap first and last name
echo "Doe, John" | sed 's/\(.*\) , \(.*\) /\2 \1/'
# Output: John Doe
```

Multi-line Processing with sed

```
# Join every pair of lines
sed 'N;s/\n/_/' file.txt
```

The -i Flag Portability Problem

The in-place edit flag differs between macOS (BSD) and Linux (GNU):

```
# macOS (BSD sed) — requires empty string argument
sed -i '' 's/old/new/g' file.txt
```

```
# Linux (GNU sed) — no argument needed
sed -i 's/old/new/g' file.txt
```

```
# Portable workaround
sed 's/old/new/g' file.txt > file.tmp && mv file.tmp file.txt
```

When to Reach for Each Tool

Task	Best Tool
Find lines matching a pattern	grep
Find and replace text	sed
Extract or rearrange columns	awk
Complex conditionals on fields	awk
Interactive code search	ripgrep (rg)
Structured data (JSON, CSV)	jq, csvkit
Delete or insert specific lines	sed
Aggregate or summarize data	awk

Media and Documents: pandoc, ffmpeg, and Friends

Command-line tools for converting documents, processing video and audio, downloading media, and manipulating images – all installed and ready in your Brewfile.

Your Setup

All of the following tools are installed via Homebrew in the master Brewfile:

```
brew "pandoc"
brew "ffmpeg"
brew "ffmpegthumbnailer"
brew "yt-dlp"
brew "aria2"
brew "imagemagick"
brew "ghostscript"      # PDF rendering backend for imagemagick
brew "poppler"          # PDF utilities, used by Yazi for previews
brew "mermaid-cli"      # Diagram generation from Markdown-style syntax
```

Notable integrations in your environment:

- **ffmpegthumbnailer** is used by **Yazi** to generate video preview thumbnails in the file manager
- **poppler** provides pdftotext and pdftoppm, used by Yazi for PDF previews
- **ghostscript** is a dependency for imagemagick's PDF operations
- **pandoc** is used to build this very book from Markdown sources
- **mermaid-cli** generates diagrams from .mmd files, useful in documentation workflows

Core Concepts

pandoc – Universal Document Converter

pandoc reads markup in one format and writes it in another. It understands Markdown, HTML, LaTeX, DOCX, EPUB, reStructuredText, Org-mode, and dozens more. The key insight is that pandoc uses an internal abstract syntax tree, so any input format can be converted to any output format. For PDF output, pandoc uses LaTeX as an intermediate step (requires a TeX distribution like MacTeX or TinyTeX).

ffmpeg – The Swiss Army Knife of Media

ffmpeg is a complete framework for decoding, encoding, transcoding, muxing, demuxing, streaming, filtering, and playing almost any media format. Its command-line syntax follows a pattern: input options, `-i` input, output options, output file.

yt-dlp – Media Downloader

yt-dlp is a feature-rich command-line downloader forked from youtube-dl. It supports thousands of sites and offers fine-grained control over format selection, subtitles, and metadata.

aria2 – High-Speed Downloader

aria2 is a multi-protocol download utility that can use multiple connections per download and supports HTTP, FTP, BitTorrent, and Metalink.

imagemagick – Image Processing

ImageMagick provides a suite of commands (`convert`, `identify`, `mogrify`, `composite`) for image manipulation from the command line.

Essential Commands – pandoc

Command	Purpose
<code>pandoc file.md -o file.pdf</code>	Markdown to PDF (requires LaTeX)
<code>pandoc file.md -o file.html</code>	Markdown to standalone HTML
<code>pandoc file.md -o file.docx</code>	Markdown to Word document
<code>pandoc file.docx -o file.md</code>	Word document to Markdown
<code>pandoc file.md -o file.epub</code>	Markdown to EPUB ebook
<code>pandoc --toc file.md -o file.pdf</code>	Include auto-generated table of contents
<code>pandoc -V geometry:margin=1in file.md -o file.pdf</code>	Set PDF margins
<code>pandoc -s file.md -o file.html</code>	Standalone HTML (full document with head)
<code>pandoc --template=tmpl.tex file.md -o file.pdf</code>	Use a custom LaTeX template
<code>pandoc --highlight-style=tango file.md -o file.pdf</code>	Set code highlight theme
<code>pandoc --list-highlight-styles</code>	Show available highlight styles
<code>pandoc -f gfm -t html file.md</code>	Explicit input/output format (GitHub Markdown)
<code>pandoc --pdf-engine=xelatex file.md -o file.pdf</code>	Use XeLaTeX for Unicode/font support
<code>pandoc --filter pandoc-crossref file.md -o file.pdf</code>	Use a filter for cross-references

Essential Commands – ffmpeg

Command	Purpose
<code>ffmpeg -i in.mp4 out.gif</code>	Convert video to GIF
<code>ffmpeg -i in.mp4 -vn out.mp3</code>	Extract audio track (drop video)
<code>ffmpeg -i in.mp4 -an out.mp4</code>	Remove audio track (keep video)

Command	Purpose
<code>ffmpeg -i in.mp4 -ss 00:01:00 -t 30 out.mp4</code>	Trim clip: start at 1:00, 30 seconds
<code>ffmpeg -i in.mp4 out.webm</code>	Convert format (container/codec auto-detected)
<code>ffmpeg -i in.mp4 -vf scale=1280:720 out.mp4</code>	Scale video to 720p
<code>ffmpeg -i in.mp4 -vf scale=-1:720 out.mp4</code>	Scale height to 720, auto width
<code>ffmpeg -i in.mp4 -r 30 out.mp4</code>	Change frame rate to 30 fps
<code>ffmpeg -i in.mp4 -crf 23 out.mp4</code>	Control quality (lower = better, 18-28 typical)
<code>ffmpeg -i in.mp4 -vf "fps=10,scale=320:-1" out.gif</code>	Optimized GIF (lower fps, small)
<code>ffmpeg -i in.mp4 -c copy out.mkv</code>	Remux without re-encoding (fast)
<code>ffmpeg -i in.mp4 -ss 00:00:30 -frames:v 1 thumb.png</code>	Extract a single frame as image
<code>ffmpeg -i in.mp4 -vf drawtext="text='Hello':font=libttxl0:y=10" out.mp4</code>	Overlay text
<code>ffmpeg -f concat -i list.txt -c copy out.mp4</code>	Concatenate multiple video files

Essential Commands – yt-dlp

Command	Purpose
<code>yt-dlp URL</code>	Download video in best available quality
<code>yt-dlp -f best URL</code>	Explicitly select best single-file format
<code>yt-dlp -x --audio-format mp3 URL</code>	Extract audio only, convert to MP3
<code>yt-dlp -x --audio-format flac URL</code>	Extract audio as FLAC
<code>yt-dlp --list-formats URL</code>	List all available formats
<code>yt-dlp -f 'bestvideo[height<=720]+bestaudio' URL</code>	Download 720p max with best audio
<code>yt-dlp --write-sub --sub-lang en URL</code>	Download with English subtitles
<code>yt-dlp --write-auto-sub URL</code>	Download with auto-generated subtitles
<code>yt-dlp -o '%(title)s.%(ext)s' URL</code>	Custom output filename template
<code>yt-dlp --playlist-items 1-5 URL</code>	Download first 5 items from a playlist
<code>yt-dlp --download-archive done.txt URL</code>	Skip already-downloaded videos
<code>yt-dlp --cookies-from-browser firefox URL</code>	Use browser cookies for auth

Essential Commands – aria2

Command	Purpose
<code>aria2c URL</code>	Download a file
<code>aria2c -x 16 URL</code>	Download with 16 connections (much faster)
<code>aria2c -x 16 -s 16 URL</code>	16 connections, 16 segments
<code>aria2c -i urls.txt</code>	Download all URLs from a file
<code>aria2c file .torrent</code>	Download a torrent
<code>aria2c file .metalink</code>	Download from a metalink file
<code>aria2c -d /path/to/dir URL</code>	Save to a specific directory
<code>aria2c -o filename URL</code>	Save with a custom filename
<code>aria2c --max-overall-download-limit=1M URL</code>	Limit download speed

Essential Commands – imagemagick

Command	Purpose
<code>convert in.png out.jpg</code>	Convert between image formats
<code>convert in.jpg -resize 50% out.jpg</code>	Resize by percentage
<code>convert in.jpg -resize 800x600 out.jpg</code>	Resize to fit within dimensions
<code>convert in.jpg -resize 800x600! out.jpg</code>	Force exact dimensions (may distort)
<code>identify image.jpg</code>	Show image dimensions, format, size
<code>identify -verbose image.jpg</code>	Detailed image metadata
<code>convert in.jpg -quality 80 out.jpg</code>	Set JPEG quality (0-100)
<code>convert in.png -trim out.png</code>	Auto-crop whitespace borders
<code>convert -append top.png bottom.png out.png</code>	Stack images vertically
<code>convert +append left.png right.png out.png</code>	Place images side by side
<code>mogrify -resize 50% *.jpg</code>	Resize all JPEGs in place (batch)
<code>convert in.pdf[0] out.png</code>	Convert first page of PDF to PNG

Practical Recipes

Build a PDF book from multiple Markdown files

```
pandoc --toc \
  -V geometry:margin=1in \
  -V fontsize=11pt \
  --highlight-style=tango \
  chapter1.md chapter2.md chapter3.md \
  -o book.pdf
```

This is the pattern used to build the book you are reading. pandoc concatenates the input files in order, generates a table of contents, and produces a single PDF.

Create a high-quality GIF from a screen recording

Step 1: Generate a colour palette for better quality

```
ffmpeg -i recording.mp4 -vf "fps=10,scale=640:-1:flags=lanczos,palettegen" palette.png
```

Step 2: Use the palette to create the GIF

```
ffmpeg -i recording.mp4 -i palette.png \
  -filter_complex "fps=10,scale=640:-1:flags=lanczos[x];[x][1:v]paletteuse" \
  output.gif
```

Download a playlist as MP3 files

```
yt-dlp -x --audio-format mp3 -o '%(playlist_index)s-%(title)s.%(ext)s' \
  'https://youtube.com/playlist?list=PLAYLIST_ID'
```

Batch convert images for web

```
mkdir -p web
for img in *.jpg; do
    convert "$img" -resize 1200x1200 -quality 85 "web/$img"
done
```

Create a contact sheet from video

```
ffmpeg -i video.mp4 -vf "select='not(mod(n,300))',scale=320:-1,tile=4x4" \
    -frames:v 1 contact_sheet.png
```

This extracts every 300th frame, scales them, and tiles them in a 4x4 grid.

Fast download with aria2 (16 connections)

```
aria2c -x 16 -s 16 https://example.com/large-file.iso
```

This is significantly faster than curl or wget for large files from servers that allow multiple connections.

Advanced Usage**pandoc Metadata and YAML Front Matter**

pandoc reads YAML metadata blocks at the top of Markdown files:

```
—
title: "My Document"
author: "zero.dev001"
date: 2025-01-15
—
```

These variables populate the title page and headers in PDF output.

ffmpeg Filter Chains

ffmpeg's `-filter_complex` allows chaining multiple video/audio filters:

```
# Overlay a watermark in the bottom-right corner
ffmpeg -i video.mp4 -i watermark.png \
    -filter_complex "overlay=W-w-10:H-h-10" \
    output.mp4
```

Automating with a Makefile

For a document project, a Makefile ties everything together:

```
book.pdf: $(wildcard chapters/*.md)
    pandoc —toc -V geometry:margin=1in $^ -o $@
```

```
diagrams/%.png: diagrams/%.mmd
  mmdc -i $< -o $@
```

The mmdc command is mermaid-cli, installed in your Brewfile, which renders Mermaid diagram syntax to PNG or SVG.

ImageMagick for Screenshots

```
# Add a shadow and border to a screenshot (great for documentation)
convert screenshot.png \
  \( +clone -background black -shadow 60x10+0+10\) \
  +swap -background none -layers merge +repage \
  screenshot-styled.png
```

System Monitoring: btop, fastfetch, and Friends

Keep an eye on your machine – CPU, memory, disk, network, and processes – with modern monitoring tools and handy shell functions.

Your Setup

System monitoring tools installed via Homebrew in the master Brewfile:

```
brew "btop"
brew "fastfetch"
cask "stats"
```

In addition, several shell aliases and functions in `~/.aliases.zsh` provide quick access to system information:

```
# File and directory sizes
fs() { du -sh -- "$@" | sort -h; }
```

```
# Compare original vs gzipped file size
```

```
gz() {
  local orig=$(wc -c < "$1")
  local gzipped=$(gzip -c "$1" | wc -c)
  echo "orig:$(echo "$orig" | numfmt --to=iec) / gzip:$(echo "$gzipped" | numfmt)
}
```

```
# Quick HTTP server
```

```
server() { open "http://localhost:${1:-8000}" && python3 -m http.server "${1:-8000}"
```

```
# Networking
```

```
alias ip="dig+short myip.opendns.com @resolver1.opendns.com"
```

```
alias localip="ipconfig_getifaddr_en0"
```

```
alias flush="dscacheutil -flushcache && killall -HUP mDNSResponder"
```

Core Concepts

btop – Interactive Resource Monitor

btop is a modern replacement for htop and top. It provides a rich terminal UI showing CPU usage per core, memory and swap usage, disk I/O, network throughput, and a filterable process list. It supports mouse interaction, vim-style keyboard navigation, and process management (send signals, renice).

fastfetch – System Information Display

fastfetch is a fast system information tool (a rewrite of neofetch in C). It displays a quick summary of your system: OS, kernel version, uptime, installed packages, shell, terminal, CPU, GPU, and memory. It is primarily used for quick reference and for showing off your terminal setup.

Stats – Menu Bar Monitor

Stats is a macOS application (installed as a cask) that places live system metrics in the menu bar. It shows CPU load, memory pressure, disk usage, network activity, and battery status at a glance without opening a terminal.

Essential Commands – btop

Command / Key	Purpose
btop	Launch the interactive monitor
f	Filter processes by name
/	Search for a process
t	Toggle tree view for processes
k	Send a signal to the selected process
s	Select signal type to send
n	Nice/renice the selected process
e	Toggle per-core CPU graph
m	Toggle memory graph style
d	Toggle disk I/O display
Up/Down	Navigate the process list
Left/Right	Change sorting column
Enter	Show detailed info for selected process
Tab	Cycle through display sections
Esc	Close menus and dialogs
q	Quit btop
h	Show help

btop Configuration

btop stores its configuration in ~/.config/btop/btop.conf. Key settings include:

Setting	Purpose
color_theme	Theme name (e.g., catppuccin_mocha)
theme_background	Whether to use the theme's background colour
vim_keys	Enable h/j/k/l navigation
update_ms	Refresh interval in milliseconds
proc_sorting	Default sort column (cpu, mem, pid, etc.)
proc_tree	Start in tree view

Essential Commands – fastfetch

Command	Purpose
fastfetch	Display full system info summary
fastfetch --logo none	Display info without the logo
fastfetch --structure OS:Kernel:Uptime:Shell	Show only selected modules
fastfetch --list-modules	List all available info modules
fastfetch --gen-config	Generate a default config file
fastfetch -c paleofetch	Use a preset configuration

fastfetch output includes:

- **OS:** macOS version and build
- **Kernel:** Darwin kernel version
- **Uptime:** How long since last restart
- **Packages:** Count from Homebrew, npm, pip, etc.
- **Shell:** zsh version
- **Terminal:** iTerm2
- **CPU:** Model and speed
- **GPU:** Integrated and discrete graphics
- **Memory:** Used / Total

Shell Functions and Aliases

fs – File and Directory Sizes

```
fs() { du -sh -- "$@" | sort -h; }
```

Usage:

Command	Purpose
fs *	Show sizes of everything in the current directory, sorted
fs ~/Projects/*	Compare project directory sizes
fs .git node_modules src	Check specific directories
fs *.log	Check sizes of all log files

The `-h` flag on both `du` and `sort` ensures human-readable sizes (K, M, G) are sorted numerically, so 2M comes before 1G.

gz – Gzip Compression Ratio

```
gz() {
  local orig=$(wc -c < "$1")
  local gzipped=$(gzip -c "$1" | wc -c)
  echo "orig: $(echo "$orig" | numfmt --to=iec) / gzip: $(echo "$gzipped" | numfmt)
```

Usage:

```
gz bundle.js
# orig: 450K / gzip: 112K / ratio: 24.89%
```

This is invaluable for web development – quickly check how well your JavaScript, CSS, or JSON compresses before deploying.

server – Quick HTTP Server

```
server() { open "http://localhost:${1:-8000}" && python3 -m http.server "${1:-8000}"
```

Usage:

Command	Purpose
<code>server</code>	Serve current directory on port 8000
<code>server 3000</code>	Serve on port 3000

This opens the browser automatically and starts Python's built-in HTTP server. Useful for pre-viewing static sites, testing HTML files, or sharing files on a local network.

Networking Aliases

Alias	Expands To	Purpose
<code>ip</code>	<code>dig +short myip.opendns.com @resolver1.opendns.com</code>	Show your public IP address
<code>localip</code>	<code>ipconfig getifaddr en0</code>	Show your local network IP
<code>flush</code>	<code>dscacheutil -flushcache && killall -HUP mDNSResponder</code>	Flush the mDNS responder

Practical Recipes

Diagnose high CPU usage

```
# Launch btop, press 'f' to filter, type the suspect process name
btop
```

Alternatively, for a quick non-interactive check:

```
ps aux | sort -rn -k 3 | head -10
```

Check disk space across all volumes

```
df -h | grep -v tmpfs
```

Or for specific directories:

```
fs ~/Projects ~/Documents ~/Downloads
```

Monitor network connections

```
# In btop, press 'd' to toggle the network panel  
# For a quick snapshot outside btop:  
lsof -i -P -n | grep LISTEN
```

Quick system overview for troubleshooting

```
fastfetch
```

This gives you OS version, kernel, uptime, and hardware specs in one shot – useful when filing bug reports or asking for help.

Find what is eating disk space

```
# Top-level breakdown  
fs /*  
  
# Drill into a specific directory  
fs ~/Library/*  
  
# Find largest files recursively  
find . -type f -exec du -h {} + | sort -rh | head -20
```

Check if a port is in use

```
lsof -i :8080
```

Kill a process by name

```
# Find it  
pgrep -l node  
  
# Kill it  
pkill node  
  
# Or use btop: navigate to the process, press 'k', select SIGTERM
```

Advanced Usage

btop Themes

btop supports themes stored in `~/.config/btop/themes/`. Community themes are available, including Catppuccin variants that match the rest of your terminal aesthetic:

```
# Download Catppuccin theme for btop
curl -o ~/.config/btop/themes/catppuccin_mocha.theme \
  https://raw.githubusercontent.com/catppuccin/btop/main/themes/catppuccin_mocha.th
```

Then set `color_theme = "catppuccin_mocha"` in `~/.config/btop/btop.conf`.

Scripting System Checks

Combine these tools in a morning startup script:

```
#!/bin/bash
echo "====System_Status===="
fastfetch --structure OS:Uptime:CPU:Memory --logo none
echo ""
echo "====Disk_Usage===="
df -h / | tail -1
echo ""
echo "====Public_IP===="
dig +short myip.opendns.com @resolver1.opendns.com
```

Stats Menu Bar Customization

Stats (the menu bar app) is configured through its GUI preferences. You can choose which monitors to display:

- CPU utilization (percentage or graph)
- Memory pressure
- Disk read/write rates
- Network upload/download speeds
- Battery percentage and time remaining
- Fan speed and temperature sensors

Each widget can be customized for colour, update interval, and display format. Since it runs as a menu bar app, it provides always-visible monitoring without consuming a terminal window.

Maintenance Aliases

Your setup also includes maintenance shortcuts:

Alias	Purpose
<code>cleanup</code>	Delete all <code>.DS_Store</code> files recursively
<code>emptytrash</code>	Force-empty Trash and system log files
<code>update</code>	Run macOS software update, Homebrew update/upgrade/cleanup

The update alias is especially useful as a single command to keep everything current:

```
alias update="sudo softwareupdate -i -a; brew update; brew upgrade; brew cleanup"
```

Shell History: atuin

A replacement for Ctrl+R that provides full-text search, cross-machine sync, and statistics for your entire shell history.

Your Setup

atuin is installed via Homebrew and initialized in `~/.zshrc`:

```
# Brewfile
brew "atuin"

# ~/.zshrc
eval "$(atuin_init_zsh)"
```

The configuration lives at `~/.config/atuin/config.toml`:

```
style = "compact"
inline_height = 30
enter_accept = true
```

These three settings define your atuin experience:

- **style = “compact”** – Uses a single-line-per-entry display instead of the default multi-line style, fitting more history entries on screen.
- **inline_height = 30** – The search UI appears inline in the terminal (not fullscreen) and shows up to 30 entries at a time.
- **enter_accept = true** – Pressing Enter immediately executes the selected command rather than pasting it into the command line for editing.

atuin is initialized after `fzf` in your `zshrc`, which means atuin’s Ctrl+R binding takes precedence over `fzf`’s default history search. This is intentional – atuin provides a richer history search experience, while `fzf` continues to serve as a general-purpose fuzzy finder for files, directories, and other uses.

Core Concepts

What atuin Replaces

By default, `zsh`’s Ctrl+R performs a simple reverse incremental search through the history file (`~/.zsh_history`). This search is limited: it only matches from the beginning of the command, it does not understand context, and history is lost when it exceeds `HISTSIZE`.

atuin replaces this with a SQLite-backed history database that stores every command along with metadata: the working directory where it was run, the exit code, the session ID, the hostname, and the timestamp. This enables searches like “show me all git commands I ran in this project that failed.”

How It Works

When you press Ctrl+R, atuin opens an inline search interface in your terminal. As you type, it performs full-text search across your entire history. Results are ranked by recency and frequency by default. You navigate with arrow keys and select with Enter.

atuin hooks into the shell’s preexec and precmd functions to record every command as it is executed, including the exit code after it completes.

Sync Across Machines

atuin can optionally sync your shell history across multiple machines using an encrypted sync server. Each command is end-to-end encrypted before leaving your machine. You can use the public atuin sync server or self-host one.

Essential Commands

Command / Key	Purpose
Ctrl+R	Open interactive history search
Type while in search	Filter results by full-text match
Up/Down	Navigate through search results
Enter	Execute the selected command (with enter_accept = true)
Tab	Paste selected command into prompt for editing
Esc	Close the search UI
Ctrl+R (again, while open)	Cycle through search modes

atuin CLI Commands

Command	Purpose
atuin search "pattern"	Search history from the command line
atuin search --exit 0 "pattern"	Search only successful commands
atuin search --exit 1 "pattern"	Search only failed commands
atuin search --cwd /path "pattern"	Search commands run in a specific directory
atuin search --after "2025-01-01" "pattern"	Search after a date
atuin search --before "2025-06-01" "pattern"	Search before a date
atuin stats	Show shell usage statistics
atuin stats --count 20	Show top 20 most-used commands
atuin history list	List recent history entries
atuin history list --cmd-only	List commands only (no metadata)
atuin import auto	Import history from zsh/bash history files
atuin import zsh	Import specifically from zsh history

Command	Purpose
<code>atuin register</code>	Create an account for sync
<code>atuin login</code>	Log in to an existing sync account
<code>atuin sync</code>	Manually trigger a sync
<code>atuin key</code>	Show your encryption key (save this)
<code>atuin default --config</code>	Print the full default configuration

Configuration Reference

The full configuration file lives at `~/.config/atuin/config.toml`. Your setup uses minimal configuration, relying on sensible defaults for most settings. Here are the key options:

Setting	Your Value	Default	Purpose
<code>style</code>	<code>compact</code>	<code>auto</code>	UI layout: compact, full, or auto
<code>inline_height</code>	<code>30</code>	<code>0 (fullscreen)</code>	Number of lines for inline display
<code>enter_accept</code>	<code>true</code>	<code>false</code>	Enter executes vs pastes to prompt

Additional Configuration Options

Setting	Default	Purpose
<code>search_mode</code>	<code>fuzzy</code>	Search algorithm: prefix, fulltext, fuzzy, skim
<code>filter_mode</code>	<code>global</code>	Scope: global, host, session, directory
<code>filter_mode_shell_up_key_binding</code>	<code>global</code>	Scope for Up arrow history
<code>show_preview</code>	<code>false</code>	Show full command preview below results
<code>show_help</code>	<code>true</code>	Show keybinding help in search UI
<code>max_preview_height</code>	<code>4</code>	Lines for command preview
<code>history_filter</code>	<code>[]</code>	Regex patterns to exclude from history
<code>secrets_filter</code>	<code>true</code>	Auto-filter commands with secrets/tokens
<code>workspaces</code>	<code>false</code>	Enable per-workspace history filtering
<code>sync_frequency</code>	<code>"1h"</code>	How often to auto-sync
<code>sync_address</code>	<code>"https://api.atuin.sh"</code>	Sync server URL

Practical Recipes

Find that docker command you ran last week

Press Ctrl+R and start typing docker. atuin searches across your entire history, not just the current session. Results show the most recent matches first.

Search by directory

```
atuin search --cwd ~/Projects/my-app "npm"
```

This finds all npm commands you ran specifically in that project directory – useful when you cannot remember the exact test or build command.

Find commands that failed

```
atuin search --exit 1 "make"
```

Only shows make commands that exited with a non-zero status. Useful for revisiting build failures.

View your most-used commands

```
atuin stats
```

This produces a ranked list of your most frequently used commands, giving you insight into your workflow patterns. Example output:

```
[#####] 542  git
[##### ] 431  nvim
[##### ] 389  cd
[##### ] 267  ls
[##### ] 198  cat
```

Remember that ls is aliased to eza and cat is aliased to bat, so these counts reflect the aliases you actually type.

Import existing history

If you are setting up atuin on a new machine, import your existing zsh history:

```
atuin import auto
```

This detects your shell and imports from the appropriate history file. All imported commands will be searchable immediately.

Exclude sensitive commands from history

Add patterns to history_filter in the config file:

```
history_filter = [
    "^export.*TOKEN",
    "^export.*SECRET",
```

```
"^export.*PASSWORD",
]
```

Commands matching these patterns will not be recorded. The `secrets_filter = true` default also provides automatic detection of common secret patterns.

Advanced Usage

Search Modes

atuin supports multiple search algorithms, cycled with Ctrl+R during an active search:

Mode	Behaviour
fuzzy	Characters can be non-contiguous (like fzf)
fulltext	Substring match anywhere in the command
prefix	Match from the start of the command
skim	Skim-based fuzzy matching

Your default is fuzzy, which is the most forgiving. If you get too many results, switch to prefix for a narrower match.

Filter Modes

The filter mode controls the scope of the search:

Mode	Scope
global	All history from all machines (if synced)
host	Only commands from the current machine
session	Only commands from the current terminal session
directory	Only commands run in the current directory

You can change the default filter mode in the config or cycle through modes during a search.

Syncing Across Machines

To set up sync:

On the first machine

```
atuin register -u your-username -e your@email.com -p your-password
atuin key # Save this key — you need it for other machines
```

On additional machines

```
atuin login -u your-username -p your-password
atuin import auto
atuin sync
```

All data is end-to-end encrypted with your key. The sync server (whether self-hosted or the public server at `api.atuin.sh`) never sees your plaintext commands.

atuin and the Shell Initialization Order

Your zshrc initializes tools in this order:

```
[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh      # fzf bindings (sets Ctrl+R)
eval "$(zoxide_init_zsh cmd cd)"          # zoxide (sets cd)
eval "$(atuin_init_zsh)"                  # atuin (overrides Ctrl+R)
eval "$(mise_activate_zsh)"               # mise (tool versions)
```

atuin is loaded after fzf deliberately. fzf sets up a Ctrl+R binding for history search, and atuin overwrites it with its own, superior implementation. If you ever want fzf's simpler history search back, swap the order or comment out the atuin line.

Up Arrow Integration

atuin can also replace the Up arrow key behaviour, turning it from “previous command” into a context-aware history search:

```
# In config.toml (not in your current config, but available)
filter_mode_shell_up_key_binding = "directory"
```

With this setting, pressing Up would cycle through commands previously run in the current directory rather than the global history. This is particularly useful in project directories where you repeatedly run the same build or test commands.

Node.js: NVM and Bun

Two runtimes, one language – NVM for compatibility, Bun for speed.

Overview

This environment runs two JavaScript runtimes side by side. NVM (Node Version Manager) provides traditional Node.js with per-project version switching. Bun provides a faster alternative runtime, bundler, test runner, and package manager in a single binary. Both are configured in the shell and managed through chezmoi.

The division of labor is straightforward: use Node.js via NVM when a project requires it (most production apps, CI pipelines, legacy codebases), and use Bun when starting fresh or when speed matters (scripting, local tooling, rapid prototyping).

NVM Configuration

NVM is initialized in `~/.zshrc` with three lines that chezmoi manages via `dot_zshrc.tmpl`:

```
export NVM_DIR="$HOME/.nvm"  
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
```

The first line sets the directory where NVM stores its Node versions and configuration. The second sources NVM itself, making the `nvm` command available. The third enables tab completion for NVM commands.

Automatic Installation

Chezmoi handles NVM and Node.js installation through a run-on-change script:

```
run_onchange_install-nvm-and-node.sh.tmpl
```

This script is a chezmoi template that reads from `.node-version` in the chezmoi source directory. When the content of `.node-version` changes, chezmoi re-runs the script, which installs the specified Node version and sets it as the default. The current pinned version is Node 22.

The script performs three steps:

1. Install NVM itself if the `$NVM_DIR` directory does not exist
2. Source NVM into the current shell
3. Run `nvm install` and `nvm alias default` for the version specified in `.node-version`

Essential NVM Commands

Command	Purpose
<code>nvm ls</code>	List all locally installed Node versions
<code>nvm ls--remote --lts</code>	List available LTS versions for download
<code>nvm use 22</code>	Switch to Node 22 in the current shell
<code>nvm install 22</code>	Download and install Node 22
<code>nvm alias default 22</code>	Set Node 22 as the default for new shells
<code>nvm current</code>	Print the currently active Node version
<code>nvm which 22</code>	Print the path to the Node 22 binary
<code>nvm uninstall 20</code>	Remove Node 20 from disk

Per-Project Version Pinning

When you enter a directory containing a `.node-version` or `.nvmrc` file, you can run `nvm use` to switch to the version specified in that file. The `.node-version` file in the chezmoi source directory serves double duty: it pins the global default version and triggers the chezmoi run-on-change script when updated.

To update the global Node version:

```
chezmoi cd
echo "22" > .node-version
chezmoi apply
```

This changes the file, triggers the install script, and ensures every machine managed by chezmoi converges on the same Node version.

Bun Configuration

Bun is configured in `~/.zshrc` with three lines:

```
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"
[ -s "$HOME/.bun/_bun" ] && source "$HOME/.bun/_bun"
```

The first line sets the Bun installation directory. The second adds the Bun binary to `PATH`. The third sources Bun's shell completion script for tab completion of Bun commands and subcommands.

Essential Bun Commands

Command	Purpose
<code>bun run dev</code>	Run the dev script from package.json
<code>bun run build</code>	Run the build script from package.json
<code>bun install</code>	Install all dependencies from package.json
<code>bun add react</code>	Add a dependency
<code>bun add -d vitest</code>	Add a dev dependency
<code>bun dev</code>	Shorthand for <code>bun run dev</code>
<code>bun build ./src/index.ts</code>	Bundle a TypeScript entry point
<code>bun test</code>	Run tests using Bun's built-in test runner
<code>bunx create-next-app</code>	Execute a package binary without installing
<code>bun upgrade</code>	Upgrade Bun itself to the latest version

Why Bun Alongside Node

Bun is not a full replacement for Node.js. Some libraries assume Node-specific APIs. CI environments typically run Node. Production deployments on most platforms target Node. But Bun offers compelling advantages for local development:

- **Speed.** `bun install` is significantly faster than `npm install` or `yarn install`. For large monorepos, this saves minutes per install.
- **Built-in bundler.** `bun build` replaces webpack, esbuild, or Rollup for many use cases.
- **Built-in test runner.** `bun test` is Jest-compatible but faster.
- **TypeScript out of the box.** Bun runs `.ts` files directly without a compilation step.
- **Single binary.** No dependency on a separate package manager installation.

The practical rule: start with Bun for new projects, fall back to Node when something does not work.

How They Interact

NVM and Bun do not conflict. NVM manages Node.js versions in `~/.nvm/versions/node/`. Bun lives in `~/.bun/`. Both are on PATH, but they serve different node binaries. When you run `node`, you get the NVM-managed version. When you run `bun`, you get Bun's runtime.

For projects that need Node.js specifically:

```
nvm use           # reads .node-version or .nvmrc
npm install       # uses Node's npm
npm run dev       # runs via Node
```

For projects that use Bun:

```
bun install       # uses Bun's package manager
bun dev           # runs via Bun's runtime
```

For projects that mix both (common in transitional codebases):

```
nvm use          # ensure correct Node version
bun install      # use Bun for faster installs (compatible lockfile)
bun run dev      # use Bun to execute the dev script
```

Chezmoi Integration

The `.node-version` file in the chezmoi source directory is the single source of truth for the default Node.js version across all machines. The install script template includes the file content in a comment line:

```
# Node version: {{ include ".node-version" | trim }}
```

This chezmoi template directive means the script's content hash changes whenever `.node-version` changes. Since the script uses `run_onchange_` prefix, chezmoi detects the content change and re-runs the script automatically during chezmoi apply.

This pattern – using chezmoi template includes to trigger re-runs – is a powerful way to tie installation scripts to data files without manual intervention.

Troubleshooting

Problem	Solution
nvm: command not found	Source NVM: <code>. "\$NVM_DIR/nvm.sh"</code>
Wrong Node version in a project	Add <code>.node-version</code> to the project root
bun: command not found	Check that <code>\$BUN_INSTALL/bin</code> is on PATH
Bun install fails on native module	Fall back to npm install for that project
NVM is slow to load	Normal on first shell load; subsequent shells cache

Quick Reference

```
# Check versions
node —version      # Node.js version (via NVM)
bun —version        # Bun version
nvm current          # Which Node NVM is using

# Switch Node versions
nvm use 22           # Use Node 22
nvm install —lts     # Install latest LTS

# Bun workflow
bun init             # Scaffold a new project
```

```
bun install          # Install dependencies
bun dev              # Start dev server
bun test            # Run tests
bunx degit user/repo my-project # Clone a template
```

Version Management: mise

One tool to manage every runtime version – faster than asdf, simpler than juggling version managers.

Overview

mise (pronounced “meez”) is a polyglot runtime version manager that replaces the need for separate version managers per language. Instead of installing nvm for Node, pyenv for Python, rbenv for Ruby, and so on, mise handles all of them through a single CLI with a unified configuration format.

mise is the spiritual successor to asdf. It is compatible with asdf’s .tool-versions file format but is written in Rust, which gives it significantly faster startup and execution times. Where asdf adds noticeable latency to shell initialization through shims, mise uses a different activation strategy that keeps the shell responsive.

Installation and Activation

mise is installed through two mechanisms in this environment.

First, it is listed in the essential Brewfile:

```
brew "mise"
```

Second, a dedicated chezmoi run-on-change script ensures mise is available even if Homebrew has not been run yet:

```
#!/bin/bash
set -euo pipefail

if ! command -v mise &>/dev/null; then
  echo "Installing mise ..."
  curl https://mise.run | sh
  echo "mise installed."
else
  echo "mise is already installed."
fi
```

Activation happens in `~/.zshrc` with a single line:

```
eval "$(mise activate zsh)"
```

This line is critical. Unlike `asdf`, which uses shims (wrapper scripts in a directory on `PATH`), `mise` modifies `PATH` directly each time you change directories. This means the correct version of a tool is always the real binary, not a shim that adds overhead to every invocation.

The activation line must come after other `PATH` modifications but before the Starship prompt initialization, which is the order maintained in `dot_zshrc.tmpl`.

Configuration Files

`mise` supports two configuration file formats. You can use either or both in a project.

.tool-versions (asdf-compatible)

```
python 3.12.0
ruby 3.3.0
golang 1.22.0
```

This format is one tool per line, with the tool name and version separated by a space. It is compatible with `asdf`, so teams that have not migrated to `mise` can still use the same file.

mise.toml (mise-native)

```
[tools]
python = "3.12.0"
ruby = "3.3.0"
golang = "1.22.0"

[env]
VIRTUAL_ENV = "${env.HOME}/.virtualenvs/myproject"
```

The TOML format is more expressive. It supports environment variable configuration, tool-specific settings, and conditional logic. For new projects, `mise.toml` is the preferred format.

Global Configuration

`mise` also reads a global configuration file at `~/.config/mise/config.toml`. This sets default versions for tools when no project-level configuration is present.

Essential Commands

Installing and Using Tools

Command	Purpose
<code>mise install python@3.12</code>	Install Python 3.12
<code>mise install python</code>	Install the version specified in config
<code>mise install</code>	Install all tools specified in the current config
<code>mise use python@3.12</code>	Install and pin Python 3.12 in current directory
<code>mise use -g python@3.12</code>	Set Python 3.12 as the global default
<code>mise uninstall python@3.11</code>	Remove a specific installed version

Querying State

Command	Purpose
<code>mise ls</code>	List all installed tool versions
<code>mise ls python</code>	List installed Python versions
<code>mise current</code>	Show currently active versions for all tools
<code>mise current python</code>	Show the active Python version
<code>mise where python@3.12</code>	Print the installation path for a version
<code>mise ls--remote python</code>	List all available Python versions for install

Managing Configuration

Command	Purpose
<code>mise use python@3.12</code>	Write version to <code>.mise.toml</code> in current dir
<code>mise use -g node@22</code>	Write version to global config
<code>mise env</code>	Print the environment variables mise would set
<code>mise doctor</code>	Diagnose common configuration problems
<code>mise settings</code>	Show all mise settings

How mise Resolves Versions

When you run a command like `python`, mise determines which version to use by checking these locations in order:

1. **Command-line override:** `mise exec python@3.11 -- python script.py`
2. **Current directory:** `.mise.toml` or `.tool-versions` in the working directory
3. **Parent directories:** mise walks up the directory tree looking for config files
4. **Global config:** `~/.config/mise/config.toml`
5. **System default:** the version installed by the OS or Homebrew

This hierarchical resolution means you can set a global default and override it per-project without any manual switching. When you `cd` into a project directory, mise automatically adjusts `PATH` to point to the correct versions.

mise vs. NVM

This environment uses both mise and NVM. This is not redundant – they serve different roles.

NVM is the established standard for Node.js version management. It is deeply integrated with the Node ecosystem. The `.node-version` and `.nvmrc` files are recognized by CI platforms, deployment services, and IDE plugins. The `chezmoi` run-on-change script that installs Node relies on NVM.

`mise` handles everything else. If a project needs Python, Ruby, Go, Terraform, or any other tool that has a `mise` plugin, `mise` manages it. For Node specifically, NVM remains the primary manager because of its ecosystem integration, but `mise` can serve as a backup or alternative.

The practical guideline:

Tool	Managed by	Reason
Node.js	NVM	Ecosystem standard, <code>chezmoi</code> integration
Python	<code>mise</code>	Replaces <code>pyenv</code>
Ruby	<code>mise</code>	Replaces <code>rvm</code>
Go	<code>mise</code>	Replaces manual downloads
Rust	<code>rustup</code>	Rust has its own excellent toolchain
All others	<code>mise</code>	Single tool, consistent interface

Practical Workflows

Setting Up a Python Project

```
mkdir ~/Projects/my-python-app && cd $_
mise use python@3.12
python --version    # Python 3.12.x
python -m venv .venv
source .venv/bin/activate
```

Setting Up a Multi-Language Project

```
mkdir ~/Projects/fullstack-app && cd $_
mise use python@3.12 golang@1.22
cat .mise.toml
# [tools]
# python = "3.12"
# golang = "1.22"
```

Checking What Is Active

```
$ mise current
python    3.12.0    ~/Projects/my-app/.mise.toml
golang    1.22.0    ~/Projects/my-app/.mise.toml
```

Updating a Tool

```
mise ls-remote python | tail -5    # see latest versions
mise use python@3.13              # update the config
mise install                      # install the new version
```

Plugins

mise uses a plugin system to support different tools. Most common tools have built-in support (called “core plugins”), which means they work without installing anything extra. For less common tools, mise can use asdf plugins.

To see available plugins:

```
mise plugins ls-remote
```

Core plugins (no installation needed) include: Python, Node, Ruby, Go, Java, Erlang, Elixir, and many others.

Troubleshooting

Problem	Solution
mise: command not found	Run the install script or <code>brew install mise</code>
Wrong version active	Check <code>mise current</code> and <code>mise doctor</code>
Version not installing	Try <code>mise install --verbose</code> for detailed output
Slow shell startup	Ensure <code>eval "\$(mise activate zsh)"</code> is used, not shims
Config file not detected	Verify filename: <code>.mise.toml</code> or <code>.tool-versions</code>

Quick Reference

```
# Install a runtime
mise install python@3.12

# Pin version to current project
mise use python@3.12

# See what is active
mise current

# See all installed versions
mise ls
```

Diagnose issues
mise doctor

Dotfiles Management: chezmoi

One source directory, every machine in sync – chezmoi turns dotfile chaos into declarative configuration.

Overview

chezmoi is a dotfiles manager that keeps configuration files in a source directory and applies them to the home directory. Unlike symlink-based tools like GNU Stow, chezmoi copies files to their target locations after optionally processing them through a template engine. This approach is more robust: the target files are real files, not symlinks, so tools that do not follow symlinks work correctly.

The source directory for this environment is:

`~/.local/share/chezmoi/`

This directory is a Git repository. Changes to dotfiles are committed, pushed, and pulled like any other code. On a new machine, a single command bootstraps the entire environment from the remote repository.

Core Concepts

Source State vs. Target State

chezmoi maintains a distinction between two states:

- **Source state:** the files in `~/.local/share/chezmoi/`. These are the templates, scripts, and data that define what the configuration should look like.
- **Target state:** the files in `~/` and `~/.config/`. These are the actual files that tools read.

The chezmoi `apply` command computes the difference between the source state and target state, then modifies the target to match the source.

Naming Conventions

chezmoi uses filename prefixes in the source directory to control how files are handled:

Source prefix	Target behavior	Example
dot_	Replaced with . in target	dot_zshrc becomes ~/.zshrc
dot_config/	Maps to ~/.config/	dot_config/starship.toml
private_	File gets 0600 permissions	private_Library/ for LaunchAgents
run_once_	Script runs once, never again	run_once_macos—defaults.sh
run_onchange_	Script re-runs when its content hash changes	run_onchange_install—zsh—plugins.sh
run_onchange_before_	Runs before other operations when content changes	run_onchange_before_install—brew—packages.sh
.tmpl	Processed through Go template engine	dot_zshrc.tmpl

These prefixes compose. A file named `run_onchange_before_install—brew—packages.sh.tmpl` is a script that runs before other targets, re-runs when its content changes, and is processed as a template before execution.

Essential Commands

Daily Usage

Command	Purpose
chezmoi cd	Open a shell in the source directory
chezmoi edit FILE	Edit a target file in its source form
chezmoi diff	Show what would change on the next apply
chezmoi apply	Apply source state to target state
chezmoi apply -v	Apply with verbose output showing each action
chezmoi status	Show files that differ between source and target
chezmoi update	Pull latest from remote and apply

Managing Files

Command	Purpose
chezmoi add ~/.gitconfig	Add a file from target to source
chezmoi add --template FILE	Add a file as a template
chezmoi managed	List all files chezmoi manages
chezmoi managed --include=files	List only managed files (not directories)
chezmoi forget FILE	Stop managing a file (remove from source)
chezmoi re-add	Update source from modified target files

Inspection

Command	Purpose
<code>chezmoi cat FILE</code>	Print the target-state content of a file
<code>chezmoi source-path FILE</code>	Show the source path for a target file
<code>chezmoi data</code>	Print all template data as JSON
<code>chezmoi doctor</code>	Diagnose common configuration problems
<code>chezmoi git status</code>	Run git commands in the source directory

Templates

Files ending in `.tmpl` are processed through Go's text/template engine before being written to the target. This allows machine-specific customization from a single source file.

Example: `dot_zshrc.tmpl`

The main shell configuration uses a template for the optional project bin path:

```
{{ if dig "projectBinPath" "" . }}
export PATH="$HOME/{{ .projectBinPath }}:$PATH"
{{ end }}
```

The `dig` function checks whether `projectBinPath` is defined in `chezmoi`'s template data. If it is, the `PATH` export is included in the rendered `.zshrc`. If not, that block is omitted entirely.

Template data is defined in `~/.config/chezmoi/chezmoi.toml`:

```
[data]
projectBinPath = "work/bin"
```

Template Functions

`chezmoi` provides template functions beyond the Go standard library:

Function	Purpose
<code>{{ .chezmoi.os }}</code>	Operating system (darwin, linux)
<code>{{ .chezmoi.arch }}</code>	Architecture (amd64, arm64)
<code>{{ .chezmoi.hostname }}</code>	Machine hostname
<code>{{ .chezmoi.sourceDir }}</code>	Path to chezmoi source directory
<code>{{ include "file" }}</code>	Include contents of another source file
<code>{{ include "file" \ trim }}</code>	Include and trim whitespace
<code>{{ output "cmd" "arg" }}</code>	Capture command output

Run Scripts

Run scripts are executable files in the source directory that chezmoi runs during chezmoi apply. They handle tasks that cannot be expressed as file copies: installing packages, compiling tools, setting system defaults.

Scripts in This Environment

Script	Type	Purpose
run_once_macos-defaults.sh	Run once	Set macOS system preferences
run_once_install-gh-extensions.sh	Run once	Install GitHub CLI extensions
run_onchange_before_install-brew-packages.sh	On change	Install Homebrew and run brew bundle
run_onchange_install-nvm-and-node.sh.tmp	On change	Install NVM and pin Node version
run_onchange_install-zsh-plugins.sh	On change	Install Oh My Zsh plugins
run_onchange_install-tmux-plugins.sh	On change	Install TPM and tmux plugins
run_onchange_install-mise.sh	On change	Install mise runtime manager

Run-once scripts execute the first time chezmoi apply runs on a machine, then never again. chezmoi records their completion in its state database. This is ideal for one-time setup like macOS defaults.

Run-on-change scripts execute whenever their content changes. For template scripts (.tmpl suffix), this means the rendered content is hashed. When a Brewfile changes, the hash of the rendered install script changes, triggering a re-run. This is the mechanism that keeps packages in sync across machines.

Before scripts (run_onchange_before_) run before chezmoi writes any files. This ensures that package managers are up to date before configuration files that depend on installed packages are written.

Auto-Sync with LaunchAgent

A macOS LaunchAgent ensures dotfiles stay synchronized automatically:

```
~/Library/LaunchAgents/com.chezmoi.update.plist
```

This LaunchAgent runs `chezmoi update --no-tty` at login. The `--no-tty` flag prevents chezmoi from prompting for confirmation, which would fail in a non-interactive context. Output is logged to `/tmp/chezmoi-update.log`.

The plist file itself is managed by chezmoi under `private_Library/LaunchAgents/`, using the `private_` prefix to ensure correct permissions on the Library directory.

Workflow: Making a Configuration Change

The typical workflow for modifying a dotfile:

```
# 1. Navigate to the source directory
chezmoi cd

# 2. Edit the file (or use chezmoi edit)
nvim dot_tmux.conf

# 3. Preview what will change
chezmoi diff

# 4. Apply changes
chezmoi apply

# 5. Commit and push
git add dot_tmux.conf
git commit -m "Update␣tmux␣configuration"
git push
```

Alternatively, for quick edits:

```
# Edit the source file for a target
chezmoi edit ~/.tmux.conf

# This opens the source file (dot_tmux.conf) in your editor.
# After saving, apply:
chezmoi apply
```

Bootstrapping a New Machine

To set up a fresh macOS machine from the dotfiles repository:

```
# 1. Install chezmoi and initialize from the repo
sh -c "$(curl␣-fsLS␣get.chezmoi.io)" — init —apply YOUR_GITHUB_USERNAME

# 2. chezmoi runs all scripts in order:
#   - before scripts: install Homebrew and packages
#   - on-change scripts: install NVM, Node, plugins
#   - once scripts: set macOS defaults
#   - file copies: deploy all dotfiles

# 3. Restart the shell
exec zsh
```

After that single command, the machine has every tool installed, every configuration file deployed, and every macOS default set.

Troubleshooting

Problem	Solution
chezmoi diff shows unexpected changes	Run chezmoi data to check template values
Script fails during apply	Check the script output, fix, re-run apply
Template renders incorrectly	Use chezmoi cat FILE to see rendered output
File permissions wrong	Use private_ or executable_ prefix
Want to stop managing a file	chezmoi forget ~/path/to/file

Package Management: Brewfiles

Four tiers of packages, from survival essentials to specialty tools – install exactly what you need.

Overview

This environment uses a tiered Brewfile system to manage Homebrew packages. Instead of a single monolithic Brewfile, packages are organized into four tiers based on how essential they are to daily work. This design serves two purposes: on a new machine, you can install only the base tier to get a functional environment in minutes, then add tiers as needed. On an existing machine, the tier structure makes it clear which packages are critical and which are optional.

All Brewfiles live in the chezmoi source directory under `brewfiles/`:

```
~/ .local/share/chezmoi/brewfiles/  
  00-base.Brewfile  
  10-essential.Brewfile  
  15-nice-to-have.Brewfile  
  20-optional.Brewfile
```

A master Brewfile in the repository root aggregates all tiers for a full install.

Tier 0: Base (00-base.Brewfile)

Core CLI tools that form the foundation of the development environment. Without these, basic navigation, editing, and shell functionality would be degraded. Every machine gets this tier.

Package	Purpose
chezmoi	Dotfiles manager (manages itself)
git	Version control
fzf	Fuzzy finder for files, history, anything
bat	cat replacement with syntax highlighting
htop	System monitor (replaces htop/top)
eza	ls replacement with icons and git status
fd	find replacement, respects .gitignore

Package	Purpose
ripgrep	grep replacement, extremely fast
zoxide	cd replacement with frequency ranking
starship	Cross-shell prompt with git integration
neovim	The editor
tmux	Terminal multiplexer
tmuxinator	Tmux session layouts in YAML
curl	HTTP client
wget	File downloader
jq	JSON processor
grep	GNU grep (for extended regex support)
atuin	Shell history with sync and search
sesh	Tmux session manager with fzf integration
git-delta	Syntax-highlighting diff pager

These 21 packages are the absolute minimum for a productive terminal session. If you had to rebuild your environment on a plane with limited bandwidth, this is the tier to install.

Installing Base Only

```
brew bundle --file=~/.local/share/chezmoi/brewfiles/00-base.Brewfile
```

Tier 1: Essential (10-essential.Brewfile)

Daily driver tools and GUI applications. These are not strictly required for basic functionality, but working without them would mean constant friction. Most development days use every tool in this tier.

CLI Tools

Package	Purpose
lazygit	Terminal UI for git (staging, rebasing, etc.)
gh	GitHub CLI (PRs, issues, actions)
ast-grep	Structural code search and replace
fastfetch	System info display
diffnav	Interactive diff navigator
yazi	Terminal file manager with previews
tlldr	Simplified man pages with examples
imagemagick	Image processing from the command line
luarocks	Lua package manager (for Neovim plugins)
stow	Symlink farm manager (backup dotfile tool)
mise	Polyglot runtime version manager

GUI Applications (Casks)

Cask	Purpose
nikitabobko/tap/aerospace	Tiling window manager for macOS
iterm2	Terminal emulator
alt-tab	Windows-style Alt+Tab with previews
hiddenbar	Hide menu bar icons
font-symbols-only-nerd-font	Nerd Font icons (for eza, starship)
stats	Menu bar system monitor
claude-code	Claude AI desktop application
cursor	AI-enhanced code editor

Installing Essentials

```
brew bundle --file=~/.local/share/chezmoi/brewfiles/10-essential.Brewfile
```

Tier 2: Nice-to-Have (15-nice-to-have.Brewfile)

Useful tools and applications that enhance the workflow but are not needed every day. You might not notice their absence for a few days, but you would eventually miss them.

CLI Tools

Package	Purpose
aria2	Multi-connection download accelerator
ffmpeg	Video/audio processing
ffmpegthumbnailer	Video thumbnail generator (for Yazi)
fish	Friendly Interactive Shell (for testing)
ghostscript	PostScript/PDF interpreter
jj	Jujutsu VCS (experimental git alternative)
llvm	Compiler infrastructure (clangd, etc.)
mermaid-cli	Render Mermaid diagrams from CLI
pandoc	Universal document converter
poppler	PDF rendering library (for previews)
sevenzip	Archive manager
syncthing	File synchronization between devices
yt-dlp	Video downloader

GUI Applications (Casks)

Cask	Purpose
discord	Community chat

Cask	Purpose
docker—desktop	Container runtime and management
firefox@developer—edition	Development browser
iina	Modern macOS media player
itsycal	Menu bar calendar
keka	Archive utility
slack	Team communication
spotify	Music streaming
sublime—text	Quick text editor (for non-code files)
telegram	Messaging

Installing Nice-to-Have

```
brew bundle —file=~/.local/share/chezmoi/brewfiles/15—nice—to—have.Brewfile
```

Tier 3: Optional (20-optional.Brewfile)

Specialized tools for specific projects or roles. A frontend developer might never need redis or mysql—client. A backend developer might never need keycastr. Install these on demand.

CLI Tools

Package	Purpose
firebase—cli	Firebase project management
iredis	Interactive Redis CLI with autocomplete
mysql—client	MySQL command-line client
openvpn	VPN client
redis	In-memory data store (local dev)
telnet	Network debugging
worktrunk	Trunk-based development tool

GUI Applications (Casks)

Cask	Purpose
kap	Screen recorder
key—codes	Display key codes for key presses
keycastr	Show keystrokes on screen (for demos)
keycue	Shortcut cheat sheet overlay
mysqlworkbench	MySQL GUI client
time—out	Break reminder

VS Code Extensions

The optional tier also includes VS Code extensions for when Cursor or VS Code is used alongside Neovim:

Extension	Purpose
catppuccin.catppuccin—vsc	Catppuccin theme
catppuccin.catppuccin—vsc—icons	Catppuccin file icons
dbaeumer.vscode—eslint	ESLint integration
esbenp.prettier—vscode	Prettier formatter
bradlc.vscode—tailwindcss	Tailwind CSS intellisense
prisma.prisma	Prisma ORM support
svelte.svelte—vscode	Svelte language support
vue.volar	Vue language support
yoavbls.pretty—ts—errors	Human-readable TypeScript errors
gruntfuggly.todo—tree	Highlight and list TODOs
streetsidesoftware.code—spell—checker	Spell checking in code

UV Tools

Tool	Purpose
posting	Terminal-based HTTP client (via uv)

Installing Optional

```
brew bundle —file=~/.local/share/chezmoi/brewfiles/20—optional.Brewfile
```

The Master Brewfile

The Brewfile in the repository root aggregates all packages from all tiers into a single file. This is what the chezmoi run-on-change script uses:

```
brew bundle —file="{{_chezmoi.sourceDir_}}/Brewfile"
```

The master Brewfile is a flat list of all brew, cask, vscode, and uv entries. It is regenerated or manually maintained to include everything from all four tiers.

How chezmoi Triggers Brew Bundle

The install script `run_onchange_before_install—brew—packages.sh.tmpl` is a chezmoi template. Its first line includes a hash of the Brewfile:

```
# Brewfile hash: {{ include "Brewfile" | sha256sum }}
```

When any package is added to or removed from the Brewfile, the SHA-256 hash changes, the rendered script content changes, and chezmoi detects the change during chezmoi apply. The before_ prefix ensures Homebrew packages are installed before configuration files are written, so tools like starship and fzf are available when their configuration files land.

Selective Installation

For a minimal setup on a constrained machine:

```
# Just the survival kit
brew bundle —file=brewfiles/00—base.Brewfile
```

```
# Add daily drivers
brew bundle —file=brewfiles/10—essential.Brewfile
```

```
# Everything
brew bundle —file=Brewfile
```

For checking what is already installed versus what a Brewfile specifies:

```
brew bundle check —file=brewfiles/00—base.Brewfile
```

For cleaning up packages not in any Brewfile:

```
brew bundle cleanup —file=Brewfile
```

Quick Reference

```
# Install all packages
brew bundle —file=~/.local/share/chezmoi/Brewfile
```

```
# Install only base tier
brew bundle —file=~/.local/share/chezmoi/brewfiles/00—base.Brewfile
```

```
# Check what is missing
brew bundle check
```

```
# Remove packages not in Brewfile
brew bundle cleanup —force
```

```
# List installed packages
brew list
brew list —cask
```

Workflow: Daily Startup

From cold boot to full development session in under thirty seconds.

Overview

The daily startup workflow is designed to eliminate friction between opening the laptop and writing code. Every layer of the stack – the terminal, the multiplexer, the session manager, the window manager – is configured to restore state automatically or to provide fast paths to the right context. There is no manual window arrangement, no hunting for project directories, no waiting for tools to initialize.

This chapter walks through the sequence from opening a terminal to having a multi-project development session running across organized workspaces.

Step 1: Open the Terminal

When iTerm2 launches, attach to an existing tmux session or create a new one:

```
tmux attach || tmux
```

This means opening the terminal is equivalent to opening tmux. If a tmux server is already running (because the previous session was not shut down), it reattaches instantly, restoring all windows and panes exactly as they were left.

If no tmux server exists (fresh boot), tmux starts with a single window. The tmux-continuum plugin then restores the previous session layout automatically, because @continuum—restore is set to on.

Result after this step: You are inside tmux with your previous session state restored. Aerospace has already placed the iTerm2 window on Workspace 2 (terminals).

Step 2: Pick or Create a Project Session

With tmux running, the next action is to navigate to the right project. There are two primary mechanisms:

Option A: Sesh (prefix + T)

Press Ctrl+b then T. This launches the sesh session picker in an fzf popup:

The popup shows all available sessions and provides keyboard shortcuts to filter:

Key	Action
Ctrl+a	Show all sessions
Ctrl+t	Show only tmux sessions
Ctrl+g	Show config-defined sessions (from sesh.toml)
Ctrl+x	Show zoxide directories
Ctrl+f	Find directories under home (fd search)
Ctrl+d	Kill the selected tmux session
Tab	Move down in the list
Shift+Tab	Move up in the list

Selecting a session either attaches to an existing tmux session or creates a new one in the selected directory.

Option B: SessionX (prefix + o)

Press Ctrl+b then o. SessionX provides a similar session picker with zoxide integration (@sessionx-zoxide-mode is on). The difference is stylistic: SessionX has a different UI and preview layout.

Pre-Defined Sessions

The sesh.toml configuration defines named sessions that appear in the picker:

```
[[ session ]]
name = "dotfiles"
path = "~/.local/share/chezmoi"
startup_command = "nvim"

[[ session ]]
name = "nvim config"
path = "~/.config/nvim"
startup_command = "nvim lua/config/lazy.lua"
```

These sessions launch with a specific command already running. Selecting “dotfiles” opens the chezmoi source directory with Neovim ready to edit.

Result after this step: You are in a tmux session named after your project, in the project directory.

Step 3: Launch the IDE Layout

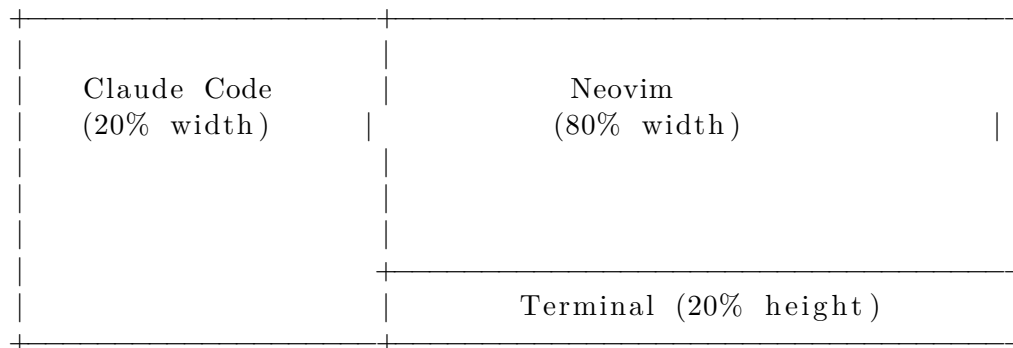
For a project that needs a full development layout, run:

```
ide .
```

Or for a specific directory:

```
ide ~/Projects/my-app
```

The `ide()` function creates a tmuxinator-driven layout with three panes:



- **Left pane (20%):** Claude Code for AI-assisted development
- **Right top pane (80%):** Neovim for editing
- **Right bottom pane (20%):** Terminal for running commands, tests, dev servers

The cursor starts in the Neovim pane. The tmux window is renamed to the project directory name.

Result after this step: You have a three-pane IDE layout with AI, editor, and terminal ready.

Step 4: Navigate Between Projects

Throughout the day, you move between project sessions. `zoxide` makes directory jumping fast:

```
cd my-app          # jumps to ~/Projects/my-app (or wherever you visit most)
cd dotfiles        # jumps to ~/.local/share/chezmoi
cd config          # jumps to ~/.config/nvim (or whichever "config" dir ranks highest)
```

Because `zoxide` is aliased to `cd` (`eval "$(zoxide init zsh --cmd cd)"`), every `cd` command is a frequency-weighted jump. Type a partial directory name and `zoxide` figures out where you mean.

To switch between tmux sessions without leaving the current one, use the `sesh` picker (prefix `+ T`) or simply:

```
tmux switch-client -t session-name
```

Step 5: Sessions Persist Automatically

Two tmux plugins handle session persistence:

- **tmux-resurrect:** Saves the complete tmux environment (sessions, windows, panes, working directories, and even running programs) to disk. It can restore Neovim sessions if `@resurrect-strategy-nvim` is set to `session`.
- **tmux-continuum:** Automatically saves the tmux environment at regular intervals and restores it when tmux starts. With `@continuum-restore` set to `on`, this happens transparently.

This means you can close the laptop, reboot, or even quit tmux, and the next time you open iTerm2, your full workspace is restored: every session, every window, every pane, in the right directories.

Step 6: Workspace Organization with Aerospace

Aerospace manages the macOS window layer above tmux. Windows are automatically assigned to workspaces based on application rules:

Workspace	Applications	Purpose
1	Firefox Developer Edition	Browsers
2	iTerm2	Terminals
3	Slack, Discord, Telegram	Communication
4	Spotify	Media / Other

Switching workspaces:

Key	Action
Alt+1	Switch to Workspace 1 (browsers)
Alt+2	Switch to Workspace 2 (terminals)
Alt+3	Switch to Workspace 3 (communication)
Alt+4	Switch to Workspace 4 (media)

Within a workspace, Aerospace tiles windows automatically. There is no manual dragging or resizing. When you open a new terminal window, it tiles next to the existing one. When you close one, the remaining windows fill the space.

Result after this step: Your screen real estate is organized by context, with instant workspace switching.

The Full Sequence

Here is the entire startup sequence, from opening the laptop to productive coding:

1. **Open iTerm2** – tmux auto-attaches, continuum restores sessions
2. **prefix + T** – pick a project session from sesh
3. **ide .** – launch the three-pane IDE layout (if not already set up)
4. **Alt+1 / Alt+2** – switch between browser and terminal workspaces

5. **Start coding** – Neovim is focused, Claude Code is in the left pane, terminal is below

Total time from cold boot to coding: under thirty seconds, most of which is macOS boot time.
From a warm resume (laptop lid open): under five seconds.

When Things Go Wrong

Problem	Solution
tmux did not auto-restore	Run <code>prefix + Ctrl+r</code> to manually restore
Wrong session after terminal launch	Use <code>prefix + T</code> to switch
IDE layout panes wrong size	Run <code>ide .</code> again in the project directory
Aerospace not tiling	Check if it is running: <code>aerospace list --windows</code>
zoxide jumps to wrong directory	<code>cd --</code> to go back, then use full path once
iTerm2 not launching on Workspace 2	Check Aerospace window rules in <code>aerospace.toml</code>

Quick Reference

Session management

```
prefix + T      # sesh session picker
prefix + o      # SessionX session picker
prefix + d      # detach from tmux
prefix + $      # rename current session
```

IDE layout

```
ide .           # three-pane layout in current dir
ide ~/Projects/app # three-pane layout in specific dir
```

Workspace switching (Aerospace)

```
Alt+1 through Alt+4 # switch workspaces
Alt+Shift+1 through Alt+Shift+4 # move window to workspace
```

Session persistence

```
prefix + Ctrl+s  # manual save (tmux-resurrect)
prefix + Ctrl+r  # manual restore (tmux-resurrect)
```

Workflow: Starting a New Project

From an empty directory to a running development session, using the full stack.

Overview

Starting a new project exercises nearly every tool in this environment: shell aliases for directory creation, NVM or Bun for runtime setup, Git for version control, chezmoi for configuration consistency, sesh for session management, the IDE layout for development, and Neovim for editing. This chapter walks through the complete sequence.

Step 1: Create the Project Directory

Use the `mkd` function, which combines `mkdir -p` and `cd`:

```
mkd ~/Projects/new-project
```

This creates the directory (including any missing parent directories) and immediately changes into it. You are now in an empty directory ready to scaffold.

Alternatively, if the directory already exists:

```
cd new-project
```

Because `cd` is aliased to `zoxide`, this jump works even if you are not in `~/Projects/`. `Zoxide` learns directory paths the first time you visit them.

Step 2: Initialize the Project

The initialization step depends on the type of project.

Node.js Project with Bun

```
bun init
```

This scaffolds a `package.json`, `tsconfig.json`, and `index.ts` entry point. Bun's init is fast and opinionated toward TypeScript.

For a framework-specific scaffold:

```
bunx create-next-app .           # Next.js
bunx create-svelte .             # SvelteKit
bunx create-vite . --template react-ts # Vite + React
```

Node.js Project with npm

If the project requires Node.js specifically:

```
nvm use           # reads .node-version if present
npm init -y       # scaffold package.json
```

Or for a framework:

```
npx create-next-app .
```

Python Project

```
mise use python@3.12
python -m venv .venv
source .venv/bin/activate
```

General Project (No Runtime)

For documentation, configuration, or non-runtime projects:

```
git init
```

Step 3: Initialize Git

If the scaffold command did not already initialize Git:

```
git init
```

Create a `.gitignore` appropriate to the project type. For a Node.js project:

```
cat > .gitignore << 'EOF'
node_modules/
dist/
.env
.env.local
*.log
.DS_Store
EOF
```

Make the initial commit:

```
git add .
git commit -m "Initial scaffold"
```

To create a GitHub repository and push:

```
gh repo create new-project --private --source=. --push
```

This single command creates a private repository on GitHub, sets it as the remote, and pushes the initial commit.

Step 4: Launch the IDE Layout

With the project initialized, launch the three-pane IDE layout:

```
ide .
```

This creates a tmux session named after the project directory with:

- Left pane (20% width): Claude Code for AI assistance
- Right top pane (80% width): Neovim
- Right bottom pane (20% height): Terminal for running commands

The cursor starts in the Neovim pane. From here, the standard development workflow begins.

Step 5: Register with Sesh (Optional)

For projects you will return to frequently, add an entry to `~/.config/sesh/sesh.toml`:

```
[[ session ]]
name = "new-project"
path = "~/Projects/new-project"
startup_command = "nvim"
preview_command = "eza --tree --level=2 --icons ~/Projects/new-project"
```

After adding this, the project appears in the sesh picker (prefix + T) with a tree preview. The `startup_command` means selecting this session immediately opens Neovim.

For projects you do not want to permanently register, zoxide handles discovery automatically. After visiting the directory a few times, `cd new-project` will jump to it from anywhere, and the zoxide list in the sesh picker (Ctrl+x) will include it.

Step 6: Start Coding in Neovim

With Neovim open in the project directory, the initial exploration usually follows this pattern:

Finding Files

Key	Action
Space+ff	Find files by name (Telescope)
Space+fg	Find files by content (live grep)
Space+fr	Find recently opened files
Space+fb	Find open buffers

File Explorer

Key	Action
Space+e	Toggle the file explorer (neo-tree)

Creating New Files

In the file explorer, press `a` to create a new file. Type the path including any subdirectories and they will be created automatically.

Alternatively, from the command line:

```
:e src/components/Header.tsx
```

This opens a new buffer for the file. The directory `src/components/` is created when you save.

Step 7: Git Workflow

As the project develops, the git workflow integrates lazygit and the GitHub CLI.

Local Development

Open lazygit from any pane:

```
lazygit
```

Or from Neovim (if configured with a lazygit plugin). Lazygit provides a visual interface for:

- Staging individual files or hunks
- Writing commit messages
- Viewing the commit graph
- Managing branches

Creating a Pull Request

When a feature is ready:

```
# Create and switch to a feature branch
git checkout -b feature/add-header

# Make commits...
# Then push and create PR
git push -u origin feature/add-header
gh pr create --fill
```

The `--fill` flag auto-fills the PR title and body from the commit messages.

Scaffold Cheat Sheet

Project Type	Command
Bun + TypeScript	<code>bun init</code>
Next.js	<code>bunx create-next-app .</code>
SvelteKit	<code>bunx create-svelte .</code>
Vite + React	<code>bunx create-vite . --template react-ts</code>
Vite + Vue	<code>bunx create-vite . --template vue-ts</code>
Node.js (npm)	<code>npm init -y</code>
Python	<code>mise use python@3.12 && python -m venv .venv</code>
Go	<code>mise use golang@1.22 && go mod init module-name</code>
Static site	<code>git init</code>

The Full Sequence

Putting it all together, here is the complete flow for a new TypeScript project:

```
# 1. Create and enter directory
mkd ~/Projects/my-app

# 2. Scaffold with Bun
bun init

# 3. Initialize git and push to GitHub
git init
git add .
git commit -m "Initial scaffold"
gh repo create my-app --private --source=. --push

# 4. Launch IDE layout
ide .

# 5. In Neovim: Space+ff to find files, start editing
```

6. In terminal pane: bun dev to start dev server
7. In Claude pane: ask for help, generate boilerplate

Total time from the idea to a running dev server: under two minutes.

Tips

- **Use mkd instead of mkdir && cd.** The function does both in one command and the `$__` variable ensures you end up in the right place.
- **Let Bun scaffold when possible.** `bun init` produces a minimal, TypeScript-ready project without the bloat of some framework CLIs.
- **Create the GitHub repo immediately.** `gh repo create --source=.` turns any local directory into a pushed repository in one command. Do not wait until the project is “ready.”
- **Register long-lived projects in sesh.toml.** If you will work on a project for more than a few days, the ten seconds spent adding a sesh entry saves minutes of navigation over the project’s lifetime.
- **Use the IDE layout from the start.** Even for quick experiments, the three-pane layout (AI + editor + terminal) is more productive than switching between separate windows.

Troubleshooting

Problem	Solution
nvm use says no .node-version	Create one: <code>echo "22" > .node-version</code>
bun init not found	Check PATH: <code>echo \$BUN_INSTALL/bin</code>
gh repo create fails auth	Run <code>gh auth login</code>
IDE layout opens in wrong dir	Pass the full path: <code>ide ~/Projects/my-app</code>
Neovim LSP not starting	Open a file of the right type; LSP auto-attaches

Workflow: Code Review

Pull request review from the terminal – checkout, read, understand, and respond without leaving the keyboard.

Overview

Code review in this environment happens entirely in the terminal. The GitHub CLI checks out pull requests, delta provides syntax-highlighted diffs with Catppuccin colors, Neovim enables deep code exploration, and lazygit offers a visual interface for navigating commits. The review is submitted back through the GitHub CLI without ever opening a browser.

This workflow is faster than browser-based review for most cases. The terminal tools provide better navigation (vim motions, fuzzy finding), better diffs (delta's side-by-side view with word-level highlighting), and better context (jump-to-definition, grep across the codebase).

Step 1: Checkout the Pull Request

Start by listing open pull requests:

```
gh pr list
```

This shows all open PRs with their numbers, titles, branches, and authors. To check out a specific PR:

```
gh pr checkout 42
```

This command does several things:

1. Fetches the PR branch from the remote
2. Creates a local branch tracking the remote PR branch
3. Switches to that branch

You are now on the PR branch with all its changes available locally.

To see a summary of the PR before diving into code:

```
gh pr view 42
```

This prints the PR title, body, labels, reviewers, and status checks.

Step 2: View the Overall Diff

Get a high-level view of what changed:

```
git diff main...HEAD
```

The three-dot syntax (main...HEAD) shows only the changes introduced by the PR branch, excluding any changes on main that happened after the branch point. This is the correct diff for reviewing a PR.

Because git-delta is configured as the default pager, this diff renders with:

- Syntax highlighting in the Catppuccin Mocha palette
- Line numbers on both sides
- Word-level diff highlighting (changed words within a line are emphasized)
- File headers with clear separation

For a summary of which files changed:

```
git diff main...HEAD --stat
```

This shows each changed file with insertion/deletion counts, giving you a map of the PR's scope before reading any code.

Step 3: Read the Diff with Delta

Delta is the syntax-highlighting pager configured for all git diff output. It transforms raw diffs into something readable.

Key delta features active in this configuration:

Feature	Effect
Catppuccin Mocha theme	Colors match the rest of the environment
Side-by-side mode	Old and new versions shown in parallel columns
Line numbers	Both old and new line numbers visible
Word-level highlighting	Changed words within a line are underlined
Navigate mode	Press n/N in the pager to jump between files

When viewing a large diff, use delta's navigation in the pager (less):

Key	Action
n	Jump to the next file in the diff
N	Jump to the previous file in the diff

Key	Action
/pattern	Search for a pattern in the diff
q	Quit the pager
g	Go to the top of the diff
G	Go to the bottom of the diff

For an interactive, navigable diff experience, use `diffnav`:

```
git diff main...HEAD | diffnav
```

`diffnav` provides a file list on the left and the diff on the right, with keyboard navigation between files.

Step 4: Explore the Code in Neovim

For deeper understanding, open Neovim in the project:

```
nvim .
```

Or if the IDE layout is already running, switch to the Neovim pane with `Ctrl+l` or `Ctrl+k` (`vim-tmux-navigator`).

Finding Changed Code

Key	Action
Space+ff	Find files by name
Space+fg	Grep for text across the project
Space+sg	Search with live grep (results update as you type)
Space+sw	Search for the word under cursor

Understanding Code

Key	Action
gd	Go to definition of symbol under cursor
gr	Find all references to symbol under cursor
K	Show hover documentation
Space+ca	Code actions (quick fixes, refactors)
Space+cd	Line diagnostics (errors, warnings)
[d /]d	Jump to previous/next diagnostic

Reviewing Specific Changes

To see the git diff for the current file inside Neovim, the Gitsigns plugin shows added, modified, and deleted lines in the gutter. Use:

Key	Action
]c	Jump to next changed hunk
[c	Jump to previous changed hunk

Step 5: Explore Commits in Lazygit

For PRs with many commits, lazygit provides a visual commit history:

```
lazygit
```

In lazygit, navigate to the commits panel to see each commit in the PR. Key navigation:

Key	Action
j/k	Move down/up in the commit list
Enter	View the diff for the selected commit
w	Toggle diff options (whitespace, context lines)
h/l	Switch between panels (files, commits, branches)
q	Quit lazygit

This is especially useful for understanding the evolution of a PR. You can step through commits one at a time to see how the author built the feature.

Step 6: Submit the Review

Once you have formed an opinion, submit the review through the GitHub CLI.

Approve

```
gh pr review 42 --approve --body "Looks good. Clean implementation."
```

Request Changes

```
gh pr review 42 --request-changes --body "The error handling in auth.ts needs a try
```

Comment Only

```
gh pr review 42 --comment --body "A few questions about the approach."
```

Adding Inline Comments

For file-specific comments, use the GitHub CLI API directly:

```
gh api repos/OWNER/REPO/pulls/42/comments \
  --method POST \
  -f body="This function could be simplified with a reduce." \
  -f path="src/utils/transform.ts" \
  -F line=25 \
  -f side="RIGHT" \
  -f commit_id="$(git rev-parse HEAD)"
```

For most reviews, the overall comment is sufficient. Inline comments are reserved for specific line-level feedback.

Step 7: Merge When Approved

If you are the one merging after approval:

```
gh pr merge 42
```

This presents an interactive prompt asking which merge strategy to use:

- **Create a merge commit** – preserves all commits
- **Rebase and merge** – linear history, replays commits
- **Squash and merge** – condenses to a single commit

Or specify directly:

```
gh pr merge 42 --squash --delete-branch
```

The `--delete-branch` flag cleans up the remote branch after merging.

Reviewing Multiple PRs

For teams with many open PRs, the `gh dash` extension (installed via chezmoi's run-once script) provides a dashboard view:

```
gh dash
```

This shows a TUI with sections for PRs you authored, PRs assigned to you, and PRs requesting your review. You can select a PR and jump directly to checkout or review.

The Full Sequence

A complete code review flow:

```
# 1. See what needs review
gh pr list

# 2. Read the PR description
gh pr view 42

# 3. Check out the code
gh pr checkout 42

# 4. View the full diff
git diff main...HEAD

# 5. Check file-by-file with stats
git diff main...HEAD --stat

# 6. Open Neovim for deep exploration
nvim .
# Use Space+fg to grep, gd to jump to definitions

# 7. Use lazygit for commit-by-commit review
lazygit

# 8. Submit the review
gh pr review 42 --approve --body "Ship it."

# 9. Merge if ready
gh pr merge 42 --squash --delete-branch

# 10. Return to main
git checkout main && git pull
```

Troubleshooting

Problem	Solution
gh pr checkout fails	Run gh auth login to authenticate
Delta not showing colors	Check ~/.gitconfig for [core] pager = delta
Neovim LSP not working on PR branch	Run npm install or bun install first
git diff main...HEAD shows nothing	Check branch name: might be master not main
PR has merge conflicts	Resolve in Neovim, then git add and commit

Workflow: Debugging with AI

Three panes, two AI tools, one problem – using Claude Code and 99.nvim to diagnose and fix bugs.

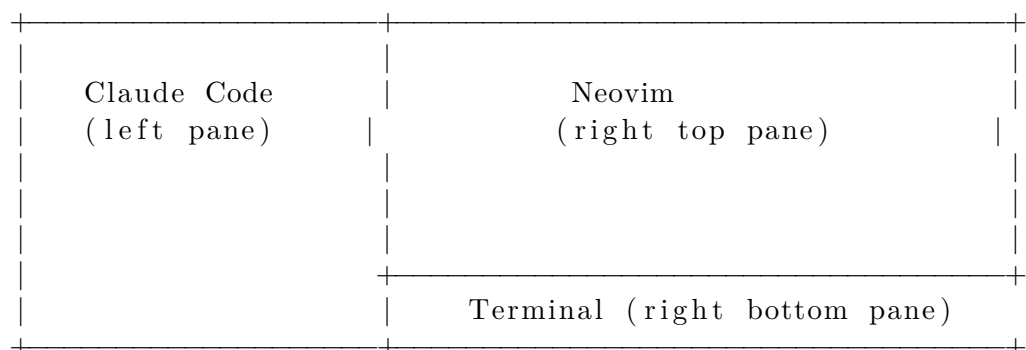
Overview

The IDE layout in this environment places two AI tools within reach of the editor at all times. Claude Code runs in a dedicated terminal pane on the left side of the screen, providing conversational AI with full context awareness. The 99.nvim plugin inside Neovim provides visual-mode AI assistance for targeted code analysis. The combination is powerful: use 99.nvim for quick, focused questions about specific code blocks, and use Claude Code for broader context, multi-file reasoning, and complex debugging.

This chapter walks through a typical debugging session that leverages both tools alongside the standard terminal workflow.

The IDE Layout

The debugging workflow assumes the three-pane IDE layout created by the `ide()` function:



Each pane serves a specific role in debugging:

- **Claude Code (left):** Long-form reasoning, multi-file analysis, suggesting fixes, explaining unfamiliar code

- **Neovim (right top):** Reading and editing code, navigating to definitions, viewing diagnostics
- **Terminal (right bottom):** Reproducing the bug, running tests, checking logs, verifying fixes

Navigation between panes uses vim-tmux-navigator:

Key	Action
Ctrl+h	Move to the left pane
Ctrl+j	Move to the pane below
Ctrl+k	Move to the pane above
Ctrl+l	Move to the right pane

Step 1: Reproduce the Bug

Start in the terminal pane (Ctrl+j from Neovim to move down). Run the failing command, test, or request:

```
bun test src/auth.test.ts
```

Or reproduce a runtime error:

```
bun dev
# Navigate to the failing route in the browser
# Observe the error in the terminal output
```

Read the error message carefully. Note the file names, line numbers, and stack trace. This information guides every subsequent step.

Step 2: Navigate to the Problem in Neovim

Move to the Neovim pane (Ctrl+k to move up). Navigate to the file mentioned in the error:

```
Space+ff → type the filename → Enter
```

Or jump directly:

```
:e src/services/auth.ts
```

Once in the file, navigate to the relevant line:

```
42G      " Go to line 42
```

Use LSP diagnostics to see if the editor already flags the issue:

Key	Action
Space+cd	Show diagnostics for the current line
]d	Jump to the next diagnostic
[d	Jump to the previous diagnostic
K	Show hover info for the symbol under cursor
gd	Go to the definition of a function/type
gr	Find all references to a symbol

Often, the LSP catches type errors, undefined variables, or import issues that explain the bug directly.

Step 3: Ask 99.nvim About Specific Code

When you have identified a suspicious code block but do not understand why it fails, use 99.nvim's visual mode integration.

1. Enter visual mode (v or V for line-wise)
2. Select the relevant code block
3. Press leader+9v (Space, then 9, then v)

This sends the selected code to the 99.nvim AI provider (Claude Code) with the visual context. A prompt window opens where you can type your question:

Why would this function return undefined when the user object exists?

The AI response appears in a Neovim buffer, formatted with syntax highlighting. This is ideal for targeted questions about a specific function, block, or expression.

99.nvim Keybindings

Key	Mode	Action
Space+9v	Visual	Send selection to AI with a prompt
Space+mm	Normal	Select the AI model
Space+9s	Normal	Stop all running AI requests

Step 4: Ask Claude Code for Broader Context

For questions that span multiple files or require understanding the architecture, switch to the Claude Code pane (Ctrl+h to move left).

Claude Code runs in the terminal and has access to the full project context. You can ask questions like:

The auth middleware is returning 401 for valid tokens.
I see the token validation in `src/services/auth.ts` calls `verifyToken()` on line 42, but the function seems correct.

Can you look at how the token is being passed from the request headers in `src/middleware/auth.ts` and trace the flow to find where it breaks?

Claude Code can read files, search the codebase, and reason across multiple modules. It excels at:

- **Tracing data flow** across files and functions
 - **Finding missing pieces** like unset environment variables or missing middleware
 - **Suggesting fixes** with full awareness of the codebase structure
 - **Explaining unfamiliar code** that was written by someone else
-

Step 5: Navigate Between Panes Seamlessly

The debugging loop involves rapid switching between panes:

1. **Terminal:** See the error, run a test
2. **Neovim:** Read the code, check diagnostics
3. **Claude Code:** Ask about the broader context
4. **Neovim:** Read the file Claude Code suggested
5. **Terminal:** Test a hypothesis

Each switch is a single `Ctrl+h/j/k/l` keypress. There is no window switching, no `Alt+Tab`, no clicking. The three panes are always visible simultaneously, so you maintain visual context even while typing in a different pane.

Step 6: Apply the Fix

Once the bug is understood, make the fix in Neovim. The typical sequence:

1. Navigate to the file and line (`Space+ff`, then `42G`)
2. Enter insert mode (`i`) and make the change
3. Save (`:w` or the auto-save plugin handles it)
4. Move to the terminal pane (`Ctrl+j`)
5. Run the test again:

```
bun test src/auth.test.ts
```

If the test passes, the fix is confirmed.

Step 7: Verify and Commit

With the fix verified, commit the change:

lazygit

In lazygit:

1. Stage the changed files (Space on each file)
2. Commit (c, type the message, Enter)
3. Push (P)

Or from the command line:

```
git add src/services/auth.ts
git commit -m "Fix token validation: handle expired tokens in verifyToken"
git push
```

AI Debugging Patterns

Over time, certain patterns emerge for effective AI-assisted debugging.

Pattern: Rubber Duck with Claude Code

Describe the problem in natural language in the Claude Code pane. Often, the act of articulating the problem reveals the solution before the AI even responds. When it does respond, it confirms or corrects your reasoning.

Pattern: Targeted Selection with 99.nvim

Select a function in Neovim visual mode and ask: “What edge cases could cause this to fail?” The AI examines the specific code and identifies potential issues you missed.

Pattern: Stack Trace Analysis

Paste the full error stack trace into Claude Code and ask: “Trace this error back to the root cause in this project.” Claude Code can follow the chain of calls across files.

Pattern: Comparison

When a function works in one context but fails in another, select both call sites in Neovim and ask 99.nvim: “What is different about these two invocations that could explain why the second one fails?”

When to Use Which AI Tool

Situation	Tool	Reason
Quick question about a code block	99.nvim	Fastest, stays in the editor
Multi-file investigation	Claude Code	Has project-wide context
Need to read/search files first	Claude Code	Can browse the filesystem

Situation	Tool	Reason
Understanding a type error	99.nvim	Select the type and ask
Generating a fix or refactor	Claude Code	Better at multi-line code generation
Explaining someone else's code	Either	99.nvim for a function, Claude Code for a module

Troubleshooting

Problem	Solution
99.nvim not responding	Check Space+9s to stop stale requests
Claude Code lost context	Start a new conversation in the pane
Pane navigation not working	Ensure vim-tmux-navigator is installed
IDE layout panes wrong size	Run <code>ide .</code> again to recreate the layout
AI suggests wrong fix	Provide more context: paste the error, show the types

Appendix A: All Keybindings

Every keybinding in every tool, organized by context.

Tmux: Prefix Commands

The tmux prefix is Ctrl+b.

Session Management

Action	Keys
Sesh session picker	prefix + T
SessionX picker	prefix + o
Detach from session	prefix + d
Rename session	prefix + \$
List sessions	prefix + s

Window Management

Action	Keys
Create window	prefix + c
Next window	prefix + n
Previous window	prefix + p
Rename window	prefix + ,
Close window	prefix + &
Select window 1-9	prefix + 1–9
Move pane to new window	prefix + !

Pane Management

Action	Keys
Split horizontal	prefix + "
Split vertical	prefix + %

Action	Keys
Select pane (vim-style)	prefix + h/j/k/l
Zoom pane (toggle)	prefix + z
Close pane	prefix + x
Floating terminal (floax)	prefix + p

Copy Mode (vi-style)

Enter copy mode with prefix + [.

Action	Keys
Begin selection	v
Rectangle selection toggle	Ctrl+v
Copy selection and exit	y
Search forward	/
Search backward	?
Half page down	Ctrl+d
Half page up	Ctrl+u
Go to top	gg
Go to bottom	G

Plugin Commands

Action	Keys
Install plugins (TPM)	prefix + I
Save session (resurrect)	prefix + Ctrl+s
Restore session (resurrect)	prefix + Ctrl+r
Open URL (fzf-url)	prefix + u

Sesh Picker Sub-Commands

When the sesh fzf popup is open (prefix + T):

Key	Action
Ctrl+a	Show all sessions
Ctrl+t	Show tmux sessions only
Ctrl+g	Show config sessions only
Ctrl+x	Show zoxide directories
Ctrl+f	Find directories (fd search)
Ctrl+d	Kill the selected session
Tab	Move down in the list
Shift+Tab	Move up in the list

Tmux: No-Prefix Commands

These keybindings work without pressing the prefix key first.

Pane Navigation (Alt + Arrow)

Action	Keys
Pane left	Alt+Left
Pane right	Alt+Right
Pane up	Alt+Up
Pane down	Alt+Down

Window Navigation

Action	Keys
Previous window	Shift+Left
Next window	Shift+Right
Previous window (vim)	Alt+H
Next window (vim)	Alt+L

Vim-Tmux-Navigator (Seamless Pane/Split Navigation)

These work identically in both tmux panes and Neovim splits:

Action	Keys
Navigate left	Ctrl+h
Navigate down	Ctrl+j
Navigate up	Ctrl+k
Navigate right	Ctrl+l

Neovim: Leader Commands

The leader key is Space.

Files and Buffers

Action	Keys
Find files	Space+ff
Live grep	Space+fg
Recent files	Space+fr
Open buffers	Space+fb
File explorer (neo-tree)	Space+e

Action	Keys
Switch buffer	Space+bb
Delete buffer	Space+bd
Previous buffer	H
Next buffer	L

Search

Action	Keys
Grep across project	Space+sg
Search word under cursor	Space+sw
Document symbols	Space+ss
Workspace symbols	Space+sS
Search and replace	Space+sr
Search in current buffer	Space+ /
Search command history	Space+sc
Search keymaps	Space+sk
Search help tags	Space+sH
Search marks	Space+sm
Search registers	Space+s"
Resume last picker	Space+sR

Code Actions and Diagnostics

Action	Keys
Code action	Space+ca
Line diagnostics	Space+cd
Rename symbol	Space+cr
Format file	Space+cf
Open Mason (LSP installer)	Space+cm

Git (from Neovim)

Action	Keys
Git commits (Telescope)	Space+gc
Git status (Telescope)	Space+gs
Next git hunk	]h
Previous git hunk	[h

AI (99.nvim)

Action	Keys	Mode
Visual AI prompt	Space+9v	Visual
Select AI model	Space+mm	Normal
Stop all AI requests	Space+9s	Normal

Plugins

Action	Keys
Toggle database UI (dadbod)	Space+D
Paste image (img-clip)	Space+p
Open Lazy plugin manager	Space+l

Neovim: LSP Commands

These work in any buffer with an active language server.

Action	Keys
Go to definition	gd
Go to declaration	gD
Go to implementation	gi
Go to type definition	gy
Find all references	gr
Hover documentation	K
Signature help	Ctrl+k
Next diagnostic	]d
Previous diagnostic	[d
Next changed hunk	]c
Previous changed hunk	[c

Neovim: General Navigation

Action	Keys
Top of file	gg
Bottom of file	G
Go to line N	NG (e.g., 42G)
Half page down	Ctrl+d
Half page up	Ctrl+u
Search forward	/pattern
Search backward	?pattern

Action	Keys
Next search result	n
Previous search result	N
Jump to matching bracket	%
Flash jump (2-char)	s then 2 chars
Navigate splits	Ctrl+h/j/k/l

Aerospace: Main Mode

Action	Keys
Toggle tile layout	Alt+ /
Accordion layout	Alt+ ,
Focus left	Cmd+Shift+H
Focus down	Cmd+Shift+J
Focus up	Cmd+Shift+K
Focus right	Cmd+Shift+L
Move window left	Ctrl+Shift+H
Move window down	Ctrl+Shift+J
Move window up	Ctrl+Shift+K
Move window right	Ctrl+Shift+L
Fullscreen	Cmd+Shift+F
Focus back-and-forth	Alt+O
Resize shrink	Alt+Shift+-
Resize grow	Alt+Shift+=
Workspace 1	Alt+1
Workspace 2	Alt+2
Workspace 3	Alt+3
Workspace 4	Alt+4
Move to workspace 1	Alt+Shift+1
Move to workspace 2	Alt+Shift+2
Move to workspace 3	Alt+Shift+3
Move to workspace 4	Alt+Shift+4
Move workspace to monitor	Alt+Shift+Tab
Enter service mode	Alt+Shift+;

Aerospace: Service Mode

Entered with Alt+Shift+;. All commands return to main mode after execution.

Action	Keys
Reload config, exit	Esc
Flatten workspace tree	r
Toggle floating/tiling	f
Close all but current	Backspace
Join with left	Alt+Shift+H
Join with down	Alt+Shift+J
Join with up	Alt+Shift+K
Join with right	Alt+Shift+L

Yazi Navigation

Movement

Action	Keys
Move down	j
Move up	k
Enter directory / open	l or Enter
Go to parent directory	h
Go to top	gg
Go to bottom	G
Half page up	Ctrl+u
Half page down	Ctrl+d
History back	H
History forward	L

File Operations

Action	Keys
Toggle selection	Space
Select all	Ctrl+a
Visual mode	v
Copy (yank)	y
Cut	x
Paste	p
Delete (trash)	d
Permanent delete	D
Create file/directory	a
Rename	r
Open in editor	e
Open shell	s

Search and Find

Action	Keys
Search by name (fd)	s
Search by content (rg)	S
Filter	f
Find next	/
Find previous	?
Next match	n
Previous match	N
Jump via fzf	z
Jump via zoxide	Z

Goto Shortcuts

Action	Keys
Go to home	gh
Go to ~/.config	gc
Go to ~/Downloads	gd

Other

Action	Keys
Toggle hidden files	.
Shell command	;
Shell command (blocking)	:
New tab	t
Switch to tab 1-9	1–9
Previous / next tab	[/]
Task manager	w
Help	~ or F1
Quit (sync CWD)	q
Quit (no CWD sync)	Q

fzf Shell Bindings

Action	Keys
Search files	Ctrl+T
Search command history	Ctrl+R
Search directories (cd)	Alt+C

Inside an fzf Picker

Action	Keys
Move down	Ctrl+j or Down
Move up	Ctrl+k or Up
Select	Enter
Cancel	Esc or Ctrl+c
Toggle selection	Tab
Select all	Ctrl+a
Preview scroll down	Shift+Down
Preview scroll up	Shift+Up

Lazygit

Action	Keys
Switch panels	h/l or Tab
Move up/down	k/j
Stage file/hunk	Space
Commit	c
Push	P
Pull	p
Amend last commit	A
New branch	n
Checkout branch	Space
Force push	Shift+P
Interactive rebase	i
Search	/
Open file in editor	e
View diff	Enter
Toggle whitespace in diff	w
Quit	q

Atuin (Shell History)

Action	Keys
Search history	Ctrl+R
Move down in results	Ctrl+n or Down
Move up in results	Ctrl+p or Up
Select entry	Enter
Cancel search	Esc

Appendix B: All Aliases and Functions

Every shell alias and function from `dot_aliases.zsh`, with usage examples and explanations.

All aliases and functions are defined in `~/aliases.zsh`, which `chezmoi` manages as `dot_aliases.zsh` in the source directory. The file is sourced from `~/zshrc` with:

```
[ -f ~/.aliases.zsh ] && source ~/.aliases.zsh
```

Editor Aliases

These aliases support the multi-Neovim configuration setup, where different `NVIM_APPNAME` values load different Neovim configurations from `~/config/<appname>/`.

Alias	Expansion	Purpose
<code>vim</code>	<code>nvim</code>	Muscle memory compatibility
<code>v</code>	<code>nvim</code>	Short alias for Neovim
<code>vtest</code>	<code>NVIM_APPNAME=nvim-test nvim</code>	Launch test Neovim configuration
<code>vkick</code>	<code>NVIM_APPNAME=nvim-kickstart nvim</code>	Launch Kickstart Neovim configuration
<code>vlazy</code>	<code>NVIM_APPNAME=nvim-lazyvim nvim</code>	Launch LazyVim configuration
<code>vmin</code>	<code>NVIM_APPNAME=nvim-minimal nvim</code>	Launch minimal Neovim configuration

Usage

```
v .                # Open Neovim in current directory
v src/index.ts     # Open a specific file
vmin               # Open Neovim with minimal config (for testing)
vlazy              # Open Neovim with LazyVim config
```

The multi-config setup is useful for testing plugins, experimenting with configurations, or using a stripped-down editor when the full config is not needed.

Miscellaneous Aliases

Alias	Expansion	Purpose
<code>c</code>	<code>clear</code>	Clear the terminal screen
<code>cat</code>	<code>bat</code>	Replace cat with bat (syntax highlighting)
<code>manual</code>	<code>bat ~/USER_MAN</code>	View personal user manual

Usage

```

c                                # Clear screen
cat package.json              # View file with syntax highlighting and line numbers
manual                          # Read the user manual

```

The `cat=bat` alias means every time you would use `cat` to view a file, you get syntax highlighting, line numbers, and git integration automatically. To use the real `cat`, escape the alias: `\cat file.txt`.

File Listing Aliases (eza)

Alias	Expansion	Purpose
<code>ls</code>	<code>eza</code>	Replace ls with eza
<code>l</code>	<code>eza -l --icons --git -a</code>	Long list, all files, icons, git status
<code>lt</code>	<code>eza --tree --level=2 --long --icons --git</code>	Tree view (2 levels deep) with details
<code>ltree</code>	<code>eza --tree --level=2 --icons --git</code>	Tree view (2 levels deep) without long

Usage

```

ls                                # Simple file listing with eza
l                                # Detailed listing with icons and git markers
lt                                # Two-level tree with file details
ltree                             # Two-level tree (compact)
lt src/                          # Tree view of a specific directory

```

The `l` alias is the workhorse. It shows file permissions and sizes, git status per file (modified, untracked, staged), file type icons (requires Nerd Font), and hidden files (the `-a` flag).

Navigation Shortcuts

Alias	Expansion	Purpose
<code>..</code>	<code>cd ..</code>	Go up one directory
<code>...</code>	<code>cd ../..</code>	Go up two directories
<code>....</code>	<code>cd ../../..</code>	Go up three directories

Alias	Expansion	Purpose
....	cd ../../../../	Go up four directories
—	cd —	Go to previous directory

Usage

```

..                # Go to parent directory
...              # Go to grandparent directory
—               # Toggle between last two directories

```

These are simple but heavily used. Combined with zoxide's cd override, navigation rarely requires typing full paths.

Navigation Functions

cx() – Change Directory and List

```
cx() { cd "$@" && l; }
```

Changes to a directory and immediately lists its contents with the detailed l alias.

```

cx ~/Projects      # cd to Projects and show file listing
cx src/components  # cd and list in one step

```

fcd() – Fuzzy Directory Picker

```
fcd() { cd "$(fd_type_d_hidden_follow_exclude .git | fzf)" && l; }
```

Uses fd to find all directories, pipes them through fzf for fuzzy selection, then changes to the selected directory and lists contents.

```

fcd                # Launch fuzzy directory picker
# Type partial name -> select -> Enter

```

f() – Fuzzy File to Clipboard

```
f() { echo "$(fd_type_f_hidden_follow_exclude .git | fzf)" | pbcopy; }
```

Finds a file with fuzzy search and copies its path to the macOS clipboard.

```

f                # Pick a file, path goes to clipboard
# Now Cmd+V to paste the path anywhere

```

```
fv() { nvim "$(\fd_type\fd_hidden\fd_follow\fd_exclude.git\fdzf)"; }
```

[illegible]

ide() – Tmuxinator IDE Layout

Creates a three-pane tmux layout for development:

Claude Code (20% width)	Neovim (80% width)
	Terminal (20% height)

The function starts a tmuxinator template, resizes panes to the correct proportions, focuses the Neovim pane, renames the window, and attaches to the session.

tab() – Rename Tmux Window or Terminal Tab

```
tab() {
    if [ -n "$TMUX" ]; then
        tmux rename-window "$1"
    else
        echo -ne "\033]1;$1\007"
```

```

    fi
}

tab "api-server"      # Rename current tmux window to "api-server"
tab "frontend"        # Rename to "frontend"

```

Inside tmux, this renames the tmux window. Outside tmux, it sets the terminal tab title using an escape sequence.

Yazi Wrapper Function

y() – Yazi with CWD Sync

```

function y() {
    local tmp="$(mktemp -t "yazi-cwd.XXXXXX")" cwd
    command yazi "$@" --cwd-file="$tmp"
    IFS= read -r -d ' ' cwd < "$tmp"
    [ "$cwd" != "$PWD" ] && [ -d "$cwd" ] && builtin cd -- "$cwd"
    rm -f -- "$tmp"
}

```

Launches Yazi file manager and syncs the working directory when you quit. If you navigate to a different directory inside Yazi and press q, the shell's working directory changes to match.

```

y                                # Open Yazi in current directory
y ~/Downloads                    # Open Yazi in Downloads
# Navigate with h/j/k/l, press q to quit and stay in that directory

```

Without this wrapper, quitting Yazi would return to the original directory regardless of where you navigated inside it.

Networking Aliases

Alias	Expansion	Purpose
ip	dig +short myip.opendns.com @resolver1.opendns.com	Show public IP address
localip	ipconfig getifaddr en0	Show local network IP
flush	dscacheutil -flushcache && killall -HUP mDNSResponder	Flush DNS cache

Usage

```

ip                                # Print your public IP (e.g., 203.0.113.42)
localip                          # Print your local IP (e.g., 192.168.1.100)
flush                            # Clear DNS cache (useful after /etc/hosts changes)

```

Cleanup and Maintenance Aliases

Alias	Expansion	Purpose
cleanup	<code>find . -type f -name '*.DS_Store' -ls -delete</code>	Delete .DS_Store files
emptytrash	Force-remove all Trash directories and ASL logs	Empty all trashes
update	<code>sudo softwareupdate -i -a; brew update; brew upgrade; brew update --force</code>	Update everything

Usage

```
cleanup          # Remove .DS_Store files from current tree
emptytrash       # Nuclear option: force-empty all trashes
update           # Update everything: macOS, Homebrew, all packages
```

The emptytrash alias runs: `sudo rm -rfv /Volumes/*/Trash;` `sudo rm -rfv ~/.Trash;` `sudo rm -rfv /private/var`

The update alias runs macOS software updates, then refreshes the Homebrew formula index, upgrades all installed packages, and cleans up old versions.

macOS Toggle Aliases

Alias	Purpose
show	Show hidden files in Finder
hide	Hide hidden files in Finder
showdesktop	Show desktop icons
hidedesktop	Hide desktop icons (clean desktop for presentations)
stfu	Mute system volume
pumpitup	Set system volume to 100%
afk	Lock screen (put display to sleep)

Expansions

```
show          # defaults write com.apple.finder AppleShowAllFiles -bool true && killall Finder
hide          # defaults write com.apple.finder AppleShowAllFiles -bool false && killall Finder
showdesktop   # defaults write com.apple.finder CreateDesktop -bool true && killall Finder
hidedesktop   # defaults write com.apple.finder CreateDesktop -bool false && killall Finder
stfu          # osascript -e 'set volume output muted true'
pumpitup      # osascript -e 'set volume output volume 100'
afk           # pmset displaysleepnow
```

Utility Functions

mkd() – Make Directory and Enter

```
mkd() { mkdir -p "$@" && cd "$_"; }
```

```
mkd ~/Projects/new-app/src/components
# Creates the full path and cds into the deepest directory
```

server() – Quick HTTP Server

```
server() { open "http://localhost:${1:-8000}" && python3 -m http.server "${1:-8000}"

server                                # Serve current directory on port 8000
server 3000                           # Serve on port 3000
```

Opens the browser automatically and starts a Python HTTP server serving the current directory.

fs() – File Sizes

```
fs() { du -sh — "$@" | sort -h; }

fs *                                # Show sizes of all items in current directory, sorted
fs node_modules dist               # Compare sizes of specific directories
```

gz() – Gzip Comparison

```
gz() {
  local orig=$(wc -c < "$1")
  local gzipped=$(gzip -c "$1" | wc -c)
  echo "orig:␣...␣/␣gzip:␣...␣/␣ratio:␣...%"
}

gz bundle.js                       # Show original size, gzipped size, and compression ratio
# orig: 245K / gzip: 58K / ratio: 23.67%
```

Useful for checking JavaScript bundle sizes and compression efficiency.

Complete Alias Quick-Scan Table

Alias	What It Does
vim / v	Opens Neovim
vtest	Neovim with test config
vkick	Neovim with Kickstart config
vlazy	Neovim with LazyVim config
vmin	Neovim with minimal config
c	Clears the terminal
cat	bat (syntax-highlighted file viewer)
ls	eza (modern file listing)
l	Detailed eza listing with icons and git

Alias	What It Does
lt	Two-level tree with details
ltree	Two-level tree (compact)
..	cd up one level
...	cd up two levels
....	cd up three levels
.....	cd up four levels
—	cd to previous directory
ip	Show public IP
localip	Show local IP
flush	Flush DNS cache
cleanup	Delete .DS_Store files
emptytrash	Force-empty all Trash
update	Update macOS + Homebrew
show / hide	Toggle hidden Finder files
showdesktop	Show desktop icons
hidedesktop	Hide desktop icons
stfu	Mute volume
pumpitup	Max volume
afk	Lock screen
manual	View ~/USER_MANUAL.md

Complete Function Quick-Scan Table

Function	Signature	What It Does
cx	cx [dir]	cd + list contents
fcd	fcd	Fuzzy directory picker + cd
f	f	Fuzzy file picker to clipboard
fv	fv	Fuzzy file picker to Neovim
ide	ide [dir]	Three-pane tmux IDE layout
tab	tab [name]	Rename tmux window or terminal tab
y	y [dir]	Yazi file manager with CWD sync
mkd	mkd [path]	mkdir -p + cd
server	server [port]	Python HTTP server + open browser
fs	fs [files ...]	Show file/directory sizes sorted
gz	gz [file]	Show gzip compression ratio

Oh My Zsh Git Aliases

These come from the Oh My Zsh git plugin (enabled in .zshrc), not from ~/.aliases.zsh:

Alias	Expansion	Alias	Expansion
gst	git status	gp	git push
ga	git add	gl	git pull
gaa	git add --all	gf	git fetch
gc	git commit	gco	git checkout
gcam	git commit -am	gcb	git checkout -b
gb	git branch	gm	git merge
gbd	git branch -d	gd	git diff
glo	git log --oneline	gds	git diff --staged
glg	git log --graph	gsta	git stash
grb	git rebase	gstp	git stash pop

Run `alias | grep git` to see the full list (there are many more).

Environment Variables

Set in `~/zshrc` (managed by `dot_zshrc.tmpl`):

Variable	Value	Purpose
EDITOR	nvim (local) / vim (SSH)	Default editor for git, etc.
MANPAGER	nvim +Man!	Man pages render in Neovim
NVM_DIR	\$HOME/.nvm	NVM installation directory
BUN_INSTALL	\$HOME/.bun	Bun installation directory
ZSH	\$HOME/.oh-my-zsh	Oh My Zsh installation

Zsh Plugins

Enabled in `.zshrc` via the `plugins` array:

Plugin	Purpose
git	Git aliases and functions
zsh-syntax-highlighting	Real-time command syntax highlighting
zsh-autosuggestions	Fish-like autosuggestions from history

Appendix C: Quick Reference Card

One-page task-oriented reference – find what you need by what you want to do.

Finding Things

Task	Command / Keys
Find a file by name	Space+ff (Neovim) or Ctrl+T (shell fzf)
Find text in files	Space+fg (Neovim) or rg "pattern" (shell)
Find text live (as you type)	Space+sg (Neovim)
Find word under cursor	Space+sw (Neovim)
Find a directory	fd (fuzzy) or fd --type d name
Find a file, copy path	f (fuzzy file to clipboard)
Find a file, open in editor	fv (fuzzy file to Neovim)
Find in command history	Ctrl+R (atuin search)
Find open buffers	Space+fb (Neovim)
Find recent files	Space+fr (Neovim)
Find hidden files too	fd --hidden or l (eza -a)
Find by file content (structural)	ast-grep -p 'pattern'
Search and replace in project	Space+sr (Neovim, grug-far)
Quick man page summary	tldr command

Editing

Task	Command / Keys
Open Neovim	v or vim or nvim
Open specific file	v filename or Space+ff inside Neovim
File explorer	Space+e (neo-tree toggle)
Create new file in explorer	a (in neo-tree)
Go to definition	gd
Go to declaration	gD
Find all references	gr

Task	Command / Keys
Rename symbol	Space+cr
Code action / quick fix	Space+ca
Format file	Space+cf
View line diagnostics	Space+cd
Next / previous diagnostic	]d / [d
Hover documentation	K
Select and ask AI	Visual select, then Space+9v
Select AI model	Space+mm
Stop AI requests	Space+9s
Open minimal Neovim	vmin
Open LazyVim config	vlazy
Flash jump	s then type 2 characters
Switch buffers	H / L or Space+bb
Delete buffer	Space+bd

Navigating

Task	Command / Keys
Jump to a directory	cd partial—name (zoxide)
Go up one directory	..
Go up two directories	...
Go up three directories	
Toggle previous directory	— or cd —
cd and list contents	cx dirname
Fuzzy pick a directory	fcd
Change dir via fzf	Alt+C (shell binding)
Browse files visually	y (yazi, syncs CWD on quit)
Switch tmux session	prefix+T (sesh) or prefix+o (sessionx)
Switch tmux window	Shift+Left/Right or Alt+H/L
Switch tmux pane	Ctrl+h/j/k/l (vim-tmux-navigator)
Switch workspace	Alt+1/2/3/4 (Aerospace)
Move window to workspace	Alt+Shift+1/2/3/4 (Aerospace)
Focus window in workspace	Cmd+Shift+H/J/K/L (Aerospace)
Move window in workspace	Ctrl+Shift+H/J/K/L (Aerospace)
Toggle focus back-and-forth	Alt+O (Aerospace)

Git

Task	Command / Keys
Open git TUI	lazygit
Status	git status or gst
Stage file	git add file or Space in lazygit
Stage all	gaa (git add -all)
Commit	gc -m "message" or c in lazygit
Push	gp or P in lazygit
Pull	gl or p in lazygit
View diff (unstaged)	gd (git diff, rendered by delta)
View diff (staged)	gds (git diff -staged)
View PR diff	git diff main...HEAD
Diff with file stats	git diff main...HEAD --stat
Interactive diff navigation	git diff main...HEAD \ diffnav
Create branch	gcb branch-name (git checkout -b)
Switch branch	gco branch-name (git checkout)
Stash / pop	gsta / gstp
Log (one line per commit)	glo (git log -oneline)
Log (graph view)	glg (git log -graph)
List open PRs	gh pr list
Checkout a PR	gh pr checkout NUMBER
View a PR	gh pr view NUMBER
Create a PR	gh pr create --fill
Review a PR (approve)	gh pr review NUMBER --approve
Review a PR (request changes)	gh pr review NUMBER --request-changes
Merge a PR	gh pr merge NUMBER --squash --delete-branch
Create repo from local dir	gh repo create name --private --source=. --push
PR dashboard	gh dash

Terminal Management (tmux)

Task	Command / Keys
Launch IDE layout	ide . or ide ~/project
New window	prefix +c
Split horizontal	prefix +"
Split vertical	prefix +%
Navigate panes	Ctrl+h/j/k/l
Zoom current pane	prefix +z
Close pane	prefix +x
Floating terminal	prefix +p (floax)
Rename window	tab name or prefix +,
Detach from tmux	prefix +d
Session picker (sesh)	prefix +T
Session picker (sessionx)	prefix +o

Task	Command / Keys
List sessions	prefix + s
Next window	Shift + Right or Alt + L
Previous window	Shift + Left or Alt + H
Enter copy mode	prefix + [
Select in copy mode	v (then y to copy)
Save session (resurrect)	prefix + Ctrl + s
Restore session (resurrect)	prefix + Ctrl + r
Install plugins (TPM)	prefix + I

System

Task	Command / Keys
Public IP address	ip
Local IP address	localip
Flush DNS cache	flush
Update everything	update (macOS + Homebrew)
System info	fastfetch
System monitor	bttop
Mute volume	stfu
Max volume	pumpitup
Lock screen	afk
Show hidden files (Finder)	show
Hide hidden files (Finder)	hide
Hide desktop icons	hidedesktop
Show desktop icons	showdesktop
Delete .DS_Store files	cleanup
Empty all trashes	emptytrash
View man page (in Neovim)	man command (uses nvim as MANPAGER)
View user manual	manual

File Operations

Task	Command / Keys
List files (simple)	ls (aliased to eza)
List files (detailed + icons)	l
Tree view (2 levels, detailed)	lt
Tree view (2 levels, compact)	ltree
View file with syntax highlight	cat file (aliased to bat)
File / directory sizes (sorted)	fs * or fs dir1 dir2

Task	Command / Keys
Gzip compression ratio	gz filename
Create dir and cd into it	mkd path/to/dir
Quick HTTP server	server (port 8000) or server 3000
Browse files in TUI	y (yazi with CWD sync)
Copy file path to clipboard	f (fuzzy picker)
Open file in Neovim	fv (fuzzy picker)
Download file	wget URL or aria2c URL (multi-connection)
Download video	yt-dlp URL
Convert document format	pandoc input.md -o output.pdf
Process image	magick input.png -resize 50% output.png

Package Management

Task	Command
Install all Homebrew packages	brew bundle
Install base tier only	brew bundle --file=brewfiles/00-base.Brewfile
Check missing packages	brew bundle check
Clean unused packages	brew bundle cleanup --force
Search for a formula	brew search name
Install a formula	brew install name
Install a cask	brew install --cask name
Update Homebrew	brew update && brew upgrade

Runtime Versions

Task	Command
Switch Node version	nvm use 22
Install Node version	nvm install 22
List installed Node versions	nvm ls
Current Node version	nvm current
Install Bun packages	bun install
Add Bun dependency	bun add package
Run Bun script	bun run dev or bun dev
Execute without install	bunx package-name
Pin runtime with mise	mise use python@3.12
See active mise versions	mise current
List all mise versions	mise ls

Dotfiles (chezmoi)

Task	Command
Go to source directory	chezmoi cd
Edit a managed file	chezmoi edit ~/.zshrc
Preview pending changes	chezmoi diff
Apply source to target	chezmoi apply
Pull and apply from remote	chezmoi update
List managed files	chezmoi managed
Add a file to chezmoi	chezmoi add ~/.gitconfig
Check for problems	chezmoi doctor
View template data	chezmoi data
See source path of target	chezmoi source-path ~/.zshrc

Workspace Layout (Aerospace)

Workspace	Switch	Contents
1	Alt+1	Browsers (Firefox Developer Edition)
2	Alt+2	Terminals (iTerm2)
3	Alt+3	Communication (Slack, Discord, Telegram)
4	Alt+4	Media and other (Spotify)

IDE Pane Layout

Claude Code (Ctrl+h)	Neovim (Ctrl+l)
	Terminal (Ctrl+j)

Launch: `ide .` or `ide ~/Projects/my-app`